Git e GitHub

Do Iniciante ao Profissional com VS Code

Haroldo Capucci

2ª Edição — Revisada e Ampliada

Capítulo 1

Introdução ao Controle de Versão

Páginas estimadas: 22

Tempo de leitura: 50–70 minutos

Objetivos do capítulo:

- Entender o que é controle de versão e por que ele é essencial
- Diferenciar sistemas centralizados e distribuídos
- Conhecer a história do Git e seu criador
- Mapear o ecossistema Git: GitHub, GitLab, Bitbucket
- Aprender os conceitos fundamentais: repositório, commit, branch, merge

---

1.1 O que é Controle de Versão?

Imagine que você está escrevendo um trabalho de conclusão de curso. Você cria o arquivo `tcc.docx`. Dias depois, faz alterações e salva como `tcc_v2.docx`. Mais tarde, `tcc_final.docx`. Depois da revisão do orientador, vira `tcc_final_revisado.docx`. Em poucas semanas, você tem uma pasta cheia de arquivos com nomes absurdos e não sabe mais qual é a versão verdadeira.

Esse cenário caótico é o que o **controle de versão** elimina.

1.1.1 Definição

Controle de versão é um sistema que **registra as alterações feitas em um arquivo ou conjunto de arquivos ao longo do tempo**, permitindo que você recupere versões específicas mais tarde.

Pense nele como uma **máquina do tempo** para seus projetos. Você pode:

- Voltar a qualquer ponto anterior
- Comparar mudanças entre versões
- Saber quem fez cada alteração e quando
- Trabalhar em paralelo com outras pessoas sem sobrescrever o trabalho alheio

1.1.2 Benefícios Imediatos

Sem Controle de Versão	Com Controle de Versão
Pastas cheias de `final_final_mesmo.docx`	Um único diretório organizado
Não sabe quem alterou o quê	Histórico completo de autoria
Medo de apagar algo importante	Tudo é recuperável
Conflitos ao trabalhar em equipe	Merge estruturado e rastreável
Backup manual e esquecível	Histórico automático e contínuo

1.2 Breve História do Controle de Versão

1.2.1 A Era dos Sistemas Locais

Os primeiros sistemas de controle de versão, como o **RCS (Revision Control System)**, criado nos anos 1980, funcionavam localmente. Eles guardavam as diferenças entre versões de arquivos em um banco de dados local. Funcionavam bem para trabalho individual, mas eram inúteis para colaboração em equipe.

1.2.2 Sistemas Centralizados (CVCS)

Com a necessidade de colaboração, surgiram os sistemas centralizados como **CVS** e **SVN (Subversion)**. Nesse modelo, existe um único servidor que contém todos os arquivos versionados, e os clientes conectam-se a ele para obter ou enviar alterações.

Vantagens:

- Visibilidade total do que todos estão fazendo
- Controle de acesso granular
- Administração simplificada

Desvantagens:

- Ponto único de falha: se o servidor cair, ninguém trabalha
- Operações lentas por depender de rede
- Sem histórico local: impossível trabalhar offline

1.2.3 Sistemas Distribuídos (DVCS)

Os sistemas distribuídos, como **Git** e Mercurial, resolvem os problemas dos centralizados. Cada desenvolvedor possui uma cópia completa do repositório, incluindo todo o histórico.

Vantagens:

- Trabalho offline completo
- Velocidade: a maioria das operações é local
- Redundância: cada clone é um backup completo
- Flexibilidade de fluxos de trabalho

1.3 A História do Git

1.3.1 O Problema que Gerou o Git

Em 2002, o projeto do kernel Linux passou a usar o **BitKeeper**, um sistema de controle de versão distribuído proprietário. Em 2005, a empresa que desenvolvia o BitKeeper revogou a licença gratuita para a comunidade open source.

Linus Torvalds, criador do Linux, decidiu então criar seu próprio sistema. Seus objetivos eram claros:

- Velocidade extrema
- Design simples
- Suporte robusto a desenvolvimento não-linear (milhares de branches paralelos)
- Totalmente distribuído
- Capaz de lidar com projetos grandes como o kernel Linux

Em apenas **algumas semanas**, Linus escreveu o Git. O primeiro commit foi feito em **7 de abril de 2005**.

1.3.2 Git Hoje

Hoje, o Git é de longe o sistema de controle de versão mais utilizado no mundo. Segundo pesquisas recentes, mais de **93% dos desenvolvedores** usam Git como principal ferramenta de versionamento.

1.4 O Ecossistema Git

O Git em si é uma ferramenta de linha de comando. Em torno dele, cresceu um ecossistema rico de serviços e plataformas.

1.4.1 Plataformas de Hospedagem

GitHub

- A maior plataforma de hospedagem de código do mundo
- Adquirida pela Microsoft em 2018
- Oferece: repositórios públicos e privados, Issues, Pull Requests, Actions (CI/CD), Pages, Copilot

GitLab

- Foco em DevOps completo, pode ser instalado on-premise
- Muito usado em empresas que precisam de controle total da infraestrutura

Bitbucket

- Da Atlassian, integra-se nativamente com Jira e Trello
- Popular em ambientes corporativos

1.4.2 Interfaces Gráficas

Embora a linha de comando seja a forma mais poderosa de usar Git, existem interfaces gráficas populares:

- **VS Code** — integração nativa (foco deste livro)
- **GitKraken** — interface visual intuitiva
- **SourceTree** — gratuito, da Atlassian
- **GitHub Desktop** — simples e direto ao ponto

1.5 Conceitos Fundamentais

Antes de usar o Git, é essencial compreender alguns termos que aparecerão constantemente ao longo deste livro.

1.5.1 Repositório (Repository)

Um **repositório** (ou "repo") é o banco de dados que armazena todo o histórico do seu projeto. Ele contém todos os arquivos e o registro completo de cada alteração já feita.

Existem dois tipos:

- **Repositório local**: fica na sua máquina
- **Repositório remoto**: fica em um servidor (GitHub, GitLab etc.)

1.5.2 Commit

Um **commit** é um registro permanente de um conjunto de alterações. Pense nele como uma "foto" do seu projeto em um determinado momento. Cada commit possui:

- Um identificador único (hash SHA-1, ex: `a3f4b2c`)
- Uma mensagem descritiva
- O autor e a data
- A referência ao commit anterior (o "pai")

1.5.3 Branch

Uma **branch** (ramificação) é uma linha independente de desenvolvimento. Por padrão, todo repositório começa com uma branch chamada `main` (ou `master` em projetos mais antigos).

Branches permitem que você trabalhe em uma funcionalidade nova sem afetar o código principal. Quando o trabalho está pronto, você **mescla** (merge) a branch de volta.

1.5.4 Merge

Merge é o processo de integrar as alterações de uma branch em outra. O Git combina automaticamente mudanças que não conflitam. Quando há conflito (duas pessoas editaram a mesma linha), ele pede que você resolva manualmente.

1.5.5 Clone, Push e Pull

- **Clone**: copia um repositório remoto para sua máquina
- **Push**: envia seus commits locais para o repositório remoto
- **Pull**: baixa e integra as alterações do repositório remoto na sua cópia local

---

1.6 Por que Usar o VS Code com Git?

O **Visual Studio Code** é hoje o editor de código mais popular do mundo. Ele possui integração nativa com Git, o que significa que você não precisa instalar nada extra para ter uma interface gráfica poderosa.

Vantagens de usar o VS Code com Git:

- Visualização de diferenças (diff) lado a lado
- Interface para staging, commit e push sem sair do editor
- Extensões como **GitLens** que enriquecem o histórico visual
- Terminal integrado para quando a linha de comando for necessária

---

1.7 Resumo do Capítulo

Neste capítulo você aprendeu:

- O que é controle de versão e qual problema ele resolve
- A evolução dos sistemas: locais → centralizados → distribuídos
- Como e por que o Git foi criado por Linus Torvalds em 2005
- As principais plataformas do ecossistema: GitHub, GitLab, Bitbucket
- Os conceitos essenciais: repositório, commit, branch, merge, clone, push e pull

No próximo capítulo, vamos instalar o Git e o VS Code e fazer toda a configuração inicial para começar a trabalhar.

1.8 Exercícios

1. Qual problema o controle de versão resolve no trabalho do dia a dia?
2. Cite duas diferenças entre sistemas centralizados e distribuídos.
3. Por que Linus Torvalds criou o Git em 2005?
4. Qual a diferença entre `push` e `pull`?
5. O que é um commit? Quais informações ele carrega?

1.9 Referências e Leitura Recomendada

- Pro Git Book (2ª edição) — Capítulo 1: Getting Started (disponível gratuitamente em git-scm.com)
- Documentação oficial do Git: https://git-scm.com/doc
- GitHub Docs: "About Git" — https://docs.github.com/en/get-started/using-git/about-git

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 2: Instalação e Configuração Inicial

Páginas estimadas: 28

Tempo de leitura: 70–90 minutos

Objetivos do capítulo:

- Instalar o Git nos sistemas Windows, macOS e Linux
- Instalar e configurar o Visual Studio Code
- Realizar as configurações iniciais do Git (identidade, editor, etc.)
- Compreender o Git Bash e o terminal integrado do VS Code
- Configurar ferramentas de diff e merge

2.1 Introdução

Antes de começar a usar o Git, é preciso preparar o ambiente de trabalho. Neste capítulo, você aprenderá a instalar o Git no seu sistema operacional, configurá-lo corretamente e integrá-lo ao Visual Studio Code, o editor que utilizaremos ao longo de todo o livro.

Uma boa configuração inicial evita dores de cabeça futuras e torna o fluxo de trabalho mais produtivo. Ao final deste capítulo, você terá um ambiente totalmente preparado para versionar seus projetos com Git e colaborar usando o GitHub.

2.2 Instalando o Git

2.2.1 Verificando se o Git já está instalado

Abra um terminal (Prompt de Comando, PowerShell, Terminal do Linux ou macOS) e digite:

```
git --version

Se o Git estiver instalado, a saída será algo como:

git version 2.40.0

Caso contrário, você receberá uma mensagem de comando não encontrado. Siga as
instruções abaixo para seu sistema operacional.

2.2.2 Instalação no Windows

Método 1: Instalador oficial

1. Acesse https://git-scm.com/download/win
2. O download do instalador será iniciado automaticamente (versão de 64
bits).
3. Execute o arquivo baixado.
4. Durante a instalação, preste atenção nas seguintes telas:
· Selecionar componentes: mantenha as opções padrão, mas recomenda■se marcar
"Git Bash Here" e "Git GUI Here" para facilitar o acesso.
· Escolher o editor padrão: selecione "Use Visual Studio Code as Git's
default editor" se já tiver o VS Code instalado (caso contrário, instale o VS
Code primeiro ou escolha outro editor como Nano ou Vim).
· Ajustar o PATH: escolha "Git from the command line and also from 3rd-party
software" (recomendado).
· Usar OpenSSL: mantenha a opção padrão.
· Configurar terminais: escolha "Use MinTTY" (terminal mais amigável).
· Opções extras: mantenha as padrões.
5. Conclua a instalação.

Método 2: via winget (Windows Package Manager)

Se você utiliza o Windows 10/11 com o gerenciador winget, abra o PowerShell
como administrador e execute:
```

winget install --id Git.Git -e --source winget

Método 3: via Chocolatey

Caso utilize o Chocolatey:

```
choco install git

Após a instalação, reinicie o terminal e confirme com git --version.

2.2.3 Instalação no macOS

Método 1: Instalador oficial

1. Acesse https://git-scm.com/download/mac
2. Baixe o instalador apropriado para sua versão do macOS.
3. Abra o arquivo .dmg e execute o instalador .pkg.
4. Siga as instruções.

Método 2: via Homebrew (recomendado)
```

```
Se você tem o Homebrew instalado, basta:
```

brew install git

Método 3: via Xcode Command Line Tools

Em versões recentes do macOS, ao tentar usar git no terminal, o sistema pode oferecer a instalação automática das ferramentas de linha de comando. Aceite e aguarde.

2.2.4 Instalação no Linux

A instalação no Linux varia conforme a distribuição. Use o gerenciador de pacotes correspondente.

Debian/Ubuntu (apt)

```
sudo apt update
sudo apt install git

Fedora/RHEL (dnf)
```

sudo dnf install git

openSUSE (zypper)

```
sudo zypper install git

Arch Linux (pacman)
```

sudo pacman -S git

Após a instalação, confirme com git --version.

2.2.5 Instalação via código-fonte (avançado)

Caso precise de uma versão específica ou queira compilar manualmente, consulte a documentação oficial em https://git-scm.com/book/en/v2/Getting-Started-Installing-Git. Para a grande maioria dos usuários, os métodos acima são suficientes.

2.3 Instalando e Configurando o Visual Studio Code

2.3.1 Instalação do VS Code

Windows

1. Acesse https://code.visualstudio.com/download
2. Escolha a versão para Windows (User Installer ou System Installer – recomendamos o System Installer para todos os usuários).
3. Execute o instalador e siga os passos, marcando as opções:

- Adicionar ação "Abrir com Code" ao menu de contexto de arquivo
- Adicionar ação "Abrir com Code" ao menu de contexto de diretório
- Registrar Code como editor para tipos de arquivo suportados
- Adicionar ao PATH (essencial para usar o comando code no terminal)

macOS

1. Baixe o arquivo .zip do site oficial.
2. Extraia e arraste o aplicativo "Visual Studio Code" para a pasta "Aplicativos".
3. Abra o VS Code, pressione Cmd+Shift+P e digite "shell command" para instalar o comando code no PATH (isso permitirá abrir arquivos do terminal com code .).

Linux

- Debian/Ubuntu: baixe o arquivo .deb do site e instale com sudo dpkg -i <arquivo>.deb ou use o repositório da Microsoft.
- Fedora: baixe o arquivo .rpm e instale com sudo rpm -i <arquivo>.rpm ou via repositório.
- Outras distribuições: utilize o gerenciador de pacotes ou o pacote .tar.gz.

Após a instalação, abra o terminal e teste:

```
code --version

2.3.2 Extensões essenciais para Git

O VS Code já possui integração nativa com Git, mas algumas extensões tornam a
experiência ainda mais produtiva. Recomendo instalar:

· GitLens — Git supercharged (eamodio.gitlens): amplia as funcionalidades do
Git, mostrando blame, histórico, explorers, etc.
· Git History (donjayamanne.githistory): visualiza e navega pelo histórico de
commits.
· Git Graph (mhutchie.git-graph): exibe um gráfico interativo dos commits e
branches.
· Markdown All in One (yzhang.markdown-all-in-one): útil para escrever a
documentação do livro.

Para instalar, clique no ícone de extensões na barra lateral (Ctrl+Shift+X) e
pesquise pelos nomes.

2.3.3 Configurando o terminal integrado

O VS Code possui um terminal integrado que pode ser aberto com Ctrl+` (acento
grave). Por padrão, ele usa o shell do sistema. Você pode alterar para o Git
Bash no Windows:

1. Abra as configurações (Ctrl+,).
2. Pesquise por "terminal integrated shell windows".
3. Em Terminal > Integrated > Automation Shell: Windows, defina o caminho
para o Git Bash (ex: C:\Program Files\Git\bin\bash.exe).
```

```
No macOS/Linux, o terminal padrão (bash ou zsh) já funciona perfeitamente.

---

2.4 Configuração Inicial do Git

O Git precisa de algumas informações básicas para registrar quem faz os
commits. Essas configurações podem ser feitas em três níveis:

· Sistema (--system): afeta todos os usuários da máquina.
· Global (--global): afeta o usuário atual em todos os repositórios.
· Local (--local): afeta apenas o repositório atual (padrão).

Na maioria dos casos, configuramos o nível global.

2.4.1 Configurando nome e e-mail
```

git config --global user.name "Haroldo Capucci"

git config --global user.email "joseharoldocapucci@gmail.com"

Importante: Este e■mail deve ser o mesmo que você utilizará para criar sua conta no GitHub (capítulo 3). Isso garante que seus commits sejam corretamente atribuídos a você nas plataformas de hospedagem.

Para verificar as configurações atuais:

```
git config --global --list

2.4.2 Configurando o editor padrão

O Git pode abrir um editor de texto para você digitar mensagens de commit
(quando não usa a opção -m). Defina o VS Code como editor padrão:
```

git config --global core.editor "code --wait"

No Windows, pode ser necessário usar o caminho completo ou garantir que o comando code esteja no PATH. A flag --wait faz com que o terminal aguarde até que você feche o arquivo no VS Code.

Outros editores comuns:

- Nano: git config --global core.editor "nano"
- Vim: git config --global core.editor "vim"
- Emacs: git config --global core.editor "emacs"

2.4.3 Configurando ferramentas de diff e merge

Para facilitar a resolução de conflitos, você pode definir uma ferramenta visual de diff e merge. Por exemplo, para usar o próprio VS Code:

```
git config --global diff.tool vscode
git config --global difftool.vscode.cmd "code --wait --diff $LOCAL $REMOTE"
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd "code --wait $MERGED"

Existem ferramentas especializadas como Beyond Compare, Meld, KDiff3. A
configuração varia, mas o princípio é o mesmo.

2.4.4 Configurando o nome da branch padrão

Desde outubro de 2020, o GitHub usa main como nome padrão da branch
principal. Para alinhar seu ambiente local:
```

git config --global init.defaultBranch main

2.4.5 Configurando aliases (atalhos)

Aliases são apelidos para comandos longos. Exemplos úteis:

```
git config --global alias.s status
git config --global alias.c commit
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.l "log --oneline --graph"

Agora, em vez de git status, você pode usar git s. O alias git l mostra um
log resumido e graficamente bonito.

2.4.6 Cache de credenciais

Para não precisar digitar usuário e senha a cada interação com o remoto (via
HTTPS), configure o cache:
```

git config --global credential.helper cache

No Windows, o Git pode usar o Gerenciador de Credenciais do Windows. Durante a instalação, há uma opção para isso. No macOS, o helper padrão é o keychain.

2.5 Git Bash e Terminal Integrado

2.5.1 O que é Git Bash?

No Windows, ao instalar o Git, você ganha o Git Bash, um terminal que emula o ambiente Bash do Linux. Ele permite usar comandos Unix-like (como ls, pwd, grep) no Windows, o que é muito útil para quem está acostumado com Linux/macOS e para seguir os exemplos deste livro.

2.5.2 Abrindo o Git Bash

- Pelo menu Iniciar: "Git Bash".
- Clique com o botão direito em uma pasta e escolha "Git Bash Here".

Dentro do Git Bash, você pode executar comandos Git e também comandos do shell.

2.5.3 Terminal integrado do VS Code

No VS Code, abra o terminal integrado (Ctrl+`). Se você configurou o VS Code para usar o Git Bash (conforme 2.3.3), terá um ambiente Unix-like diretamente no editor. Isso facilita a execução de comandos sem sair do contexto de edição.

2.5.4 Primeiros comandos no terminal

Vamos testar se tudo está funcionando:

```
git version
code --version
pwd # mostra diretório atual (no Git Bash)
ls -la # lista arquivos

Se todos os comandos forem reconhecidos, seu ambiente está pronto.

---

2.6 Escolhendo o Editor Padrão e Ferramentas de Diff

2.6.1 Editores de texto para mensagens de commit

Quando você executa git commit sem -m, o Git abre o editor configurado. A
escolha do editor é pessoal, mas para iniciantes recomendamos o VS Code por
sua interface amigável e integração com Git.

2.6.2 Ferramentas de diff

O comando git diff mostra as diferenças entre versões de arquivos. Se você
preferir uma visualização gráfica, pode usar o git difftool com a ferramenta
configurada. Com a configuração acima, basta digitar:
```

git difftool

E o VS Code abrirá uma janela de comparação lado a lado.

2.7 Verificando a Configuração

Para ter certeza de que tudo foi configurado corretamente, execute:

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 3: Autenticação Segura com GitHub

Páginas estimadas: 26

Tempo de leitura: 65–85 minutos

Objetivos do capítulo:

- Criar uma conta no GitHub
- Entender as diferenças entre autenticação HTTPS e SSH
- Gerar e configurar chaves SSH no Windows, macOS e Linux
- Adicionar a chave pública à sua conta do GitHub
- Configurar o GitHub CLI (opcional)
- Gerenciar múltiplas contas e chaves

3.1 Introdução

No capítulo anterior, você preparou seu ambiente local com Git e VS Code. Agora é hora de conectar esse ambiente ao **GitHub**, a plataforma que hospedará seus repositórios e permitirá a colaboração com outros desenvolvedores.

Para que a comunicação entre seu computador e o GitHub seja segura, existem duas formas principais de autenticação: **HTTPS** (com usuário/senha ou token) e **SSH** (com chaves criptográficas). Neste capítulo, você aprenderá ambas, com ênfase no SSH por ser mais seguro e prático após a configuração inicial.

Ao final, você estará apto a clonar, enviar e receber alterações de repositórios remotos sem digitar senha a cada operação.

3.2 Criando uma Conta no GitHub

3.2.1 Passo a passo

1. Acesse https://github.com
2. Clique no botão **"Sign up"** (Inscrever■se)
3. Preencha os campos:

• **Email:** use o mesmo que você configurou no Git (`joseharoldocapucci@gmail.com`). Isso facilitará a atribuição dos commits.

• **Password:** crie uma senha forte (mínimo 8 caracteres, com letras, números e símbolos)

• **Username:** escolha um nome de usuário único. Exemplos: `haroldocapucci`, `josecapucci`, `hcapucci`. Esse nome aparecerá na URL do seu perfil: `github.com/seu-usuario`

• **Email preferences:** opte por receber ou não atualizações (recomendamos desmarcar se não quiser spam)

4. Complete o desafio de verificação (quebra-cabeça ou código enviado por e■mail)
5. Clique em **"Create account"**
6. Verifique seu e■mail com o código enviado pelo GitHub

3.2.2 Escolhendo o plano

Após a verificação, o GitHub perguntará se você deseja o plano **Free** ou **Team**. Escolha **Free** (gratuito). Ele já oferece repositórios públicos e privados ilimitados, com até 3 colaboradores em repositórios privados (suficiente para a maioria dos usuários individuais).

3.2.3 Confirmando o e■mail no Git local

Lembre■se de que você configurou o Git com o e■mail `joseharoldocapucci@gmail.com`. Para garantir que seus commits fiquem associados à sua conta do GitHub, verifique novamente:

```
git config --global user.email

A saída deve ser <joseharoldocapucci@gmail.com>. Se não for, corrija:
```

git config --global user.email "joseharoldocapucci@gmail.com"

3.3 Autenticação via HTTPS vs. SSH

3.3.1 HTTPS (Hypertext Transfer Protocol Secure)

Como funciona:

• Você se autentica com seu nome de usuário e senha (ou token) do GitHub.

- Para cada operação que exija autenticação (push, pull de repositório privado), o Git pede suas credenciais.
- Desde agosto de 2021, o GitHub não aceita mais senhas comuns no terminal; você deve usar um token de acesso pessoal (PAT – Personal Access Token).

Vantagens:

- Mais simples para iniciantes (não requer criação de chaves).
- Funciona em qualquer rede (porta 443 geralmente aberta).

Desvantagens:

- Precisa digitar usuário/token a cada operação (a menos que use cache de credenciais).
- Token tem validade e pode expirar, exigindo renovação.

3.3.2 SSH (Secure Shell)

Como funciona:

- Você gera um par de chaves: uma pública (que você coloca no GitHub) e uma privada (que fica no seu computador, jamais compartilhada).
- A comunicação é criptografada e autenticada automaticamente.
- Não é necessário digitar senha/token a cada operação.

Vantagens:

- Mais seguro (chave privada nunca trafega na rede).
- Mais prático (após configurado, não pede credenciais).
- Recomendado para uso frequente.

Desvantagens:

- Configuração inicial um pouco mais complexa.
- Pode ser bloqueado em redes corporativas muito restritivas.

3.3.3 Qual escolher?

Para o dia a dia, SSH é a melhor opção. Neste capítulo, focaremos nela, mas também explicaremos como gerar um token HTTPS para quem precisar.

3.4 Passo a Passo Detalhado: Gerando e Configurando Chaves SSH

3.4.1 Verificando se você já possui chaves SSH

Abra o terminal e execute:

```
ls -al ~/.ssh

Se você já tiver chaves SSH, verá arquivos como id_rsa (privada) e id_rsa.pub
```

```
(pública), ou id_ed25519/id_ed25519.pub. Caso existam, você pode usá■las ou
gerar novas. Se não houver, siga os próximos passos.

3.4.2 Gerando uma nova chave SSH (recomendado: ed25519)

O algoritmo Ed25519 é mais moderno e seguro que o RSA. Use o comando:
```

ssh-keygen -t ed25519 -C "joseharoldocapucci@gmail.com"

Explicação:

- -t ed25519: tipo de chave
- -C "email": comentário (geralmente seu e■mail, para identificar a chave)

Se seu sistema não suportar Ed25519 (muito raro), use RSA:

```
ssh-keygen -t rsa -b 4096 -C "joseharoldocapucci@gmail.com"

Durante a execução, o comando perguntará:

Enter file in which to save the key (/home/você/.ssh/id_ed25519):

Pressione Enter para aceitar o local padrão.

Em seguida, solicitará uma frase secreta (passphrase):

Enter passphrase (empty for no passphrase):

Você pode deixar em branco (apenas Enter) ou definir uma frase. Uma frase
secreta adiciona uma camada extra de segurança: mesmo que alguém obtenha sua
chave privada, precisará da frase para usá■la. A desvantagem é que você
precisará digitá■la sempre que usar a chave (a menos que use um agente SSH).
Para a maioria dos usuários domésticos, deixar em branco é aceitável.

3.4.3 Adicionando a chave ao agente SSH (opcional, mas recomendado)

O agente SSH gerencia suas chaves e evita que você digite a frase secreta
repetidamente.

No Linux/macOS:
```

Inicie o agente em segundo plano

eval "$(ssh-agent -s)"

Adicione a chave privada

ssh-add ~/.ssh/id_ed25519

No Windows (Git Bash):

```
# Inicie o agente
eval $(ssh-agent -s)

# Adicione a chave
ssh-add ~/.ssh/id_ed25519

Se você definiu uma frase secreta, ela será solicitada agora.

3.4.4 Exibindo a chave pública

A chave pública é o arquivo com extensão .pub. Para vê■la e copiá■la:
```

cat ~/.ssh/id_ed25519.pub

A saída será algo como:

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIG1yR3b5... joseharoldocapucci@gmail.com

Selecione todo o conteúdo (inclusive o ssh-ed25519 no início e o e■mail no final) e copie para a área de transferência.

3.5 Adicionando a Chave SSH à Sua Conta do GitHub

1. Faça login no GitHub.
2. No canto superior direito, clique na sua foto de perfil e escolha "Settings".
3. Na barra lateral esquerda, clique em "SSH and GPG keys".
4. Clique no botão verde "New SSH key".
5. No campo "Title", dê um nome descritivo para identificar a chave (ex: "Meu notebook pessoal").
6. No campo "Key", cole a chave pública que você copiou.
7. Clique em "Add SSH key".
8. Confirme sua senha do GitHub, se solicitado.

Pronto! Agora seu computador está autorizado a se comunicar com o GitHub via SSH.

3.6 Testando a Conexão SSH

Para verificar se tudo funcionou:

```
ssh -T git@github.com

A primeira vez que você conectar, verá uma mensagem como:

The authenticity of host 'github.com (140.82.113.4)' can't be established.
ED25519 key fingerprint is
```

```
SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
Are you sure you want to continue connecting (yes/no)?

Digite yes e pressione Enter. Em seguida, se tudo estiver correto, você verá:

Hi HaroldoCapucci! You've successfully authenticated, but GitHub does not
provide shell access.

(Substitua HaroldoCapucci pelo seu nome de usuário.)

Caso receba uma mensagem de permissão negada, revise os passos anteriores.

---

3.7 Configurando o GitHub CLI (opcional)

O GitHub CLI (gh) é uma ferramenta de linha de comando que permite interagir
com o GitHub sem sair do terminal: criar repositórios, abrir pull requests,
ver issues, etc.

3.7.1 Instalação

· Windows: winget install --id GitHub.cli ou baixe o instalador em
<https://cli.github.com>
· macOS: brew install gh
· Linux: siga as instruções para sua distribuição em
<https://github.com/cli/cli#installation>

3.7.2 Autenticação

Após instalar, execute:
```

gh auth login

O assistente perguntará:

- Qual protocolo usar: escolha SSH.
- Se deseja usar suas chaves SSH existentes: escolha yes.
- Ele fará o teste de conexão e autenticará.

Pronto! Agora você pode usar comandos como gh repo create, gh pr list, etc.

3.8 Gerenciando Múltiplas Contas e Chaves

Se você trabalha com contas diferentes (ex: pessoal e empresarial), pode precisar de chaves SSH distintas e configurar o Git para usar a chave correta em cada repositório.

3.8.1 Criando chaves específicas por conta

Gere uma chave com nome diferente:

```
ssh-keygen -t ed25519 -C "pessoal@email.com" -f ~/.ssh/id_ed25519_pessoal
ssh-keygen -t ed25519 -C "empresa@email.com" -f ~/.ssh/id_ed25519_empresa
```

```
3.8.2 Configurando o arquivo ~/.ssh/config

Crie (ou edite) o arquivo ~/.ssh/config com blocos como:

# Conta pessoal
Host github.com-pessoal
HostName github.com
User git
IdentityFile ~/.ssh/id_ed25519_pessoal

# Conta empresarial
Host github.com-empresa
HostName github.com
User git
IdentityFile ~/.ssh/id_ed25519_empresa

Agora, ao clonar um repositório da conta pessoal, você usará o host
github.com-pessoal:
```

git clone git@github.com-pessoal:usuario-pessoal/repo.git

3.8.3 Configurando o Git localmente por repositório

Dentro de cada repositório, você pode configurar o nome e e■mail específicos:

```
git config user.name "Haroldo Pessoal"
git config user.email "pessoal@email.com"

Isso sobrescreve a configuração global apenas naquele repositório.

---

3.9 Autenticação via HTTPS com Token

Caso prefira ou precise usar HTTPS, siga estes passos:

3.9.1 Gerando um token de acesso pessoal

1. No GitHub, vá para Settings → Developer settings → Personal access tokens
→ Tokens (classic).
2. Clique em "Generate new token" → "Generate new token (classic)".
3. Dê um nome (ex: "Meu computador").
4. Selecione os escopos (permissões). Para operações básicas de repositório,
marque repo (acesso total a repositórios). Se for apenas para clonar
repositórios públicos, pode marcar apenas public_repo.
5. Clique em "Generate token".
6. Copie o token imediatamente (ele não será mostrado novamente).

3.9.2 Usando o token

Ao clonar com HTTPS, você usará o token como senha:
```

git clone https://github.com/usuario/repo.git

Quando pedir usuário, digite seu nome de usuário

Quando pedir senha, cole o token (não sua senha do GitHub)

Para não precisar digitar toda vez, configure o cache:

Capítulo 4

Primeiros Passos com Repositórios

Páginas estimadas: 28

Tempo de leitura: 60-90 minutos

Pré-requisitos: Git instalado, VS Code configurado (Capítulos 2 e 3)

4.1 Introdução: O Coração do Git

Imagine que você está construindo uma casa. Você não começa levantando paredes sem antes preparar o terreno e fazer a fundação, certo? No universo do Git, o **repositório** é exatamente isso: a fundação de todo o projeto.

Um repositório Git (ou "repo") é como um "escritório mágico" que fica vigiando todas as pastas e arquivos do seu projeto. Ele não apenas guarda os arquivos, mas **registra cada alteração feita ao longo do tempo**, permitindo que você volte a qualquer momento anterior como se fosse uma máquina do tempo.

Neste capítulo, você vai aprender a criar esse "escritório mágico" de duas formas:

1. **Criando do zero** (`git init`) - quando você está começando um projeto novo
2. **Clonando um existente** (`git clone`) - quando quer trabalhar em algo que já existe (no GitHub, por exemplo)

Vamos também explorar o ciclo de vida dos arquivos dentro desse repositório e os comandos fundamentais que você usará centenas de vezes: `status`, `add` e `commit`.

4.2 Criando um Repositório do Zero: `git init`

4.2.1 O Comando Mágico

O comando `git init` é o ponto de partida. Ele transforma uma pasta comum do seu computador em um repositório Git. É como dizer: "A partir de agora, quero que o Git monitore tudo o que acontecer aqui dentro".

Sintaxe básica:

```
git init
```

```
4.2.2 Mão na Massa: Seu Primeiro Repositório

Vamos criar nosso primeiro projeto. Siga cada passo no seu computador:

Passo 1: Abra o terminal (ou Git Bash no Windows)

Passo 2: Crie uma pasta para o projeto e entre nela:
```

mkdir meu-primeiro-repo

cd meu-primeiro-repo

Passo 3: Inicialize o repositório:

```
git init

Resposta esperada:

Initialized empty Git repository in /caminho/para/meu-primeiro-repo/.git/

Passo 4: Veja o que aconteceu:
```

ls -la

Você notará uma pasta nova chamada .git. Ela é o "coração" do repositório. Nunca mexa nela manualmente a menos que saiba exatamente o que está fazendo. É lá que o Git guarda todo o histórico, configurações e metadados.

4.2.3 O que Acontece nos Bastidores?

Quando você executa git init, o Git:

1. Cria a subpasta .git
2. Dentro dela, prepara estruturas para:

• objects/ – Onde os arquivos e commits são armazenados
• refs/ – Onde as branches e tags são registradas
• HEAD – Um arquivo que indica em qual branch você está
• config – Configurações específicas deste repositório
• description – Usado apenas por programas gráficos

Curiosidade: Você pode ter repositórios Git dentro de outras pastas que também são repositórios? Sim, mas isso é avançado (submódulos) e veremos em capítulos futuros.

4.2.4 Inicializando com Nome da Branch Diferente

Desde outubro de 2020, o GitHub mudou o nome padrão da branch principal de master para main. Se você quiser que seu repositório local já comece com main:

```
git init -b main
```

```
Ou, se já inicializou, pode renomear depois:
```

git branch -m master main

4.2.5 Exercício Prático 4.1

Crie três pastas diferentes (projeto-a, projeto-b, projeto-c) e inicialize repositórios em cada uma. Em seguida, liste o conteúdo de cada pasta para ver a pasta .git. Depois, remova os repositórios com rm -rf .git dentro de cada pasta (cuidado: isso apaga todo o histórico, mas mantém os arquivos).

4.3 Clonando um Repositório Existente: git clone

4.3.1 Quando Usar o Clone

O git clone é usado quando você quer copiar um repositório que já existe – geralmente em um servidor remoto como GitHub, GitLab ou Bitbucket – para o seu computador.

É como "baixar" o projeto, mas com um superpoder: você baixa tudo, inclusive todo o histórico de versões, todas as branches, todas as tags.

4.3.2 O Comando na Prática

Sintaxe:

```
git clone <url-do-repositorio>

Exemplo clonando um repositório real do GitHub:
```

git clone https://github.com/facebook/react.git

Isso criará uma pasta chamada react com todo o código fonte, histórico e configurações.

4.3.3 Clonando com Nome Diferente

Se quiser que a pasta local tenha um nome diferente do repositório original:

```
git clone https://github.com/facebook/react.git meu-react

Agora a pasta será meu-react, não react.

4.3.4 Protocolos de Clone: HTTPS vs. SSH

Você pode clonar usando dois protocolos principais:

Protocolo Exemplo Autenticação
HTTPS https://github.com/usuario/repo.git Usuário/senha ou token
SSH git@github.com:usuario/repo.git Chave SSH (mais seguro)

Recomendação: Use SSH se já configurou chaves (Capítulo 3). É mais seguro e
você não precisa digitar senha toda vez.
```

```
4.3.5 O que Realmente Acontece num Clone?

Quando você executa git clone, o Git:

1. Cria uma nova pasta com o nome do repositório
2. Inicializa um repositório Git vazio dentro dela (git init)
3. Adiciona o repositório original como "remote" chamado origin
4. Busca (fetch) todos os objetos (commits, branches, tags) do remoto
5. Cria uma branch local apontando para a branch padrão do remoto (geralmente
main ou master)
6. Dá checkout nessa branch, ou seja, baixa os arquivos para sua área de
trabalho

4.3.6 Clonando Apenas o Último Commit (Shallow Clone)

Para repositórios enormes (como o Linux, com milhões de commits), você pode
querer clonar apenas o histórico recente para economizar tempo e espaço:
```

git clone --depth 1 https://github.com/torvalds/linux.git

Isso baixa apenas o último commit. Mas atenção: você perde a capacidade de ver o histórico completo ou voltar a versões antigas.

4.3.7 Exercício Prático 4.2

1. Clone o repositório https://github.com/jlord/git-it-electron.git (um tutorial interativo de Git)
2. Entre na pasta clonada
3. Execute git log para ver o histórico
4. Execute git branch -a para ver todas as branches (locais e remotas)
5. Crie um clone raso (--depth 1) do mesmo repositório em outra pasta e compare o tamanho das pastas com du -sh

---

4.4 Entendendo a Área de Trabalho: Os Três Estados do Git

Um dos conceitos mais importantes – e que mais confunde iniciantes – é que o Git gerencia seus arquivos em três áreas principais. Visualize assim:

[Working Directory] <--> [Staging Area] <--> [Repositório Local]

(sua pasta) (index/área de preparo) (.git/commits)

4.4.1 Working Directory (Árvore de Trabalho)

É simplesmente sua pasta do projeto, onde você edita arquivos, cria pastas, deleta coisas. É o que você vê no Explorer/Finder e no VS Code.

Estado: Os arquivos aqui podem estar "modificados" mas ainda não marcados para commit.

4.4.2 Staging Area (Área de Preparo / Index)

É uma área intermediária, como um "rascunho" do próximo commit. Aqui você seleciona quais alterações vão entrar no próximo snapshot.

Pense nisso como um palco: os artistas (arquivos modificados) ficam nos bastidores (working directory). Você escolhe quais sobem ao palco (staging area) para a apresentação (commit).

4.4.3 Repositório Local (Histórico / .git)

É onde o Git armazena permanentemente os commits. Cada commit é um snapshot completo do projeto naquele momento, com metadados (autor, data, mensagem).

4.4.4 Por Que Três Áreas?

Essa separação dá um poder imenso: você pode trabalhar em várias alterações diferentes, mas só commitar aquelas que estão prontas. Por exemplo:

- Você modificou 10 arquivos
- 5 são correções de bugs importantes
- 5 são experimentos ainda não testados

Com o staging, você pode adicionar apenas os 5 primeiros, commitar, e depois decidir o que fazer com os outros.

4.4.5 Fluxo Básico de Trabalho

1. Você modifica arquivos no Working Directory
2. Você adiciona alguns ao Staging Area
3. Você commita o que está no Staging para o Repositório Local

---

4.5 O Ciclo de Vida dos Arquivos

Cada arquivo no seu repositório pode estar em um de quatro estados:

+----------+

Untracked

+----------+

(git add)

v

+----------+ +----------+ +----------+

Unmodified	<-->	Modified	<-->	Staged

+----------+ +----------+ +----------+

^ |

(commit)

+----------+

4.5.1 Untracked (Não Rastreado)

Arquivos que o Git não está monitorando. São novos na pasta e o Git não sabe se você quer versioná-los ou não.

Exemplo: Você acabou de criar README.md e o Git mostra como "untracked".

4.5.2 Unmodified (Não Modificado)

Arquivos que já estão no repositório e não foram alterados desde o último commit.

Exemplo: index.html depois que você fez commit e não tocou nele.

4.5.3 Modified (Modificado)

Arquivos rastreados que foram alterados, mas ainda não foram marcados para commit.

Exemplo: Você editou style.css mas não executou git add.

4.5.4 Staged (Preparado)

Arquivos modificados que você marcou para entrar no próximo commit.

Exemplo: Você executou git add style.css e agora ele aguarda o commit.

4.6 Comandos Essenciais: status, add, commit

Vamos colocar a mão na massa com os três comandos que você usará em 90% do tempo.

4.6.1 git status – O "Radar" do Git

O comando mais importante para iniciantes. Ele mostra exatamente:

- Em qual branch você está
- Quais arquivos foram modificados
- Quais estão no staging
- Quais não são rastreados

Crie um cenário prático:

```
# Dentro do seu repositório (meu-primeiro-repo)
echo "# Meu Projeto" > README.md
echo "console.log('Olá')" > app.js
git status

Saída esperada:

On branch main

No commits yet

Untracked files:
(use "git add <file>..." to include in what will be committed)
README.md
app.js

nothing added to commit but untracked files present (use "git add" to track)

O Git está dizendo: "Olha, tem dois arquivos novos aqui, mas ainda não estou
rastreando eles. Se quiser que eu comece a rastrear, use git add."

4.6.2 git add – Movendo para o Staging

O comando git add tem várias funções:

· Começar a rastrear arquivos untracked
· Marcar arquivos modified como staged
· Marcar arquivos removidos para o próximo commit

Sintaxes comuns:
```

Adicionar um arquivo específico

git add README.md

Adicionar vários arquivos

git add app.js style.css

Adicionar todos os arquivos (cuidado!)

git add .

Adicionar todos os arquivos .js

git add *.js

Adicionar tudo (versão mais explícita)

git add -A

Aplique no nosso exemplo:

```
git add README.md
git status

Saída:

On branch main

No commits yet

Changes to be committed:
(use "git rm --cached <file>..." to unstage)
new file: README.md

Untracked files:
(use "git add <file>..." to include in what will be committed)
app.js

Agora README.md está "staged" (pronto para commit) e app.js continua
untracked.

4.6.3 git commit – Tirando o Snapshot

O commit é o coração do Git. É como tirar uma foto do projeto naquele
momento.

Sintaxe básica:
```

git commit -m "mensagem descritiva"

A mensagem é obrigatória e deve ser clara. Uma boa mensagem responde: "O que este commit faz?"

Exemplo:

```
git commit -m "Adiciona README inicial com título do projeto"

Saída:

[main (root-commit) a1b2c3d] Adiciona README inicial com título do projeto
```

```
1 file changed, 1 insertion(+)
create mode 100644 README.md

Vamos analisar a saída:

· main – branch onde o commit foi feito
· a1b2c3d – os primeiros 7 caracteres do hash do commit (identificador único)
· 1 file changed, 1 insertion(+) – estatística: 1 arquivo alterado, 1 linha
adicionada

Agora execute git status novamente:

On branch main
Untracked files:
(use "git add <file>..." to include in what will be committed)
app.js

nothing added to commit but untracked files present

O README.md sumiu da lista? Ele está "unmodified" agora, pois foi salvo no
último commit.

4.6.4 Commit com Mensagem Multilinha

Para mensagens mais longas, use o comando sem -m para abrir o editor
configurado:
```

git commit

Isso abre o editor (Vim, Nano, VS Code) onde você pode escrever:

- Primeira linha: título (até 50 caracteres)
- Linha em branco
- Corpo da mensagem (descrição detalhada)

4.6.5 Boas Práticas para Mensagens de Commit

Uma boa mensagem de commit segue convenções:

Tipo: Resumo curto (até 50 caracteres)

Corpo opcional explicando o que foi feito e por que, não como.

Pode ter várias linhas, com quebra de linha em 72 caracteres.

Tipos comuns:

- feat: – nova funcionalidade
- fix: – correção de bug
- docs: – documentação
- style: – formatação (espaços, ponto-e-vírgula, etc)
- refactor: – refatoração de código

- test: – testes
- chore: – tarefas de manutenção

Exemplo ruim:

git commit -m "alteracoes"

Exemplo bom:

git commit -m "feat: adiciona validação de email no formulário de cadastro"

4.6.6 Exercício Prático 4.3

1. No repositório meu-primeiro-repo, crie mais um arquivo: style.css com qualquer conteúdo
2. Use git status para ver os untracked files
3. Adicione app.js e style.css ao staging (git add app.js style.css)
4. Faça um commit com a mensagem "feat: adiciona scripts e estilos iniciais"
5. Modifique README.md (adicione uma linha, por exemplo)
6. Execute git status – agora README.md aparece como "modified"
7. Adicione a modificação ao staging e faça um novo commit

4.7 Visualizando Tudo no VS Code

O VS Code tem uma integração fantástica com Git. Você pode fazer quase tudo sem digitar comandos.

4.7.1 A Guia Controle de Código Fonte

Clique no ícone de branch na barra lateral esquerda (ou Ctrl+Shift+G).

Lá você verá:

- Changes – Arquivos modificados (working directory)
- Staged Changes – Arquivos preparados para commit

4.7.2 Fazendo Tudo pelo VS Code

Para adicionar ao staging: Passe o mouse sobre o arquivo e clique no "+" que aparece.

Para commitar: Digite a mensagem no campo superior e clique no "✔" (check) ou pressione Ctrl+Enter.

Para ver o status: Tudo é mostrado visualmente.

4.7.3 O Editor de Diff

Clique duas vezes em um arquivo modificado. O VS Code abre uma visualização "diff":

- Lado esquerdo: versão antiga (último commit)

- Lado direito: versão atual
- Linhas vermelhas: removidas
- Linhas verdes: adicionadas

4.7.4 Desfazendo Ações no VS Code

- Desfazer staging: clique no "-" ao lado do arquivo na área "Staged Changes"
- Descartar alterações: clique no ícone de "desfazer" (seta curvada) ao lado do arquivo em "Changes"

Atenção: Descartar alterações é permanente! Use com cuidado.

4.8 Ignorando Arquivos com .gitignore

4.8.1 Por que Ignorar Arquivos?

Nem tudo deve ser versionado:

- Arquivos de configuração local (com senhas)
- Pastas node_modules (podem ser recriadas com npm install)
- Arquivos compilados (.exe, .class, .pyc)
- Logs, caches, arquivos temporários
- Arquivos do sistema (.DS_Store no Mac)

4.8.2 Criando um .gitignore

Crie um arquivo chamado .gitignore na raiz do repositório. Dentro, liste padrões de arquivos/pastas a ignorar.

Exemplo básico:

Dependências

node_modules/

Arquivos compilados

*.exe

*.dll

*.so

*.dylib

*.class

*.pyc

Logs e caches

logs/

*.log

npm-debug.log*

yarn-debug.log*

yarn-error.log*

Sistema operacional

.DS_Store

Thumbs.db

Ambiente

.env

.vscode/

4.8.3 Como Funciona?

- node_modules/ – ignora a pasta inteira
- *.log – ignora todos os arquivos terminados em .log
- !important.log – exclamação significa "não ignore este" (exceção)

4.8.4 Git Ignore Templates Prontos

O GitHub mantém uma coleção de templates para diferentes linguagens:

https://github.com/github/gitignore

Para um projeto Node.js, por exemplo, você pode baixar o template específico.

4.8.5 Verificando o que está sendo ignorado

```
git status --ignored
```

```
Mostra todos os arquivos ignorados.

4.8.6 Exercício Prático 4.4

1. No seu repositório, crie uma pasta logs/ e dentro dela um arquivo app.log
2. Crie um arquivo .gitignore com logs/ dentro
3. Execute git status – note que a pasta logs não aparece mais
4. Execute git status --ignored – agora você vê logs/ como ignorado
5. Commit o .gitignore

---

4.9 Removendo e Movendo Arquivos

4.9.1 Removendo Arquivos (git rm)

Se você simplesmente deletar um arquivo com rm arquivo.txt, o Git percebe a
remoção, mas ela fica como "modified" (deleted). Você precisa adicionar essa
remoção ao staging.

Forma correta de remover:
```

Remove do sistema de arquivos e do staging

git rm arquivo.txt

Se já tiver modificado e quiser remover mesmo assim (forçar)

git rm -f arquivo.txt

Remove apenas do staging, mantendo o arquivo local (útil para .gitignore)

git rm --cached arquivo.txt

4.9.2 Movendo/Renomeando (git mv)

```
# Renomear
git mv antigo.txt novo.txt

# Mover para pasta
git mv arquivo.txt pasta/

Isso é equivalente a:
```

mv antigo.txt novo.txt

git add -A

4.9.3 Exercício Prático 4.5

1. Crie um arquivo teste.txt
2. Adicione e commite
3. Use git rm teste.txt
4. Execute git status – veja a remoção staged
5. Commit a remoção com git commit -m "remove arquivo teste"

4.10 Histórico de Comandos: Recap e Dicas

4.10.1 Comandos Aprendidos

Comando Função Uso comum

git init Cria repositório git init ou git init -b main

git clone Copia repositório remoto git clone URL

git status Mostra situação atual git status -s (resumido)

git add Prepara arquivos git add . ou git add arquivo

git commit Salva snapshot git commit -m "msg"

git rm Remove arquivos git rm arquivo

git mv Move/renomeia gitmv velho novo

4.10.2 Atalhos e Configurações Úteis

Crie atalhos (aliases) para comandos frequentes:

```
git config --global alias.s status
git config --global alias.c "commit -m"
git config --global alias.a "add ."

Agora você pode usar:
```

git s # ao invés de git status

git c "mensagem" # ao invés de git commit -m "mensagem"

git a # ao invés de git add .

4.10.3 Flags Úteis para status

```
# Versão resumida (mais limpa)
git status -s

# Mostrar arquivos ignorados
git status --ignored

# Mostrar branch e informações curtas
git status -sb

4.10.4 Flags Úteis para commit
```

Adiciona todos os arquivos modificados e já abre o editor

git commit -a

Adiciona tudo e já passa a mensagem

git commit -am "mensagem"

Altera o último commit (amend)

git commit --amend -m "nova mensagem"

Cuidado com --amend: Ele reescreve o histórico. Use apenas em commits que ainda não foram enviados para o remoto.

4.11 Problemas Comuns e Soluções

4.11.1 "fatal: not a git repository"

Você está tentando executar um comando Git fora de um repositório. Entre na pasta correta ou execute git init.

4.11.2 "nothing added to commit but untracked files present"

Você tem arquivos novos, mas não os adicionou ao staging. Use git add primeiro.

4.11.3 Esqueci de adicionar um arquivo no último commit

```
git add arquivo-esquecido.txt
git commit --amend --no-edit

Isso adiciona o arquivo ao commit anterior sem alterar a mensagem.

4.11.4 Quero desfazer o staging de um arquivo
```

git restore --staged arquivo.txt

Antes do Git 2.23, usava-se git reset HEAD arquivo.txt.

4.11.5 Quero desfazer as alterações de um arquivo (voltar ao último commit)

```
git restore arquivo.txt

Cuidado: Isso perde as alterações permanentemente.

4.11.6 Mensagem de commit errada
```

git commit --amend -m "mensagem correta"

4.11.7 Adicionei arquivos grandes demais

Se você acidentalmente adicionou um arquivo muito grande (e ainda não fez push), você pode removê-lo do histórico com git rm --cached e depois adicioná-lo ao .gitignore.

Para remover completamente do histórico (mais avançado), use git filter-branch ou a ferramenta BFG Repo-Cleaner.

4.12 Resumo do Capítulo

Neste capítulo, você aprendeu:

1. Criar repositórios com git init e git clone
2. Os três estados do Git: Working Directory, Staging Area, Repositório Local
3. O ciclo de vida dos arquivos: untracked, unmodified, modified, staged
4. Comandos essenciais: status, add, commit
5. Usar o VS Code para gerenciar versionamento visualmente
6. Ignorar arquivos com .gitignore
7. Remover e mover arquivos com git rm e git mv
8. Resolver problemas comuns do dia a dia

Você agora tem uma base sólida para começar a versionar seus projetos. No próximo capítulo, aprenderemos a navegar pelo histórico, comparar versões e viajar no tempo com git log, git diff, git checkout e outros comandos poderosos.

4.13 Exercícios de Fixação (Respostas no Apêndice D)

1. Qual comando transforma uma pasta comum em um repositório Git?
2. Explique a diferença entre Working Directory, Staging Area e Repositório Local.
3. O que significa um arquivo estar "untracked"?
4. Como você adiciona todos os arquivos .js ao staging?
5. Qual a diferença entre git commit -m e git commit sem a flag?
6. Por que é importante usar um arquivo .gitignore?
7. Como você desfaz o staging de um arquivo sem perder as alterações?
8. Qual comando baixa um repositório remoto para seu computador?
9. Como você vê arquivos ignorados pelo Git?
10. O que faz git commit --amend?

4.14 Referências e Leitura Recomendada

- Pro Git Book (2ª edição) - Capítulo 2: Git Basics
- Documentação oficial: git help init, git help clone
- GitHub Guides: "Getting started with Git"
- Visual Git Reference: https://marklodato.github.io/visual-git-guide/

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 5: Navegando pela História do Projeto

Páginas estimadas: 30

Tempo de leitura: 75–95 minutos

Objetivos do capítulo:

- Visualizar o histórico de commits com `git log` e suas variações
- Navegar entre diferentes versões do projeto usando `git checkout` e `git switch`
- Comparar mudanças entre commits, branches e arquivos com `git diff`
- Corrigir erros com `git commit --amend` e `git restore`
- Utilizar a interface gráfica do VS Code para explorar o histórico

---

5.1 Introdução

No capítulo anterior, você aprendeu a criar repositórios, fazer commits e entender o ciclo de vida dos arquivos. Agora é hora de explorar o **histórico** do projeto – uma das funcionalidades mais poderosas do Git.

Saber navegar pela história permite:

- Entender a evolução do código
- Encontrar quando e por que uma determinada mudança foi feita
- Comparar versões para depurar problemas
- Voltar a um estado anterior caso algo dê errado

Neste capítulo, você dominará os comandos que tornam o Git uma verdadeira "máquina do tempo".

5.2 Visualizando o Histórico com `git log`

O comando `git log` é a porta de entrada para o histórico. Ele lista os commits em ordem cronológica reversa (do mais recente para o mais antigo).

5.2.1 Uso básico

```
git log

A saída padrão mostra, para cada commit:

· Hash (identificador único)
· Autor
· Data
· Mensagem

Pressione espaço para rolar para baixo, q para sair.

5.2.2 Visualizando de forma resumida
```

git log --oneline

Cada commit aparece em uma única linha com o hash abreviado e a mensagem. Exemplo:

a1b2c3d Adiciona validação de email

e4f5g6h Corrige bug no formulário

i7j8k9l Commit inicial

5.2.3 Exibindo estatísticas

```
git log --stat

Mostra, além da mensagem, quais arquivos foram alterados e quantas linhas
foram adicionadas/removidas.

5.2.4 Visualizando o gráfico de branches
```

git log --graph --oneline --all

Essa combinação mágica desenha um gráfico ASCII mostrando a relação entre branches e merges. É uma das visualizações mais úteis.

5.2.5 Filtros úteis

- Por autor: git log --author="Haroldo"
- Por data: git log --since="2 weeks ago" ou --until="2023-12-01"
- Por mensagem: git log --grep="bugfix" (busca na mensagem)
- Por arquivo: git log -- README.md (histórico de um arquivo específico)

5.2.6 Formatação personalizada

Você pode criar seu próprio formato com --pretty=format:

```
git log --pretty=format:"%h - %an, %ar : %s"

Os placeholders comuns:

· %h – hash abreviado
· %an – author name
· %ar – data relativa ("2 dias atrás")
· %s – assunto (mensagem)

5.2.7 Exercício prático 5.1

No repositório que você criou no capítulo anterior, faça alguns commits
extras (pelo menos 5) com mensagens variadas. Depois execute:

· git log --oneline
· git log --graph --oneline
· git log --author="seu nome"
· git log --grep="feat" (se tiver usado essa palavra)

---

5.3 Navegando entre Commits com git checkout e git switch

5.3.1 Movendo o ponteiro HEAD

O Git mantém um ponteiro especial chamado HEAD que indica onde você está no
momento (qual commit ou branch está "checkoutado").

Para viajar para um commit específico, use:
```

git checkout <hash-do-commit>

Exemplo:

```
git checkout a1b2c3d

Após esse comando, você entra em estado "detached HEAD" – você não está em
nenhuma branch, mas sim em um commit específico. Pode explorar o projeto como
estava naquele momento.

5.3.2 Retornando para a branch principal

Para voltar ao estado normal (na branch main):
```

git checkout main

Ou, se usou switch (mais moderno):

```
git switch main

5.3.3 Diferença entre checkout e switch

O comando git checkout é versátil demais: serve tanto para trocar de branch
```

```
quanto para restaurar arquivos. Por isso, versões recentes do Git
introduziram git switch (focado em branches) e git restore (focado em
arquivos), deixando o comando mais intuitivo.

Recomendação: Use git switch para mudar de branch e git restore para
manipular arquivos. Mas git checkout ainda funciona e é amplamente usado.

5.3.4 Navegação segura

Quando você navega para um commit antigo, o diretório de trabalho reflete
exatamente aquele snapshot. Você pode:

· Compilar/testar o código como era na época
· Criar uma nova branch a partir dali (git switch -c nova-branch)
· Simplesmente inspecionar e voltar

Importante: Se você fizer alterações enquanto estiver em detached HEAD, elas
não pertencerão a nenhuma branch. Se quiser salvá-las, crie uma nova branch
antes de voltar.

5.3.5 Exercício prático 5.2

1. No seu repositório, execute git log --oneline e escolha um commit antigo
(não o mais recente).
2. Use git checkout <hash> para ir até ele.
3. Execute git status – note a mensagem "detached HEAD".
4. Explore os arquivos (estão como eram naquele momento).
5. Volte para main com git switch main.

---

5.4 Comparando Mudanças com git diff

O git diff mostra as diferenças entre versões de arquivos.

5.4.1 Diferenças entre working directory e staging
```

git diff

Mostra as linhas modificadas nos arquivos que ainda não foram adicionados ao staging.

5.4.2 Diferenças entre staging e último commit

```
git diff --staged

Mostra o que será enviado no próximo commit (o que está no staging).

5.4.3 Diferenças entre dois commits
```

git diff <hash1> <hash2>

Exibe as mudanças entre dois commits específicos. A ordem importa: o que mudou de hash1 para hash2.

5.4.4 Diferenças entre branches

```
git diff branch1..branch2

Mostra o que há em branch2 que não está em branch1.

5.4.5 Diferenças em um arquivo específico
```

git diff -- README.md

Limita a comparação a um único arquivo.

5.4.6 Ferramentas visuais

Se configurou uma ferramenta de diff (como no capítulo 2), use:

```
git difftool

Isso abrirá a ferramenta visual configurada (ex: VS Code, Meld).

5.4.7 Exercício prático 5.3

1. Modifique um arquivo no seu repositório (adicione uma linha).
2. Execute git diff e veja a saída.
3. Adicione o arquivo ao staging (git add).
4. Execute git diff --staged – agora você vê as mudanças que estão prontas
para commit.
5. Faça um commit.
6. Compare o commit atual com o anterior usando git diff HEAD^ HEAD (HEAD^
significa o commit anterior).

---

5.5 Corrigindo Erros: git commit --amend e git restore

5.5.1 Alterando o último commit

Às vezes você esquece de incluir um arquivo ou digita a mensagem errada. O
comando --amend permite corrigir o commit mais recente.
```

git add arquivo-esquecido.txt

git commit --amend

Isso abre o editor para que você possa modificar a mensagem. Se quiser manter a mensagem original:

```
git commit --amend --no-edit

Cuidado: Só use --amend em commits que ainda não foram enviados para o remoto
(push). Se já tiver enviado, reescrever o histórico pode causar problemas
para quem já baixou.

5.5.2 Desfazendo alterações no working directory

Se você modificou um arquivo mas quer descartar as alterações (voltar ao
```

```
estado do último commit):
```

git restore arquivo.txt

Antes do Git 2.23, usava-se git checkout -- arquivo.txt. O restore é mais explícito.

5.5.3 Removendo arquivos do staging

Se você adicionou um arquivo ao staging por engano:

```
git restore --staged arquivo.txt

O arquivo volta para modified (as alterações permanecem, mas não estão mais
marcadas para commit).

5.5.4 Desfazendo commits com git reset (cuidado!)

O git reset pode mover o ponteiro da branch para trás, efetivamente
"desfazendo" commits. Existem três modos:

· --soft: mantém as alterações no staging
· --mixed (padrão): mantém as alterações no working directory, mas não no
staging
· --hard: descarta completamente as alterações (perigoso!)

Exemplo: voltar um commit, mantendo as alterações no working directory:
```

git reset HEAD~1

Atenção: git reset reescreve o histórico. Use apenas em commits locais (não enviados).

5.5.5 Exercício prático 5.4

1. Faça um commit com uma mensagem errada (ex: "corrige tudo").
2. Corrija a mensagem com git commit --amend -m "Mensagem correta".
3. Crie um arquivo temporário, adicione ao staging e depois desfaça o staging com git restore --staged.
4. Modifique um arquivo e depois descarte as alterações com git restore.

5.6 Visualizando Detalhes com git show

O comando git show exibe informações detalhadas sobre um objeto (commit, tag, árvore).

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 6: Trabalhando com Branches (Ramificações)

Páginas estimadas: 32

Tempo de leitura: 80–100 minutos

Objetivos do capítulo:

- Compreender o conceito de branches e sua importância
- Criar, listar, renomear e excluir branches
- Navegar entre branches com `git switch` e `git checkout`
- Mesclar branches com `git merge` (fast-forward e merge de três vias)
- Resolver conflitos de merge manualmente e com ferramentas visuais
- Conhecer fluxos de trabalho com branches: Git Flow e GitHub Flow
- Utilizar a interface gráfica do VS Code para gerenciar branches

---

6.1 Introdução

Até agora, trabalhamos em uma linha reta do tempo: cada commit sucedia o anterior na branch `main` (ou `master`). Mas o desenvolvimento de software raramente é linear. Muitas vezes você precisa:

- Desenvolver uma nova funcionalidade sem atrapalhar o código estável
- Corrigir um bug urgente enquanto continua trabalhando em outra tarefa
- Experimentar uma ideia sem risco de quebrar o projeto principal
- Colaborar com outras pessoas de forma paralela

Para tudo isso, o Git oferece um recurso poderoso e leve: **branches** (ramificações). Uma branch é simplesmente um ponteiro móvel para um commit. Criar uma branch é instantâneo e não consome espaço extra, pois ela apenas referencia o histórico existente.

Neste capítulo, você dominará o uso de branches – a funcionalidade que torna o Git imbatível para trabalho paralelo.

6.2 O Conceito de Branches: Por que usar?

6.2.1 Linha do tempo alternativa

Imagine um livro sendo escrito. O autor tem a versão principal (main). Para escrever um novo capítulo experimental, ele cria uma cópia do manuscrito, trabalha nela e, se gostar, incorpora ao original. No Git, essa "cópia" é uma branch.

Analogia com árvore: O tronco principal é a branch `main`. Dele partem galhos (branches) que crescem independentemente. Podem dar frutos (commits) e, se desejado, ser enxertados de volta ao tronco (merge).

6.2.2 Vantagens

- **Isolamento:** alterações em uma branch não afetam outras.
- **Paralelismo:** várias pessoas podem trabalhar em diferentes funcionalidades ao mesmo tempo.
- **Experimentação:** crie uma branch para testar uma ideia; se falhar, basta deletá-la.
- **Organização:** branches podem representar features, hotfixes, releases, etc.

6.3 Gerenciando Branches: Comandos Básicos

6.3.1 Listando branches

```
git branch

Mostra as branches locais. A branch atual é destacada com um asterisco (*
main).

Para ver branches remotas também:
```

git branch -a

6.3.2 Criando uma nova branch

```
git branch nome-da-branch

Isso cria uma nova branch apontando para o mesmo commit onde você está. Mas
você não muda para ela automaticamente.
```

```
6.3.3 Criando e mudando para a nova branch (atalho)
```

git switch -c nome-da-branch

Ou, com o comando antigo:

```
git checkout -b nome-da-branch

Esse é o modo mais comum: cria e já troca para a nova branch.

6.3.4 Trocando de branch

Com git switch (recomendado):
```

git switch nome-da-branch

Com git checkout:

```
git checkout nome-da-branch

6.3.5 Renomeando uma branch
```

git branch -m nome-antigo nome-novo

Se você já estiver na branch que quer renomear:

```
git branch -m nome-novo

6.3.6 Excluindo uma branch
```

git branch -d nome-da-branch

O Git só permite deletar se a branch já tiver sido mesclada (para não perder trabalho). Para forçar a exclusão (mesmo sem mesclar):

```
git branch -D nome-da-branch

Cuidado: branches deletadas localmente podem ser recuperadas via reflog
(capítulo 11) se você tiver o hash do último commit.

---

6.4 Trocando de Branch: git switch vs. git checkout

Como vimos, git checkout é um comando sobrecarregado: serve para trocar de
branch, restaurar arquivos, etc. O Git 2.23 introduziu git switch e git
restore para separar essas responsabilidades.

Recomendação moderna: use git switch para branches e git restore para
arquivos. Mas git checkout ainda é amplamente usado e aceito.

Exemplos comparativos:
```

```
Ação git checkout git switch
Trocar para branch existente git checkout main git switch main
Criar e trocar para nova branch git checkout -b nova git switch -c nova
Voltar ao commit anterior (detached HEAD) git checkout HEAD~1 git switch
--detach HEAD~1

---

6.5 Unindo Históricos com Merge: git merge

Quando você termina o trabalho em uma branch, geralmente quer integrá-la à
branch principal. O comando git merge faz isso.

6.5.1 Preparando o merge

Suponha que você criou uma branch feature/login a partir de main, fez alguns
commits nela e agora quer mesclá-la de volta.

1. Vá para a branch de destino (aquela que receberá as mudanças):
```

git switch main

```
2. Execute o merge:
```

git merge feature/login

```
6.5.2 Tipos de merge

O Git é inteligente e tenta mesclar automaticamente. Existem dois cenários
principais:

Fast-forward (avanço rápido)

Se a branch main não teve novos commits desde que feature/login foi criada, o
Git simplesmente move o ponteiro de main para o commit mais recente de
feature/login. É um merge linear, sem commit extra.

antes: A---B---C (main)
\
D---E (feature/login)

depois: A---B---C---D---E (main, feature/login)

Merge de três vias (recursive)

Se main avançou enquanto você trabalhava na feature, o Git precisa combinar
as duas linhas. Ele cria um novo commit de merge com dois pais.

antes: A---B---C (main)
\
D---E (feature/login)

depois: A---B---C---F (main, commit de merge)
\ /
```

```
D-------E

6.5.3 Visualizando o merge

Após um merge, git log --graph --oneline mostrará o histórico entrelaçado.

6.5.4 Exercício prático 6.1

1. No seu repositório de testes, crie uma branch feature/1 e faça dois
commits nela.
2. Volte para main e faça um commit diferente.
3. Agora mescle feature/1 em main. Observe o tipo de merge (deve ser merge de
três vias).
4. Use git log --graph --oneline para ver o histórico.

---

6.6 Resolvendo Conflitos de Merge na Prática

Conflitos ocorrem quando duas branches alteram a mesma parte do mesmo arquivo
de maneiras diferentes. O Git não sabe qual versão escolher e pede sua ajuda.

6.6.1 Simulando um conflito

1. Crie uma branch feature/a e altere a linha 1 de README.md para "A".
Commit.
2. Volte para main e altere a mesma linha de README.md para "B". Commit.
3. Tente mesclar feature/a em main:
```

git merge feature/a

```
Você verá:

Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.

6.6.2 Marcadores de conflito

Abra o arquivo conflitante. Ele conterá algo como:
```

<<<<<<< HEAD

B

=======

A

>>>>>>> feature/a

- <<<<<<< HEAD – sua versão atual (na branch main)
- ======= – separador
- >>>>>>> feature/a – versão da branch que está sendo mesclada

6.6.3 Resolvendo manualmente

Edite o arquivo para a versão desejada (pode ser uma combinação das duas) e remova os marcadores. Por exemplo, se quiser "A e B", deixe:

A e B

Salve o arquivo.

6.6.4 Finalizando o merge

Após resolver todos os conflitos:

```
git add README.md
git commit

O Git abrirá o editor com uma mensagem de merge pré-preenchida. Você pode
aceitar ou modificar.

6.6.5 Usando ferramentas visuais

Se configurou uma ferramenta de merge (capítulo 2), use:
```

git mergetool

Ele abrirá a ferramenta (ex: VS Code) mostrando as três versões: local (HEAD), remota (feature) e mesclada (a ser salva). No VS Code, após instalar o GitLens, você tem uma interface visual para resolver conflitos.

6.6.6 Abortando um merge

Se a confusão for grande e você quiser voltar ao estado anterior:

```
git merge --abort

Isso desfaz o merge e retorna ao estado anterior.

6.6.7 Exercício prático 6.2

1. Crie uma situação de conflito propositalmente (como no exemplo).
2. Resolva manualmente, adicione e faça commit.
3. Repita o processo, mas desta vez use git mergetool (com VS Code
configurado).

---

6.7 Boas Práticas com Branches: Git Flow e GitHub Flow

Existem modelos (fluxos) de como usar branches em equipe. Dois dos mais
conhecidos são Git Flow e GitHub Flow.

6.7.1 Git Flow

Criado por Vincent Driessen, é um modelo mais estruturado, com branches
```

específicas:

· main: código em produção
· develop: branch de integração, onde as features se juntam
· feature/*: branches para novas funcionalidades (partem de develop, retornam a develop)
· release/*: preparação para uma nova versão (correções finais)
· hotfix/*: correções urgentes em produção (partem de main, mesclam em main e develop)

Vantagens: organizado, previsível.
Desvantagens: complexo, pode ser exagerado para projetos pequenos.

6.7.2 GitHub Flow

Mais simples, usado por equipes que fazem deploy contínuo:

· main é sempre a versão pronta para deploy
· Para qualquer mudança, crie uma branch descritiva (ex: feature/novo-botao)
· Faça commits, abra um Pull Request (PR) para discussão
· Após revisão, faça o merge para main e faça o deploy imediatamente

Vantagens: simplicidade, integração com Pull Requests.
Desvantagens: menos adequado para releases programadas.

6.7.3 Escolhendo um fluxo

Para projetos pessoais ou pequenas equipes, o GitHub Flow é suficiente e mais fácil. Para projetos maiores com ciclos de release definidos, o Git Flow pode ser útil.

6.7.4 Exercício prático 6.3

Pesquise sobre GitLab Flow (uma variação que adiciona branches de ambiente) e compare com os dois modelos acima. Escreva um pequeno resumo.

6.8 Gerenciando Branches no VS Code

6.8.1 Visualizando branches

Na barra inferior esquerda do VS Code, há um indicador da branch atual (ex: main). Clique nele para abrir a paleta de comandos com opções de branches.

6.8.2 Criando e trocando branches

· Criar nova branch: clique no ícone de branch na barra inferior → "Create new branch..."
· Trocar de branch: clique no nome da branch e escolha a desejada na lista.

6.8.3 Publicando uma branch

Após criar uma branch local, você pode publicá-la no remoto com um clique (ícone de "Publicar branch" ao lado do nome).

```
6.8.4 Mesclando branches

Na guia "Source Control" (Ctrl+Shift+G), clique nos três pontos (...) e
escolha "Branch" → "Merge Branch...". Selecione a branch a ser mesclada na
atual.

6.8.5 Resolvendo conflitos visualmente

Quando ocorre um conflito, o VS Code destaca os arquivos. Abra o arquivo e
você verá opções como "Accept Current Change", "Accept Incoming Change",
"Accept Both Changes", ou "Compare Changes". Use os botões para resolver
rapidamente.

---

6.9 Exemplo Prático: Criando uma Nova Funcionalidade com Branches

Vamos simular o desenvolvimento de uma nova funcionalidade seguindo o GitHub
Flow.

1. Parta de main atualizada:
```

git switch main

git pull origin main

```
2. Crie uma branch descritiva:
```

git switch -c feature/adiciona-contador

```
3. Faça as alterações necessárias (adicione um arquivo contador.js).
4. Commit:
```

git add contador.js

git commit -m "feat: adiciona contador de cliques"

```
5. Publique a branch no GitHub:
```

git push -u origin feature/adiciona-contador

```
6. No GitHub, abra um Pull Request (comparando feature/adiciona-contador com
main).
7. Após revisão e aprovação, faça o merge (pode ser feito pelo botão no
GitHub).
8. Volte ao terminal e atualize main localmente:
```

git switch main

git pull origin main

```
9. Opcional: delete a branch local (já que foi mesclada):
```

git branch -d feature/adiciona-contador

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 7: Markdown para Documentação

Páginas estimadas: 24

Tempo de leitura: 60–80 minutos

Objetivos do capítulo:

- Compreender a importância da documentação em projetos de software
- Aprender a sintaxe básica e avançada do Markdown
- Criar arquivos README.md atraentes e informativos
- Utilizar o VS Code para escrever e visualizar Markdown
- Aplicar boas práticas de documentação em repositórios Git

7.1 Introdução

De que adianta ter um código brilhante se ninguém sabe como usá■lo, instalá■lo ou contribuir com ele? A documentação é a ponte entre o desenvolvedor e os usuários (incluindo você mesmo no futuro).

No universo Git/GitHub, o formato padrão para documentação é o **Markdown**. Ele é simples, legível mesmo em texto puro e se converte facilmente em HTML bonito no site do GitHub. Todo repositório que se preze tem um arquivo `README.md` na raiz – é a porta de entrada do projeto.

Neste capítulo, você aprenderá a dominar o Markdown e a criar documentações profissionais que farão seus projetos se destacarem.

7.2 O que é Markdown?

Markdown é uma linguagem de marcação leve criada por John Gruber em 2004. O objetivo é ser fácil de ler e fácil de escrever, mesmo em texto plano, e poder ser

convertida para HTML (e outros formatos) de maneira simples.

7.2.1 Por que usar Markdown?

- **Simplicidade:** a sintaxe é intuitiva e rápida de aprender.
- **Legibilidade:** mesmo sem processamento, o texto é claro.
- **Versatilidade:** GitHub, GitLab, Bitbucket, fóruns, sites estáticos (Jekyll, Hugo) – todos aceitam Markdown.
- **Versionamento:** por ser texto puro, o Git pode mostrar diffs das alterações na documentação.

7.2.2 Onde o Markdown é usado no ecossistema Git

- `README.md` – apresentação do projeto
- `CONTRIBUTING.md` – guia para contribuidores
- `LICENSE.md` – texto da licença
- Issues e Pull Requests (comentários podem usar Markdown)
- Wikis do GitHub

7.3 Sintaxe Básica do Markdown

Vamos explorar os elementos mais comuns. Em todos os exemplos, mostramos a sintaxe e a saída esperada.

7.3.1 Títulos

Use `#` para títulos. Quanto mais `#`, menor o nível.

```
# Título nível 1 (equivalente a <h1>)
## Título nível 2 (<h2>)
### Título nível 3 (<h3>)
#### Título nível 4 (<h4>)
##### Título nível 5 (<h5>)
###### Título nível 6 (<h6>)

7.3.2 Ênfase (itálico e negrito)
```

itálico ou *itálico*

negrito ou __negrito__

itálico e negrito ou ***combinação***

7.3.3 Listas

Não ordenadas: use -, * ou +

```
- Item 1
- Item 2
- Subitem 2.1 (dois espaços antes do traço)
- Subitem 2.2
* Item com asterisco
+ Item com mais

Ordenadas: use números seguidos de ponto
```

1. Primeiro
2. Segundo
1. Subitem (com 4 espaços)
2. Outro subitem
3. Terceiro

7.3.4 Links

```
[Texto do link](https://exemplo.com)
[Link com título](https://exemplo.com "Título opcional")

Links relativos dentro do repositório:
```

[Veja o código](src/index.js)

[Leia a licença](LICENSE.md)

7.3.5 Imagens

Sintaxe similar aos links, mas com ! na frente:

```
![Texto alternativo](caminho/para/imagem.jpg)
![Logo do projeto](images/logo.png "Título opcional")

No GitHub, você pode usar imagens hospedadas ou relativas ao repositório.

7.3.6 Citações em bloco

Use > no início da linha:
```

> Esta é uma citação.

> Pode ter várias linhas.

>> E até citações aninhadas.

7.3.7 Linhas horizontais

Três ou mais hífens, asteriscos ou underscores:

```
---
***
```

```
___

7.3.8 Código

Código inline: envolva com crases `
```

O comando `git status` mostra o estado do repositório.

Bloco de código: use três crases antes e depois, opcionalmente com a linguagem para syntax highlighting:

---

function saudacao(nome) {

return `Olá, ${nome}!`;

}

Para um bloco genérico (sem destaque):

```
Isso é um bloco de código simples.

7.3.9 Exercício prático 7.1

Crie um arquivo teste.md no seu repositório de estudos e escreva nele:

· Um título principal e dois subtítulos
· Uma lista de compras (não ordenada)
· Uma lista de passos (ordenada)
· Um link para o Google
· Uma imagem qualquer (pode ser um placeholder tipo
https://via.placeholder.com/150)
· Um bloco de código em JavaScript com uma função simples

Visualize o resultado no VS Code (veja seção 7.6).

---

7.4 Sintaxe Avançada

O Markdown básico é suficiente para a maioria dos casos, mas alguns recursos
extras são suportados por muitas plataformas (incluindo GitHub).

7.4.1 Tabelas

Use pipes (|) e hífens para separar cabeçalho e linhas:
```

Comando	Descrição
`git status`	Mostra o estado dos arquivos
`git add`	Adiciona arquivos ao staging

Comando	Descrição
`git commit`	Cria um novo commit

Você pode alinhar colunas com dois pontos:

```
| Esquerda | Centro | Direita |
|:---------|:------:|--------:|
| Texto | Texto | Texto |

7.4.2 Listas de tarefas (task lists)

No GitHub, você pode criar checkboxes:
```

- [x] Tarefa concluída
- [ ] Tarefa pendente
- [ ] Outra pendente

7.4.3 Menções a usuários e equipes

Use @usuario para notificar alguém em issues/comentários. No README, apenas mostra o link para o perfil.

7.4.4 Emojis

O GitHub suporta códigos de emoji como :smile: ou :rocket:. Lista completa em https://www.webfx.com/tools/emoji-cheat-sheet/.

Exemplo: :octocat: vira o mascote do GitHub.

7.4.5 Notas de rodapé

Alguns processadores suportam notas de rodapé:

```
Texto com nota[^1].

[^1]: Explicação da nota.

7.4.6 Escapando caracteres especiais

Se quiser mostrar um caractere que tem significado especial (como * ou #),
use barra invertida:
```

Isto não será itálico\

7.4.7 Exercício prático 7.2

No mesmo arquivo teste.md, adicione:

- Uma tabela comparando Git e SVN (pelo menos 3 linhas)
- Uma lista de tarefas com dois itens concluídos e um pendente
- Uma menção ao seu próprio usuário do GitHub (ex: @HaroldoCapucci)

- Um emoji (por exemplo, :tada:)

7.5 Criando um README Profissional

O arquivo README.md é o cartão de visitas do seu projeto. Um bom README responde a perguntas como:

- O que é este projeto?
- Para que serve?
- Como instalar?
- Como usar?
- Onde obter ajuda?
- Quem mantém?
- Como contribuir?

7.5.1 Estrutura recomendada

```
# Nome do Projeto

Breve descrição (1-2 frases) do que o projeto faz.

## Índice

- [Instalação](#instalação)
- [Uso](#uso)
- [Contribuição](#contribuição)
- [Licença](#licença)

## Instalação

Passo a passo para instalar e configurar o ambiente.
```

comando de instalação

Uso

Exemplos de como utilizar o projeto, com código e prints.

```
exemplo de código

Contribuição

Instruções para quem quiser contribuir (pull requests, issues). Mencione o
arquivo CONTRIBUTING.md se houver.

Licença

Informe a licença (ex: MIT, GPL-3.0).


### 7.5.2 Boas práticas
```

```
- Use badges (emblemas) para mostrar status: build passing, cobertura de
testes, versão, licença. Exemplo:
`![Build Status](https://img.shields.io/...)`
- Inclua uma tabela de conteúdos para READMEs longos.
- Mantenha a linguagem clara e direta.
- Atualize sempre que houver mudanças significativas.
- Use exemplos reais de código.

### 7.5.3 Badges (emblemas)

Serviços como [Shields.io](https://shields.io) geram badges personalizáveis.
Exemplos:
```

![GitHub license](https://img.shields.io/github/license/HaroldoCapucci/livro-git-completo)

![GitHub last commit](https://img.shields.io/github/last-commit/HaroldoCapucci/livro-git-completo)

7.5.4 Exercício prático 7.3

Crie um README.md para o repositório do seu livro (ou para um projeto fictício) seguindo a estrutura acima. Inclua pelo menos:

- Título e descrição
- Índice com links funcionais
- Seção de instalação (pode ser fictícia)
- Seção de uso com um exemplo de código
- Pelo menos dois badges (licença e último commit)

7.6 Preview de Markdown no VS Code

O VS Code tem excelente suporte a Markdown nativamente.

7.6.1 Abrindo o preview

- Abra um arquivo .md.
- Clique no ícone de "Abrir visualização" no canto superior direito (ou Ctrl+Shift+V).
- O preview é atualizado em tempo real conforme você digita.

7.6.2 Atalhos úteis

- Ctrl+K V – abre preview ao lado (split view).
- Ctrl+Shift+P e digite "Markdown: Open Preview" – mesma coisa.

7.6.3 Extensões recomendadas

- Markdown All in One (yzhang.markdown-all-in-one): atalhos, auto-completar, TOC automático, formatação.

- markdownlint (DavidAnson.vscode-markdownlint): alerta sobre más práticas na sintaxe.
- GitHub Markdown Preview (bierner.github-markdown-preview): preview idêntico ao do GitHub.

7.6.4 Customizando o preview

Você pode escrever CSS customizado para o preview nas configurações do VS Code, se desejar.

7.6.5 Exportando Markdown

Com extensões como "Markdown PDF", você pode exportar seus arquivos .md para PDF, HTML, etc. Isso é útil para gerar documentação em outros formatos.

7.7 Markdown em Issues e Pull Requests

Ao abrir uma issue ou pull request no GitHub, você pode usar Markdown na descrição e nos comentários. Isso ajuda a estruturar informações:

- Listas de verificação (task lists) para acompanhar progresso.
- Blocos de código com syntax highlighting.
- Menções a usuários.
- Referências a outros issues/PRs com #numero.

7.7.1 Exemplo de issue bem formatada

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 8: Trabalhando com Repositórios Remotos

Páginas estimadas: 28

Tempo de leitura: 70–90 minutos

Objetivos do capítulo:

- Compreender o conceito de repositório remoto e sua importância
- Adicionar um repositório remoto ao projeto local
- Enviar alterações para o remoto com `git push`
- Baixar e integrar alterações com `git pull` e `git fetch`
- Sincronizar forks e trabalhar com múltiplos remotos
- Utilizar o VS Code para gerenciar operações remotas

8.1 Introdução

Até agora, trabalhamos exclusivamente em repositórios locais. O Git, porém, é uma ferramenta distribuída – e uma de suas maiores vantagens é a capacidade de sincronizar o trabalho com outros repositórios, chamados de **remotos**.

Um repositório remoto é uma cópia do seu projeto hospedada em um servidor (como GitHub, GitLab ou Bitbucket) ou mesmo em outra máquina da sua rede. Ele serve como:

- **Backup** do seu código
- **Ponto de colaboração** para outras pessoas
- **Fonte oficial** do projeto (geralmente a versão "principal")

Neste capítulo, você aprenderá a conectar seu repositório local a um remoto, enviar e receber alterações, e gerenciar múltiplos remotos – habilidades essenciais para qualquer desenvolvedor que trabalhe em equipe ou queira compartilhar seu código.

8.2 Entendendo o Conceito de "Remote"

8.2.1 O que é um remote?

Um **remote** é simplesmente um apelido (nome) que aponta para uma URL de outro repositório Git. Por convenção, o remote principal é chamado de `origin`.

Quando você clona um repositório, o Git automaticamente cria um remote chamado `origin` apontando para a URL de onde você clonou.

8.2.2 Por que usar remotos?

- **Colaboração:** várias pessoas podem enviar e receber alterações do mesmo remote.
- **Backup:** mesmo que seu computador queime, o código está seguro no servidor.
- **Integração contínua:** serviços como GitHub Actions podem rodar testes automaticamente quando você envia código.
- **Compartilhamento:** qualquer pessoa pode clonar seu projeto público.

8.2.3 Visualizando remotos existentes

```
git remote -v

O -v (verbose) mostra as URLs de fetch e push. Para um repositório clonado, a
saída típica é:

origin https://github.com/usuario/repo.git (fetch)
origin https://github.com/usuario/repo.git (push)

Se você inicializou um repositório local com git init e ainda não conectou a
nenhum remoto, o comando não retornará nada.

---

8.3 Adicionando um Repositório Remoto: git remote add

8.3.1 Criando um repositório no GitHub (recapitulação)

Antes de adicionar um remote, você precisa de um repositório vazio no GitHub:

1. Acesse github.com e faça login.
2. Clique no botão verde "New" (ou no ícone + no canto superior direito e
"New repository").
3. Dê um nome (ex: meu-projeto).
4. Escolha se será público ou privado.
5. Não marque "Initialize this repository with a README" – queremos um
repositório vazio.
6. Clique em "Create repository".
```

```
Você verá uma página com instruções. Copie a URL do repositório (HTTPS ou
SSH).

8.3.2 Conectando o repositório local ao remoto

No seu terminal, dentro do repositório local, execute:
```

git remote add origin https://github.com/HaroldoCapucci/meu-projeto.git

Ou, se for usar SSH:

```
git remote add origin git@github.com:HaroldoCapucci/meu-projeto.git

Agora seu repositório local tem um remote chamado origin apontando para o
GitHub.

8.3.3 Verificando a conexão
```

git remote -v

Deve mostrar as URLs de fetch e push.

8.3.4 Renomeando ou removendo remotes

- Renomear: git remote rename origin novo-nome
- Remover: git remote remove origin (útil se você errou a URL)

8.4 Enviando Alterações (Upload): git push

O comando git push envia os commits da sua branch local para o repositório remoto.

8.4.1 Primeiro push (estabelecendo upstream)

```
git push -u origin main

· origin é o nome do remote.
· main é a branch local que você quer enviar.
· -u (ou --set-upstream) configura o vínculo entre a branch local e a remota,
para que da próxima vez você possa usar apenas git push.

Após esse comando, seus commits estarão no GitHub. Acesse a URL do
repositório para conferir.

8.4.2 Push subsequentes

Depois que o upstream está configurado, basta:
```

git push

O Git sabe que deve enviar a branch atual para a branch remota correspondente.

8.4.3 Enviando para outra branch remota

Se quiser enviar sua branch local feature-x para uma branch remota com nome diferente:

```
git push origin feature-x:outro-nome

8.4.4 Forçando o push (cuidado!)

Se você reescreveu o histórico local (com rebase ou amend) e precisa
sobrescrever a branch remota, use --force:
```

git push --force

Atenção: isso pode causar problemas para quem já baixou a branch. Use apenas em branches que são só suas e com comunicação à equipe.

8.4.5 Exercício prático 8.1

1. Crie um repositório vazio no GitHub (sem README).
2. No seu computador, crie uma pasta projeto-remoto, inicialize um repositório Git e faça um commit inicial (ex: README.md com uma linha).
3. Adicione o remote com a URL do GitHub.
4. Envie a branch main com git push -u origin main.
5. Verifique no GitHub se o arquivo apareceu.

8.5 Baixando Alterações: git pull e git fetch

Tão importante quanto enviar é receber as alterações que outras pessoas fizeram no repositório remoto.

8.5.1 git fetch – baixar sem integrar

O comando git fetch baixa as alterações do remoto, mas não as mescla na sua branch atual. Ele apenas atualiza as referências remotas (ex: origin/main).

```
git fetch origin

Depois do fetch, você pode comparar sua branch local com a remota:
```

git log HEAD..origin/main --oneline # commits que estão no remoto e não no local

8.5.2 git pull – baixar e integrar

O git pull é uma combinação de fetch seguido de merge (ou rebase, se configurado). Ele baixa as alterações e as incorpora à sua branch atual.

```
git pull origin main

Se você já tem upstream configurado, apenas git pull já funciona.
```

```
8.5.3 Modos de pull: merge vs. rebase

Por padrão, git pull faz um merge. Se preferir que ele faça rebase (para
evitar commits de merge), configure:
```

git config --global pull.rebase true

Ou use git pull --rebase em casos específicos.

8.5.4 Simulando um cenário de colaboração

1. Crie dois clones do mesmo repositório (ou use duas pastas diferentes).
2. No primeiro clone, faça um commit e push.
3. No segundo clone, faça um commit diferente e tente push (será rejeitado, pois o histórico divergiu).
4. No segundo clone, execute git pull para integrar as alterações do primeiro.
5. Agora faça push do segundo.

8.5.5 Exercício prático 8.2

Reproduza o cenário acima usando duas pastas no seu computador (ou com um colega). Observe as mensagens do Git e como ele lida com a divergência.

8.6 Buscando Alterações sem Aplicar: git fetch detalhado

O git fetch é mais seguro que o pull, pois permite que você veja o que mudou antes de decidir como integrar.

8.6.1 Atualizando todas as branches remotas

```
git fetch --all

Isso atualiza as referências de todos os remotos.

8.6.2 Visualizando diferenças após o fetch
```

git log main..origin/main # commits que estão em origin/main e não em main

git diff main origin/main # diff entre a branch local e a remota

8.6.3 Integrando manualmente após o fetch

Se preferir, depois do fetch você pode fazer o merge:

```
git merge origin/main

Ou rebase:
```

git rebase origin/main

8.6.4 Quando usar fetch em vez de pull?

- Quando você quer ver o que mudou antes de integrar.
- Quando você está em uma branch diferente e quer atualizar as refs remotas sem afetar o working directory.
- Em scripts ou automações, para evitar integração automática.

8.7 Clonando e Sincronizando Forks

No GitHub, um fork é uma cópia de um repositório de outra pessoa para sua conta. É a base do modelo de contribuição open source.

8.7.1 Fazendo um fork

1. Acesse um repositório público (ex: facebook/react).
2. Clique no botão "Fork" no canto superior direito.
3. Escolha sua conta (se houver mais de uma).
4. Aguarde – agora você tem uma cópia do repositório em github.com/seu-usuario/react.

8.7.2 Clonando seu fork

```
git clone https://github.com/HaroldoCapucci/react.git
cd react

Agora você tem o repositório localmente.

8.7.3 Adicionando o repositório original como upstream

Para manter seu fork sincronizado com o original (que pode ter novos
commits), adicione um segundo remote:
```

git remote add upstream https://github.com/facebook/react.git

Verifique:

```
git remote -v

Você verá:

· origin → seu fork
· upstream → repositório original

8.7.4 Sincronizando seu fork

Para trazer as alterações do original:
```

git fetch upstream

git checkout main

git merge upstream/main

git push origin main

Ou, em um único comando (se estiver na main):

```
git pull upstream main
git push origin main

8.7.5 Trabalhando com branches de feature

Crie uma branch para sua contribuição:
```

git switch -c minha-feature

Faça commits, push para origin e abra um Pull Request para o repositório original.

8.7.6 Exercício prático 8.3

1. Faça um fork de um repositório público qualquer (ex: github.com/octocat/Hello-World).
2. Clone seu fork.
3. Adicione o remote upstream apontando para o original.
4. Busque as alterações do upstream com git fetch upstream.
5. Mescle as alterações na sua main local e envie para seu fork.

8.8 Trabalhando com Múltiplos Remotos

Você pode ter quantos remotos quiser. Isso é útil para:

- Colaborar com diferentes forks.
- Fazer deploy para diferentes ambientes.
- Manter mirrors do repositório.

8.8.1 Adicionando um segundo remote

```
git remote add backup git@gitlab.com:haroldo/meu-projeto.git

Agora você pode enviar para ele:
```

git push backup main

8.8.2 Listando todos os remotos

```
git remote -v

8.8.3 Enviando para múltiplos remotos de uma vez

Não há um comando nativo, mas você pode criar um script ou usar:
```

git push origin main && git push backup main

8.8.4 Removendo um remote

```
git remote remove backup

---

8.9 Boas Práticas com Repositórios Remotos

1. Sempre faça pull antes de começar a trabalhar – evita divergências.
2. Commits pequenos e frequentes – facilita a integração.
3. Nunca force push em branches compartilhadas – a menos que toda a equipe
esteja ciente.
4. Use mensagens de commit claras – ajuda no entendimento do histórico
remoto.
5. Proteja branches principais – no GitHub, configure regras para exigir
revisões antes do merge.
6. Mantenha seu fork sincronizado – se você contribui com projetos open
source.

---

8.10 Operações Remotas no VS Code

O VS Code facilita muito o trabalho com remotos.

8.10.1 Publicando um repositório local

Se você tem um repositório local sem remote, pode publicá-lo diretamente:

1. Abra a guia Source Control (Ctrl+Shift+G).
2. Clique nos três pontos (...) e escolha "Publish Branch".
3. Escolha "Publicar repositório privado" ou "público".
4. O VS Code cria o repositório no GitHub (autenticado) e faz o push.

8.10.2 Clonando um repositório

Na tela inicial do VS Code, escolha "Clone Git Repository" e cole a URL. Ou
use a paleta de comandos (Ctrl+Shift+P) e digite "Git: Clone".

8.10.3 Sincronizando (pull + push)

Na barra inferior, há um ícone de sincronização (setas circulares) que
executa git pull seguido de git push. Útil para atualizar rapidamente.

8.10.4 Gerenciando remotos

Na guia Source Control, clique nos três pontos → "Remote" → "Add Remote".
Você pode adicionar, remover e renomear remotos graficamente.

8.10.5 Visualizando branches remotas

No explorador de branches (clique no nome da branch na barra inferior), você
vê as branches remotas listadas. Pode alternar para uma branch remota com um
clique.
```

```
8.10.6 Exercício prático 8.4

Use o VS Code para:

· Clonar um repositório seu (ou de exemplo)
· Fazer uma alteração, commit e push (usando os botões)
· Adicionar um novo remote via interface
· Sincronizar usando o ícone de setas

---

8.11 Resolução de Problemas Comuns

8.11.1 Push rejeitado (non-fast-forward)

Isso acontece quando o remoto tem commits que você não tem localmente.
Solução: git pull primeiro, resolva conflitos se houver, depois push.

8.11.2 "failed to push some refs"

Mensagem genérica. Geralmente é o caso acima. Leia a mensagem completa – o
Git costuma sugerir o que fazer.

8.11.3 "remote origin already exists"

Significa que você já tem um remote chamado origin. Se quiser mudar a URL:
```

git remote set-url origin nova-url

8.11.4 Esqueceu de configurar upstream no primeiro push

Se você fez git push sem -u e deu erro, pode configurar depois:

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 9: Colaboração com Forks e Pull Requests

Páginas estimadas: 30

Tempo de leitura: 75–95 minutos

Objetivos do capítulo:

- Compreender o modelo de colaboração baseado em forks e pull requests
- Aprender a fazer um fork de um repositório no GitHub
- Configurar o repositório local para trabalhar com o fork e o original
- Criar uma branch para sua contribuição e enviar alterações
- Abrir um Pull Request (PR) e descrevê■lo adequadamente
- Participar do processo de revisão de código
- Manter o fork sincronizado com o repositório original
- Utilizar o VS Code e GitHub CLI para agilizar o fluxo

9.1 Introdução

O GitHub não é apenas um lugar para guardar código; é uma plataforma social de colaboração. Milhares de projetos open source vivem no GitHub e recebem contribuições de pessoas do mundo todo. Mas como alguém que não tem permissão de escrita em um repositório pode contribuir? A resposta é o modelo **fork + pull request**.

- **Fork:** uma cópia do repositório original para a sua conta no GitHub. Você tem total liberdade para modificar esse fork.
- **Pull Request (PR):** uma solicitação para que o mantenedor do projeto original incorpore as alterações que você fez no seu fork.

Esse fluxo é a espinha dorsal da colaboração open source e também é amplamente usado em empresas para revisão de código antes de integrar mudanças à branch principal.

Neste capítulo, você aprenderá na prática como contribuir com projetos alheios e também como receber contribuições nos seus próprios projetos.

9.2 O Modelo de Colaboração: Fork e Pull Request

9.2.1 Fluxo básico

1. **Fork** o repositório desejado para sua conta.
2. **Clone** seu fork para o computador local.
3. **Adicione o repositório original como remote** (upstream) para manter-se sincronizado.
4. **Crie uma branch** para sua contribuição (ex: `feature/minha-mudanca`).
5. Faça as alterações, **commits** e **push** para o seu fork.
6. No GitHub, **abra um Pull Request** comparando sua branch com a branch principal do original.
7. Participe da **revisão**: discuta, faça ajustes se necessário.
8. Quando aprovado, o mantenedor **faz o merge** (ou você, se tiver permissão).
9. Após o merge, você pode **sincronizar seu fork** com o original e deletar a branch usada.

9.2.2 Por que esse modelo?

- **Segurança:** contribuidores não têm acesso direto ao repositório principal.
- **Qualidade:** todo código passa por revisão antes de ser integrado.
- **Rastreabilidade:** fica claro quem fez o quê e por quê.
- **Transparência:** discussões ficam públicas e documentadas.

9.3 Fazendo um Fork de um Projeto

9.3.1 Escolhendo um projeto

Para este capítulo, usaremos um repositório de exemplo: `octocat/Hello-World` (um repositório público criado pelo GitHub para testes). Você pode usar qualquer repositório público que desejar.

9.3.2 Executando o fork

1. Acesse o repositório no GitHub: `https://github.com/octocat/Hello-World`
2. No canto superior direito, clique no botão **"Fork"**.
3. Escolha sua conta pessoal como destino (se houver mais de uma).
4. Aguarde – o GitHub criará uma cópia exata do repositório em `https://github.com/seu-usuario/Hello-World`.

Agora você tem seu próprio fork. Ele contém todo o histórico, branches e tags do original.

9.3.3 Verificando o fork

No seu fork, você verá uma mensagem "forked from octocat/Hello-World" abaixo do nome do repositório, indicando a origem.

9.4 Clonando o Fork e Configurando o Upstream

9.4.1 Clonando o fork para o computador

```
git clone https://github.com/HaroldoCapucci/Hello-World.git
cd Hello-World

9.4.2 Adicionando o repositório original como remote upstream
```

git remote add upstream https://github.com/octocat/Hello-World.git

Verifique os remotos:

```
git remote -v

Saída esperada:

origin https://github.com/HaroldoCapucci/Hello-World.git (fetch)
origin https://github.com/HaroldoCapucci/Hello-World.git (push)
upstream https://github.com/octocat/Hello-World.git (fetch)
upstream https://github.com/octocat/Hello-World.git (push)

Agora você tem dois remotos:

· origin: seu fork (onde você tem permissão de escrita).
· upstream: o repositório original (somente leitura para você).

9.4.3 Importância do upstream

Manter o upstream configurado permite que você sincronize seu fork com as
```

```
atualizações do projeto original, evitando que sua cópia fique desatualizada.

---

9.5 Criando uma Branch para a Contribuição

Nunca trabalhe diretamente na branch main do seu fork. Crie uma branch
descritiva para sua contribuição.

9.5.1 Atualize sua main local com a upstream

Antes de criar a branch, certifique-se de que sua main local está igual à do
projeto original:
```

git switch main

git pull upstream main

git push origin main # opcional: atualizar seu fork no GitHub

9.5.2 Crie e mude para a nova branch

```
git switch -c minha-contribuicao

Use um nome que identifique o propósito, como fix/typo-readme,
feature/adiciona-botao, docs/atualiza-exemplo.

---

9.6 Fazendo as Alterações e Commit

9.6.1 Edite os arquivos necessários

Por exemplo, vamos corrigir um pequeno erro no README do repositório
Hello-World.

Abra o arquivo README (pode ser README.md) e faça uma alteração simples
(adicione uma linha, corrija um typo).

9.6.2 Commit das alterações
```

git add README.md

git commit -m "docs: corrige pequeno erro de digitação no README"

Lembre-se das boas práticas de mensagens de commit (capítulo 5).

9.6.3 Push para o seu fork

```
git push origin minha-contribuicao

Se for o primeiro push dessa branch, use -u para configurar upstream:
```

git push -u origin minha-contribuicao

Agora sua branch está no seu fork no GitHub.

9.7 Abrindo um Pull Request (PR)

9.7.1 Pelo site do GitHub

1. Acesse seu fork no GitHub: https://github.com/HaroldoCapucci/Hello-World
2. O GitHub geralmente mostra um banner sugerindo "minha-contribuicao had recent pushes" com um botão "Compare & pull request". Clique nele.

• Se não aparecer, vá até a aba "Pull requests" e clique no botão verde "New pull request".

3. Na página de comparação, verifique:

• base repository: octocat/Hello-World (o original)

• base: main (a branch do original que receberá as mudanças)

• head repository: HaroldoCapucci/Hello-World (seu fork)

• compare: minha-contribuicao (sua branch)

4. O GitHub mostrará um diff das alterações. Confira se está tudo correto.
5. Clique em "Create pull request".
6. Preencha o título e a descrição do PR (veja seção 9.8).
7. Clique novamente em "Create pull request".

Pronto! Seu PR está aberto e visível para os mantenedores do projeto original.

9.7.2 Pelo GitHub CLI

Se você tem o GitHub CLI instalado (capítulo 3), pode abrir o PR diretamente do terminal:

```
gh pr create --base main --head minha-contribuicao --title "docs: corrige
typo no README" --body "Correção de um pequeno erro de digitação."

O comando interativo também pode ser usado: gh pr create e siga as
instruções.

9.7.3 Pelo VS Code

Com a extensão "GitHub Pull Requests and Issues" instalada, você pode abrir
PRs diretamente do editor:

1. Abra a guia "Source Control".
2. Clique nos três pontos e escolha "Pull Request" → "Create Pull Request".
3. Preencha os campos na interface.

---

9.8 Escrevendo uma Boa Descrição de Pull Request

Uma descrição clara ajuda os revisores a entenderem sua contribuição e
acelera o processo.
```

```
9.8.1 Estrutura recomendada
```

Descrição

Breve explicação do que este PR faz. Se for uma correção de bug, descreva o problema. Se for uma nova funcionalidade, explique o motivo e o comportamento esperado.

Tipo de mudança

- [ ] Correção de bug
- [x] Nova funcionalidade
- [ ] Documentação
- [ ] Refatoração
- [ ] Outro (especifique)

Como testar

Passos para testar as alterações:

1. Acesse a página X
2. Clique no botão Y
3. Verifique se Z aparece

Checklist

- [x] Meu código segue as diretrizes do projeto
- [x] Eu testei minhas alterações
- [ ] Atualizei a documentação, se necessário
- [ ] Adicionei testes, quando aplicável

Issues relacionadas

Closes #123 (se este PR resolve uma issue)

9.8.2 Dicas

- Seja educado e grato.

- Referencie issues usando #numero.
- Use listas e cabeçalhos para organizar.
- Inclua prints ou GIFs se a mudança for visual.

9.9 Revisando Código: Comentários, Sugestões e Aprovações

Após abrir o PR, os mantenedores (e outros contribuidores) podem revisar seu código.

9.9.1 Tipos de comentários no GitHub

- Comentário geral: no topo do PR, sobre o conjunto.
- Comentário em linha: em uma linha específica do código (passe o mouse sobre a linha e clique no ícone de +).
- Sugestão: o revisor pode propor uma alteração usando o bloco de sugestão (suggestion), que você pode aceitar com um clique.

9.9.2 Respondendo a revisões

- Seja receptivo a críticas construtivas.
- Faça os ajustes solicitados na mesma branch, faça novos commits e push – o PR será atualizado automaticamente.
- Marque os comentários como resolvidos após implementar.
- Se discordar de algo, explique educadamente o motivo.

9.9.3 Aprovação e merge

Quando o revisor estiver satisfeito, ele aprovará o PR (botão "Approve"). O mantenedor então fará o merge (ou você, se tiver permissão). Existem opções de merge:

- Create a merge commit: mantém histórico completo.
- Squash and merge: combina todos os commits em um só.
- Rebase and merge: reescreve os commits na base.

9.10 Mantendo seu Fork Sincronizado

Enquanto seu PR está aberto, o projeto original pode receber outras contribuições. É importante manter seu fork atualizado para evitar conflitos.

9.10.1 Sincronização manual

```
git switch main
git pull upstream main
git push origin main

Se você ainda está trabalhando na branch de contribuição, pode mesclar as
atualizações da main nela:
```

git switch minha-contribuicao

git merge main # ou git rebase main

resolva conflitos, se houver

git push origin minha-contribuicao

9.10.2 Sincronização pelo GitHub

O GitHub oferece um botão "Fetch upstream" no seu fork, que permite atualizar a branch main diretamente pelo site. Após usar, você precisará dar pull localmente.

9.10.3 Automatizando com GitHub Actions

É possível configurar um workflow que mantém seu fork sincronizado automaticamente (tópico avançado, veremos no capítulo 14).

9.11 Boas Práticas para Contribuições

1. Comunique-se antes: se a mudança for grande, abra uma issue primeiro para discutir.
2. Siga as diretrizes do projeto: muitos têm CONTRIBUTING.md.
3. Faça commits atômicos: cada commit deve fazer uma única mudança lógica.
4. Escreva mensagens de commit claras.
5. Teste suas alterações.
6. Mantenha o foco: um PR deve resolver um problema ou adicionar uma funcionalidade, não várias ao mesmo tempo.
7. Seja paciente: mantenedores são voluntários e podem demorar a responder.

9.12 Recebendo Pull Requests no Seu Próprio Repositório

Se você mantém um projeto, receberá PRs de outras pessoas. O fluxo como revisor:

1. Analise o código, teste localmente se possível.
2. Use comentários e sugestões para pedir melhorias.
3. Quando estiver satisfeito, faça o merge.
4. Agradeça ao contribuidor.

9.12.1 Testando PRs localmente

Para testar um PR de outra pessoa, você pode fazer fetch da branch dela:

```
git fetch origin pull/ID/head:nome-local
git switch nome-local
```

```
Ou usar o GitHub CLI: gh pr checkout ID.

---

9.13 Exemplo Completo com VS Code e GitHub CLI

Vamos simular uma contribuição real usando as ferramentas.

9.13.1 Cenário

Repositório original: HaroldoCapucci/livro-git-completo (seu próprio livro).
Você quer corrigir um typo no capítulo 1.

9.13.2 Passos

1. Fork o repositório (se for seu, você já tem acesso, mas para treino, use
outro).
2. Clone o fork:
```

gh repo clone HaroldoCapucci/livro-git-completo # se tiver gh

```
3. Adicione upstream:
```

git remote add upstream https://github.com/HaroldoCapucci/livro-git-completo.git

```
4. Crie branch:
```

git switch -c fix/typo-cap1

```
5. Edite capitulos/cap01.md, corrija o typo.
6. Commit:
```

git add capitulos/cap01.md

git commit -m "fix: corrige pequeno erro de digitação no capítulo 1"

```
7. Push:
```

git push -u origin fix/typo-cap1

```
8. Abra PR (pelo gh):
```

gh pr create --base main --head fix/typo-cap1 --title "Correção de typo no capítulo 1" --body "Corrige um pequeno erro de digitação."

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 10: Gerenciamento de Projetos no GitHub

Páginas estimadas: 28

Tempo de leitura: 70–90 minutos

Objetivos do capítulo:

- Compreender as ferramentas de gerenciamento de projetos oferecidas pelo GitHub
- Utilizar Issues para rastrear tarefas, bugs e melhorias
- Criar e gerenciar milestones para agrupar issues
- Organizar o trabalho com labels e assignees
- Utilizar Projects (quadros Kanban) para visualizar o fluxo de trabalho
- Configurar automações para mover cards automaticamente
- Criar e manter wikis para documentação extensa
- Gerenciar releases e tags para versionamento
- Explorar GitHub Discussions para comunicação assíncrona

10.1 Introdução

O GitHub é muito mais do que um hospedeiro de código. Ele oferece um conjunto integrado de ferramentas de **gerenciamento de projetos** que ajudam equipes a planejar, acompanhar e entregar software de forma eficiente. Essas ferramentas são nativas da plataforma e se integram perfeitamente com o fluxo Git.

Dominar esses recursos permite:

- Organizar o trabalho de forma transparente
- Facilitar a comunicação entre membros da equipe
- Automatizar partes do fluxo de trabalho
- Manter a documentação próxima ao código
- Entregar versões com qualidade e rastreabilidade

Neste capítulo, exploraremos cada uma dessas ferramentas, desde o básico até configurações avançadas.

10.2 Utilizando Issues para Gerenciar Tarefas e Bugs

Issues são a unidade fundamental de trabalho no GitHub. Elas podem representar:

- Relatos de bugs
- Solicitações de novas funcionalidades
- Tarefas técnicas
- Discussões

10.2.1 Criando uma Issue

1. No repositório, clique na aba **"Issues"**.
2. Clique no botão verde **"New issue"**.
3. Escolha um template (se houver) ou comece do zero.
4. Preencha:

- **Title:** descrição concisa
- **Comment:** detalhamento, podendo usar Markdown

5. Opcionalmente, adicione **labels**, **assignees**, **projects**, **milestone**.
6. Clique em **"Submit new issue"**.

10.2.2 Estruturando uma Issue

Uma issue bem escrita aumenta a produtividade. Exemplo de issue para bug:

```
## Descrição
O botão "Salvar" não responde ao clique.

## Passos para reproduzir
1. Acesse a página de edição de perfil.
2. Altere o campo "Nome".
3. Clique em "Salvar".

## Comportamento esperado
Uma mensagem de sucesso deve aparecer.

## Ambiente
- SO: Windows 10
- Navegador: Chrome 98

## Logs
```

```
Erro no console: Uncaught TypeError: Cannot read property 'save' of undefined


Para uma nova funcionalidade:
```

Problema

Atualmente não há como recuperar a senha.

Solução proposta

Adicionar um link "Esqueci minha senha" na tela de login, que envia um e■mail com instruções.

Critérios de aceitação

- [ ] Link visível na tela de login
- [ ] Envio de e■mail funcional
- [ ] Token expira em 1 hora

10.2.3 Templates de Issues

Repositórios podem ter templates padronizados para bugs, features, etc. Eles ficam na pasta .github/ISSUE_TEMPLATE/ e são escritos em Markdown ou YAML.

Exemplo de template (simples) em .github/ISSUE_TEMPLATE/bug_report.md:

```
---
name: Relatório de bug
about: Crie um relatório para nos ajudar a melhorar
title: "[BUG] "
labels: bug
assignees: ''

---

**Descrição do bug**
Uma descrição clara e concisa do bug.

**Para reproduzir**
Passos:
1. Vá para '...'
2. Clique em '...'
3. Role até '...'
4. Veja o erro
```

```
**Comportamento esperado**
Descrição clara do que deveria acontecer.

**Screenshots**
Se aplicável, adicione imagens.

**Ambiente:**
- SO: [ex: Windows, macOS]
- Navegador: [ex: Chrome, Safari]
- Versão: [ex: 22]

**Contexto adicional**
Qualquer outra informação.
```

10.2.4 Labels (Rótulos)

Labels categorizam issues. O GitHub fornece algumas padrão (bug, enhancement, question, etc.), mas você pode criar as suas.

Criando labels: Vá em Issues → Labels → New label.

Exemplos úteis:

- bug – algo não funciona
- enhancement – nova funcionalidade
- documentation – melhoria na documentação
- good first issue – ideal para novos contribuidores
- help wanted – precisa de ajuda
- priority: high – prioridade alta
- status: blocked – aguardando algo

10.2.5 Milestones (Marcos)

Milestones agrupam issues com um objetivo comum e uma data alvo. Eles ajudam a planejar releases ou sprints.

Criando um milestone:

1. Em Issues → Milestones → New milestone.
2. Defina título, descrição e data de entrega.
3. Associe issues ao milestone (ao editar a issue, escolha o milestone).

10.2.6 Assignees (Responsáveis)

Atribuir uma issue a uma pessoa deixa claro quem está trabalhando nela. Uma issue pode ter vários assignees.

10.2.7 Exercício prático 10.1

No seu repositório de testes (ou no repositório do livro), crie:

- Duas labels personalizadas (ex: documentation, good first issue).
- Um milestone chamado "Versão 1.0" com data futura.

```
· Três issues: um bug, uma melhoria e uma tarefa de documentação. Atribua
labels, milestone e assignees adequadamente.

---

10.3 Projetos: Kanban Integrado (GitHub Projects)

Projects são quadros estilo Kanban que permitem visualizar o fluxo de
trabalho. Eles podem ser vinculados a um repositório ou a uma organização.

10.3.1 Tipos de Projects

O GitHub oferece dois tipos principais:

· Projects (clássico): o modelo antigo, baseado em colunas e cards.
· Projects (beta – novo): mais flexível, com campos personalizados,
visualizações e automações robustas. Neste capítulo, focaremos no novo
Projects (acessível em Projects na barra superior do
repositório/organização).

10.3.2 Criando um Project

1. No repositório, clique na aba "Projects" (ou no perfil da organização).
2. Clique em "New project".
3. Escolha um template (Board, Roadmap, etc.) ou comece do zero.
4. Dê um nome (ex: "Desenvolvimento do Livro").
5. Defina a visibilidade (privado/público).
6. Clique em "Create".

10.3.3 Estrutura do Project (beta)

O novo Projects é como uma planilha/database. Você pode ter visualizações
diferentes:

· Board view: colunas (como Kanban).
· Table view: lista tabular.
· Roadmap: cronograma.

As colunas no Board view são baseadas em um campo (ex: "Status") com opções
(To do, In progress, Done).

10.3.4 Adicionando issues ao Project

· Dentro do Project, clique em "Add item" e digite # para buscar issues do
repositório.
· Ou, na própria issue, no painel lateral direito, há a seção "Projects" para
adicioná-la a um projeto.

10.3.5 Personalizando campos

Você pode adicionar campos personalizados:

· Texto
· Número
```

```
· Data
· Seleção única (dropdown)
· Iteração (sprints)

Exemplo: campo "Prioridade" com valores Alta, Média, Baixa.

10.3.6 Automações (Workflows)

No Projects, você pode criar automações para mover cards automaticamente.
Exemplos:

· Quando uma issue é aberta, mover para "To do".
· Quando um PR é aberto, mover para "In review".
· Quando a issue é fechada, mover para "Done".

Para criar automações:

1. No Project, clique no menu do projeto (três pontos) e escolha "Workflows".
2. Crie um novo workflow com gatilhos e ações.

10.3.7 Vinculando Projects a repositórios

Projects podem ser organizacionais (visíveis em todos os repositórios da org)
ou vinculados a um repositório específico. No novo Projects, você escolhe ao
criar.

10.3.8 Exercício prático 10.2

1. Crie um Project (beta) no seu repositório chamado "Meu Kanban".
2. Configure a visualização Board com colunas: To do, In progress, Done.
3. Adicione as três issues criadas anteriormente ao Project.
4. Crie uma automação que mova issues para "In progress" quando um PR for
aberto (simule com um PR de teste).
5. Experimente mudar a visualização para Table e Roadmap.

---

10.4 Discussões e Wikis: Documentação Colaborativa

10.4.1 GitHub Discussions

Discussions é um fórum integrado ao repositório, ideal para:

· Perguntas e respostas
· Anúncios
· Ideias
· Conversas que não se encaixam em issues (muito abertas)

Ativando Discussions:
Nas configurações do repositório, marque a opção "Discussions".

Categorias: você pode criar categorias (ex: Q&A, Ideas, Show and tell) para
organizar as conversas.
```

```
Diferença entre Issues e Discussions: Issues são para trabalho rastreável
(bugs, tarefas). Discussions são para conversas e ideias que podem ou não se
transformar em issues.

10.4.2 Wikis

Wiki é um espaço de documentação livre, versionado em um repositório Git
separado (com extensão .wiki.git). Você pode criar páginas interligadas, em
Markdown.

Ativando Wiki: nas configurações do repositório, habilite Wikis.

Editando a Wiki: Você pode clonar o repositório da wiki e editar localmente,
ou editar diretamente pelo site.

Exemplo de uso: documentação detalhada do projeto, guias de contribuição,
FAQs.

10.4.3 Boas práticas para Wikis

· Mantenha uma página inicial (Home.md) com índice.
· Use links relativos entre páginas.
· Organize por categorias (ex: Instalação, Uso, API).
· Mantenha atualizada – assim como o código, a wiki precisa de manutenção.

10.4.4 Exercício prático 10.3

1. Ative Discussions no seu repositório e crie uma categoria "Dúvidas".
2. Inicie uma discussão perguntando algo sobre o livro (ex: "Qual capítulo
você achou mais difícil?").
3. Ative a Wiki e crie uma página "Guia de Contribuição" com Markdown.
4. Na página inicial da Wiki, adicione um link para essa página.

---

10.5 Gerenciando Releases e Tags no GitHub

Releases são versões empacotadas do seu projeto, geralmente associadas a
tags. Elas permitem disponibilizar arquivos binários (.zip, .exe) e notas de
versão.

10.5.1 Criando uma tag

Tags são referências a commits, geralmente usadas para marcar versões (v1.0,
v2.1.3). No Git:
```

git tag -a v1.0 -m "Versão 1.0 - Lançamento inicial"

git push origin v1.0

10.5.2 Criando uma Release pelo site

1. No repositório, vá em "Releases" (na aba Code).
2. Clique em "Create a new release" ou "Draft a new release".

3. Escolha a tag existente ou crie uma nova.
4. Preencha:
• Release title: título amigável (ex: "Versão 1.0 – Estável")
• Description: notas de versão em Markdown (destaque novas funcionalidades, correções, breaking changes)
5. Anexe arquivos binários (opcional).
6. Marque como pré■release se não for estável.
7. Clique em "Publish release".

10.5.3 Notas de versão (Release Notes)

Boas notas de versão incluem:

• Resumo das principais mudanças
• Lista de novas funcionalidades
• Correções de bugs
• Mudanças que quebram compatibilidade (se houver)
• Agradecimentos a contribuidores

O GitHub pode gerar notas automaticamente a partir dos commits desde a última tag, mas é melhor personalizar.

10.5.4 Automatizando releases com GitHub Actions

É possível criar workflows que geram releases automaticamente quando uma tag é criada (capítulo 14).

10.5.5 Exercício prático 10.4

1. Crie uma tag v0.1.0 no seu repositório.
2. Crie uma release associada a essa tag, com notas de versão descrevendo os primeiros capítulos do livro.
3. Publique a release e verifique como ela aparece na página do repositório.

10.6 Integrando com VS Code

O VS Code oferece suporte a algumas dessas funcionalidades via extensões.

10.6.1 Visualizando Issues e PRs

Com a extensão "GitHub Pull Requests and Issues", você pode:

• Listar issues atribuídas a você
• Criar issues diretamente do editor
• Comentar e revisar PRs
• Gerenciar labels e assignees

10.6.2 Acessando Projects

A extensão não tem suporte direto a Projects, mas você pode abrir o navegador rapidamente com o comando Ctrl+Shift+P → "GitHub: Open on GitHub".

10.6.3 Editando a Wiki localmente

Você pode clonar o repositório da wiki:

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 11: Git Avançado e Boas Práticas

Páginas estimadas: 32

Tempo de leitura: 80–100 minutos

Objetivos do capítulo:

- Dominar o rebase interativo para reescrever a história
- Utilizar `git stash` para guardar trabalho temporário
- Compreender e usar `git reflog` como rede de segurança
- Comparar merge e rebase: quando usar cada um
- Adotar boas práticas em mensagens de commit e organização
- Configurar `.gitignore` avançado para diferentes cenários
- Assinar commits com GPG para verificação de autenticidade
- Usar `git bisect` para encontrar commits que introduziram bugs
- Introdução a submódulos e seu gerenciamento

11.1 Introdução

Nos capítulos anteriores, você aprendeu os comandos essenciais do Git e como colaborar com outras pessoas. Agora é hora de elevar seu nível: mergulhar em funcionalidades avançadas que tornam o Git uma ferramenta ainda mais poderosa e flexível.

Dominar esses recursos permite:

- Corrigir erros no histórico de forma elegante
- Salvar temporariamente alterações sem commits
- Recuperar commits "perdidos"
- Manter um histórico limpo e compreensível
- Garantir a autenticidade do seu código
- Encontrar bugs de forma sistemática

Além disso, discutiremos boas práticas que transformam um usuário comum em um profissional que respeita as convenções e facilita a vida de quem colabora com ele.

11.2 Reescrevendo a História: `git rebase` Interativo

O rebase interativo (`rebase -i`) é uma das ferramentas mais poderosas do Git. Ele permite modificar, reorganizar, combinar ou deletar commits antes de compartilhá-los.

11.2.1 O que é rebase?

Rebase significa "mudar a base". Em vez de mesclar (merge), que cria um commit de junção, o rebase reaplica os commits de uma branch sobre outra, criando uma história linear.

Exemplo: se você tem uma branch `feature` baseada em um commit antigo de `main`, ao fazer `git rebase main` na branch `feature`, o Git pega os commits da `feature` e os coloca após o último commit de `main`, como se tivessem sido feitos depois.

11.2.2 Rebase interativo

O rebase interativo permite escolher o que fazer com cada commit durante o processo.

```
git rebase -i HEAD~3 # últimos 3 commits

O editor abrirá com uma lista como:

pick a1b2c3d Adiciona funcionalidade X
pick e4f5g6h Corrige bug na funcionalidade X
pick i7j8k9l Adiciona testes

Para cada linha, você pode substituir pick por comandos:

· pick (p) – mantém o commit como está
· reword (r) – mantém o conteúdo, mas altera a mensagem
· edit (e) – para modificar o commit (conteúdo ou mensagem)
· squash (s) – combina o commit com o anterior (a mensagem é mesclada)
· fixup (f) – combina com o anterior, descartando a mensagem
· drop (d) – remove o commit

11.2.3 Exemplos práticos

Corrigir mensagem de um commit antigo:
Altere pick para reword na linha desejada, salve e feche. O Git abrirá o
editor para a nova mensagem.

Combinar commits:
```

```
Suponha que os dois primeiros commits sejam muito atômicos e queira
combiná-los. Altere o segundo para squash ou fixup.

pick a1b2c3d Adiciona funcionalidade X
squash e4f5g6h Corrige bug na funcionalidade X
pick i7j8k9l Adiciona testes

Ao salvar, o Git combinará os commits e pedirá uma mensagem para o novo
commit único.

Remover um commit: troque pick por drop na linha.

11.2.4 Cuidados com rebase

· Nunca faça rebase em commits que já foram enviados para o remoto e
compartilhados com outros. Isso reescreve a história e causa problemas para
quem já tem a versão antiga.
· Use rebase localmente para organizar seu trabalho antes do push.
· Se precisar forçar o push após um rebase em uma branch que você é o único a
usar, use git push --force-with-lease (mais seguro que --force).

11.2.5 Exercício prático 11.1

1. Em um repositório de testes, crie uma sequência de 4 commits com mensagens
confusas.
2. Use git rebase -i HEAD~4 para:
· Corrigir a mensagem do segundo commit
· Combinar o terceiro e quarto commits em um só
· Remover o primeiro commit (apenas para treino)
3. Verifique o histórico com git log --oneline.

---

11.3 Salvando Trabalho Temporário: git stash

Imagine que você está no meio de uma alteração e precisa urgentemente mudar
de branch para corrigir um bug, mas não quer commitar código incompleto. O
git stash é a solução.

11.3.1 Comandos básicos

· git stash – guarda as alterações atuais (working directory e staging) e
reverte para o último commit.
· git stash list – lista os stashes salvos.
· git stash apply – aplica o stash mais recente, mas mantém na lista.
· git stash pop – aplica e remove da lista.
· git stash drop – remove um stash específico.
· git stash clear – remove todos os stashes.

11.3.2 Stash com nome
```

git stash push -m "WIP: ajustes na validação"

11.3.3 Stash parcial

Se quiser guardar apenas arquivos específicos:

```
git stash push -m "ajustes no README" README.md

11.3.4 Recuperando stashes
```

git stash apply stash@{2} # aplica o stash de índice 2

11.3.5 Criando branch a partir de um stash

Se você aplicou um stash e decidiu que ele merece uma branch própria:

```
git stash branch nova-branch

11.3.6 Exercício prático 11.2

1. Em um repositório, modifique dois arquivos diferentes.
2. Use git stash para guardar as alterações.
3. Verifique que os arquivos voltaram ao estado do último commit.
4. Liste os stashes.
5. Aplique o stash com pop.
6. Repita o processo, mas desta vez crie uma branch a partir do stash.

---

11.4 Comandos de Emergência e Recuperação: git reflog

O reflog é um registro de todas as movimentações do ponteiro HEAD no seu
repositório local. É a "rede de segurança" do Git: mesmo commits que parecem
perdidos podem ser encontrados aqui.

11.4.1 Visualizando o reflog
```

git reflog

Saída típica:

a1b2c3d HEAD@{0}: commit: Adiciona funcionalidade X

e4f5g6h HEAD@{1}: rebase -i (finish): refatora histórico

i7j8k9l HEAD@{2}: commit: Corrige bug

Cada entrada mostra o hash do commit, a ação e uma referência tipo HEAD@{n}.

11.4.2 Recuperando um commit "perdido"

Suponha que você fez um git reset --hard e perdeu commits. No reflog, você encontra o hash do commit anterior e pode voltar:

```
git checkout a1b2c3d
# ou criar uma branch a partir dele
git branch recuperacao a1b2c3d

11.4.3 Entendendo o reflog
```

```
O reflog é local e expira após 90 dias (configurável). Ele não é
compartilhado com o remoto.

11.4.4 Exercício prático 11.3

1. Faça alguns commits e depois um git reset --hard HEAD~2.
2. Use git reflog para encontrar os commits anteriores.
3. Recupere um deles criando uma nova branch.

---

11.5 Merge vs. Rebase: Quando Usar Cada Um

A discussão entre merge e rebase é clássica. Ambos integram mudanças, mas de
formas diferentes.

11.5.1 Merge

· Cria um commit de merge (com dois pais).
· Preserva a história exata de quando as branches divergiram e foram unidas.
· Mais fiel à realidade temporal.
· Pode poluir o histórico com muitos commits de merge.

11.5.2 Rebase

· Reescreve a história, aplicando os commits da branch sobre a base mais
recente.
· Gera uma história linear, mais fácil de ler.
· Evita commits de merge.
· Perde o contexto de quando a branch foi criada.

11.5.3 Boas práticas

· Use merge em branches compartilhadas (ex: main recebendo develop) para
preservar o contexto.
· Use rebase em branches locais ou de feature antes de integrar, para manter
a história limpa.
· Nunca faça rebase de branches públicas que outras pessoas podem ter usado.

11.5.4 Integração com git pull

Você pode configurar o git pull para fazer rebase em vez de merge:
```

git config --global pull.rebase true

Assim, git pull equivale a git fetch + git rebase.

11.5.5 Exercício prático 11.4

1. Crie duas branches a partir de um mesmo ponto.
2. Faça commits em ambas.
3. Em uma, faça merge da outra. Observe o histórico.

4. Na outra (recriando a situação), faça rebase. Compare os históricos com git log --graph.

11.6 Boas Práticas com .gitignore

Já vimos o básico do .gitignore. Agora, vamos aprofundar.

11.6.1 Padrões avançados

- **/logs – ignora qualquer pasta logs em qualquer nível.
- *.log – ignora todos os arquivos .log.
- !important.log – exclui o arquivo important.log da regra anterior (exceção).
- build/ – ignora a pasta build na raiz.
- **/__pycache__/ – ignora pastas __pycache__ em qualquer nível.

11.6.2 Ignorar arquivos já rastreados

Se um arquivo já está sendo versionado e você quer ignorá-lo a partir de agora, precisa removê-lo do índice:

```
git rm --cached arquivo.conf

Depois adicione ao .gitignore.

11.6.3 Modelos prontos

O GitHub mantém uma coleção de .gitignore templates em github/gitignore. Você
pode baixar o específico para sua linguagem.

11.6.4 .gitignore global

Para ignorar arquivos em todos os repositórios (ex: .DS_Store no macOS,
backups de editor), crie um arquivo global:
```

git config --global core.excludesfile ~/.gitignore_global

Nesse arquivo, coloque padrões como:

.DS_Store

*.swp

*.swo

Thumbs.db

11.7 Assinatura de Commits com GPG

Para garantir que um commit realmente foi feito por você (e não por alguém se passando), você pode assinar commits com GPG.

11.7.1 Gerando uma chave GPG

No Linux/macOS, instale o GPG (se necessário) e gere a chave:

```
gpg --full-generate-key

Escolha o tipo RSA (4096 bits) e preencha seu nome e e-mail (o mesmo do Git).

11.7.2 Listando chaves e obtendo o ID
```

gpg --list-secret-keys --keyid-format LONG

Saída:

sec rsa4096/AAAAAAAAAAAAAAAA 2023-01-01 [SC]

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

uid [ultimate] Haroldo Capucci <joseharoldocapucci@gmail.com>

O ID é AAAAAAAAAAAAAAAA (após a barra).

11.7.3 Exportando a chave pública

```
gpg --armor --export AAAAAAAAAAAAAAAA

Copie a saída, incluindo as linhas -----BEGIN PGP PUBLIC KEY BLOCK----- e
-----END-----.

11.7.4 Adicionando a chave pública ao GitHub

1. GitHub → Settings → SSH and GPG keys → New GPG key.
2. Cole a chave pública.
3. Adicione.

11.7.5 Configurando o Git para assinar commits
```

git config --global user.signingkey AAAAAAAAAAAAAAAA

git config --global commit.gpgsign true # assina todos os commits por padrão

11.7.6 Assinando commits manualmente

Se não quiser assinar todos, use git commit -S para assinar um commit específico.

11.7.7 Verificando assinaturas

No GitHub, commits assinados mostram um selo "Verified". Localmente:

```
git log --show-signature

11.7.8 Exercício prático 11.5

1. Gere uma chave GPG (se não tiver).
2. Configure o Git para usá-la.
3. Faça um commit assinado e veja o selo no GitHub.
```

```
4. Verifique a assinatura localmente.

---

11.8 Encontrando Bugs com git bisect

git bisect faz uma busca binária no histórico para encontrar o commit que
introduziu um bug.

11.8.1 Fluxo básico

1. Inicie o bisect: git bisect start
2. Marque um commit bom (sem o bug): git bisect good <commit>
3. Marque um commit ruim (com o bug): git bisect bad <commit>
4. O Git checkout um commit intermediário. Teste se o bug existe.
5. Informe: git bisect good ou git bisect bad
6. Repita até que o Git aponte o commit culpado.
7. Finalize: git bisect reset

11.8.2 Automatizando com script

Se você tem um script de teste que retorna 0 para sucesso e !=0 para falha:
```

git bisect start HEAD <commit-bom>

git bisect run npm test # ou pytest, make test, etc.

11.8.3 Exercício prático 11.6

1. Em um repositório, introduza um bug em um commit antigo (ex: quebre uma função).
2. Use git bisect para encontrar o commit que introduziu o bug.
3. Após encontrar, finalize e corrija o bug.

11.9 Submódulos (Submodules)

Submódulos permitem incluir um repositório Git dentro de outro. Útil quando você quer usar uma biblioteca externa ou compartilhar código entre projetos.

11.9.1 Adicionando um submódulo

```
git submodule add https://github.com/usuario/biblioteca.git libs/biblioteca

Isso cria um arquivo .gitmodules e clona o repositório na pasta especificada.

11.9.2 Clonando um repositório com submódulos
```

git clone --recursive https://github.com/meu/projeto.git

Se já clonou sem --recursive, atualize:

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 12: Git Inside VS Code – A Interface Gráfica

Páginas estimadas: 26

Tempo de leitura: 65–85 minutos

Objetivos do capítulo:

- Conhecer a integração nativa do Git no VS Code
- Navegar pela guia Controle de Código Fonte (Source Control)
- Realizar operações comuns (commit, pull, push, branch) sem usar o terminal
- Utilizar o editor de merge integrado para resolver conflitos visualmente
- Explorar extensões que potencializam o Git (GitLens, Git History, Git Graph)
- Configurar o VS Code para um fluxo Git eficiente
- Aprender atalhos de teclado e dicas de produtividade

12.1 Introdução

O Visual Studio Code se tornou um dos editores mais populares entre desenvolvedores, e uma das razões é sua excelente integração com Git. Embora o terminal seja poderoso, a interface gráfica do VS Code oferece uma maneira mais intuitiva e rápida de executar tarefas cotidianas, especialmente para quem está começando.

Neste capítulo, você aprenderá a usar todas as funcionalidades Git diretamente da interface do VS Code, desde o básico até operações mais avançadas, além de conhecer extensões que transformam o editor em uma verdadeira central de controle de versionamento.

12.2 Visão Geral da Guia Controle de Código Fonte

12.2.1 Acessando a guia

A guia **Source Control** (Controle de Código Fonte) é o coração da integração Git no VS Code. Você pode abri-la de três formas:

- Clique no ícone de branch na barra lateral esquerda (terceiro ícone, geralmente).
- Use o atalho `Ctrl+Shift+G` (Windows/Linux) ou `Cmd+Shift+G` (macOS).
- Através da paleta de comandos (`Ctrl+Shift+P`), digite "View: Show Source Control".

12.2.2 Layout da guia

A guia é dividida em seções:

1. **Changes (Alterações):** arquivos modificados no working directory (ainda não adicionados ao staging).
2. **Staged Changes (Alterações preparadas):** arquivos que já foram adicionados (`git add`) e estão prontos para commit.
3. **Merge Changes (Alterações de merge):** aparece apenas durante conflitos, listando os arquivos com conflitos.
4. **Timeline (linha do tempo):** na parte inferior do explorador, mostra eventos como commits e salvamentos (não exclusivo do Git).

Cada arquivo listado pode ter ações representadas por ícones:

- `+` (ou `U`): adicionar ao staging (ou começar a rastrear).
- `-` (ou `R`): remover do staging (reverter para modified).
- ■ (desfazer): descartar alterações no arquivo (cuidado!).
- `D` (diff): abrir o editor de comparação.

12.2.3 Mensagem de commit

Na parte superior da guia, há um campo de texto para digitar a mensagem do commit. Abaixo dele, um botão com ícone de "check" (✔) confirma o commit. Ao lado, um botão com "..." abre um menu com opções adicionais (pull, push, branch, etc.).

12.3 Realizando Commits, Pulls e Pushes pela Interface

12.3.1 Fazendo um commit

Passos:

1. Modifique ou crie arquivos no seu projeto.
2. Na guia Source Control, os arquivos aparecem em "Changes".
3. Passe o mouse sobre um arquivo e clique no `+` para adicioná-lo ao staging. Você pode adicionar todos de uma vez clicando no `+` que aparece ao lado de "Changes".
4. Digite uma mensagem de commit no campo superior.
5. Clique no ícone de "check" (✔) ou pressione `Ctrl+Enter` (Windows/Linux) / `Cmd+Enter` (macOS).
6. O commit é criado localmente.

Dica: Se você quitar fazer um commit direto (sem staging explícito), pode usar a opção "Commit" no menu de três pontos, que faz o commit de todas as alterações (equivalente a `git commit -a`). Mas cuidado: isso inclui todos os arquivos modificados, mesmo os que você não quer.

12.3.2 Sincronizando com o remoto (pull/push)

No canto inferior esquerdo da janela do VS Code, há uma barra de status que mostra a branch atual e ícones de sincronização:

- Ícone de setas circulares: sincronizar (pull + push).
- Ícone de nuvem com seta para baixo: pull.
- Ícone de nuvem com seta para cima: push.

Você também pode acessar essas ações pelo menu de três pontos na guia Source Control:

- **Pull, Push** – comandos individuais.
- **Sync** – executa pull seguido de push.

Importante: Se houver conflitos durante o pull, o VS Code notificará e você precisará resolvê-los antes de continuar.

12.3.3 Publicando um repositório local

Se você tem um repositório local sem remote, o VS Code oferece a opção "Publish to GitHub" no menu de três pontos. Ele cria um repositório no GitHub (público ou privado) e faz o push inicial. Você precisa estar autenticado no GitHub (extensão GitHub Authentication).

12.3.4 Clonando um repositório

Na tela inicial do VS Code, há a opção "Clone Git Repository". Basta colar a URL e escolher uma pasta. O VS Code abrirá o repositório automaticamente.

12.3.5 Exercício prático 12.1

1. Crie um novo projeto local (ou use um existente).
2. Faça algumas alterações e commit usando apenas a interface do VS Code.
3. Adicione um remote (se não tiver) via menu de três pontos → "Remote" → "Add Remote".
4. Faça um push.
5. Simule uma alteração no GitHub (edite um arquivo diretamente pelo site) e depois faça pull no VS Code.

12.4 Gerenciando Branches Visualmente

12.4.1 Visualizando branches

Clique no nome da branch na barra inferior esquerda (ex: `main`). Uma lista suspensa aparece mostrando:

- **Branches locais**
- **Branches remotas** (precedidas por `origin/`)
- Opção para criar nova branch

12.4.2 Criando e trocando de branch

- Para criar uma nova branch a partir da atual, clique em "Create new branch..." na lista e dê um nome.
- Para trocar, clique no nome da branch desejada na lista.
- Se houver alterações não commitadas, o VS Code perguntará o que fazer (stash, commit, descartar).

12.4.3 Publicando uma branch

Quando você cria uma branch local e tenta fazer push, o VS Code pergunta se deseja publicá-la (configurar upstream). Basta confirmar.

12.4.4 Excluindo branches

No menu de três pontos da guia Source Control, escolha "Branch" → "Delete Branch...". Selecione a branch a ser deletada. O VS Code impede a exclusão da branch atual.

12.4.5 Fazendo merge

Para mesclar uma branch na atual:

1. Certifique-se de estar na branch de destino (ex: `main`).
2. Menu de três pontos → "Branch" → "Merge Branch...".
3. Escolha a branch a ser mesclada.
4. Se houver conflitos, o VS Code entra em modo de resolução (ver seção 12.5).

12.4.6 Exercício prático 12.2

1. Crie uma branch `feature/nova` a partir da `main`.
2. Faça um commit nela.
3. Publique a branch no GitHub.
4. Volte para `main`, faça uma alteração diferente e commit.
5. Mescle `feature/nova` na `main` usando a interface. Resolva conflitos se necessário.

12.5 Resolvendo Conflitos com o Editor de Merge Integrado

12.5.1 Identificando conflitos

Quando um merge ou pull resulta em conflitos, o VS Code:

- Exibe um alerta.
- Na guia Source Control, aparece a seção **Merge Changes** listando os arquivos conflitantes.
- Os arquivos também são marcados no explorador com um ícone de alerta.

12.5.2 Abrindo o editor de merge

Clique duas vezes em um arquivo conflitante. O VS Code abre uma visualização dividida em três partes:

- **Esquerda:** versão atual (HEAD) – a sua branch.
- **Direita:** versão de entrada (incoming) – a branch que está sendo mesclada.
- **Centro:** resultado da mesclagem (onde você edita).

Acima do centro, há botões para aceitar:

- **Accept Current Change** – mantém a versão da esquerda.
- **Accept Incoming Change** – mantém a da direita.
- **Accept Both Changes** – combina as duas (cada uma em sua linha).
- **Compare Changes** – visualiza as diferenças.

12.5.3 Resolvendo manualmente

Você também pode editar diretamente a área central, removendo os marcadores de conflito (`<<<<<<<`, `=======`, `>>>>>>>`) e deixando o código desejado.

12.5.4 Finalizando a resolução

Após resolver todos os conflitos em todos os arquivos, você deve:

1. Adicionar os arquivos resolvidos ao staging (clique no `+` ao lado de cada arquivo em "Merge Changes" ou no `+` geral).
2. Digitar uma mensagem de commit (geralmente o VS Code já preenche uma padrão).
3. Clicar no ícone de "check" para concluir o merge.

12.5.5 Abortando um merge

Se a confusão for grande, você pode abortar o merge: menu de três pontos → "Merge" → "Abort Merge".

12.5.6 Exercício prático 12.3

1. Provocando um conflito (como no capítulo 6).
2. Use o editor de merge do VS Code para resolver.
3. Finalize o merge e commit.

12.6 Visualizando Histórico com Extensões

O VS Code nativo mostra o histórico de forma limitada (apenas o último commit via Timeline). Para uma experiência rica, precisamos de extensões.

12.6.1 GitLens — Git supercharged

GitLens é a extensão mais popular para Git no VS Code. Ela adiciona uma infinidade de funcionalidades:

- **Blame inline:** mostra, ao lado de cada linha, quem a modificou pela última vez e quando.
- **Explorador de commits:** uma nova guia na barra lateral com histórico, branches, stashes, etc.
- **Navegação:** links para commits, autores, repositórios remotos.
- **Comparação fácil:** selecione um trecho e veja sua evolução.
- **GitLens Inspect:** painel detalhado do commit atual.

Instalação: procure por "GitLens" na guia de extensões.

12.6.2 Git History

Essa extensão (de donjayamanne) adiciona uma visualização gráfica do histórico. Para usá-la:

1. Abra a paleta de comandos (`Ctrl+Shift+P`).
2. Digite "Git: View History" ou "Git: View File History".
3. Uma nova aba mostra o gráfico de commits, permitindo filtrar por autor, branch, etc.

12.6.3 Git Graph

Outra extensão muito boa: **Git Graph** (de mhutchie). Ela exibe um gráfico interativo de commits no estilo `git log --graph`. Você pode:

- Clicar em commits para ver detalhes.
- Criar branches, tags, fazer merge, checkout, etc. diretamente do gráfico.
- Ver os commits de todas as branches.

12.6.4 Git Blame

Extensão mais leve que o GitLens, focada apenas em blame inline. Se você quer algo simples, pode ser suficiente.

12.6.5 Exercício prático 12.4

1. Instale GitLens e Git Graph.
2. Explore o blame inline: abra um arquivo e veja as anotações.
3. Abra o Git Graph e navegue pelo histórico do seu repositório.
4. Use o GitLens para comparar duas versões de um arquivo.

12.7 Outras Extensões Úteis

12.7.1 GitHub Pull Requests and Issues

Extensão oficial da Microsoft que integra PRs e issues do GitHub. Permite:

- Listar, revisar e mergear PRs.
- Criar issues.
- Comentar e aprovar revisões.
- Checkout de PRs localmente.

12.7.2 Remote Repositories

Outra extensão oficial que permite abrir repositórios remotos (GitHub, Azure) diretamente no VS Code, sem clonar localmente. Útil para revisões rápidas.

12.7.3 GitLens (já mencionado)

Vale reforçar que o GitLens também integra funcionalidades de PRs e issues.

12.7.4 Gitmoji

Para quem gosta de usar emojis em commits, a extensão Gitmoji facilita a inserção de emojis padronizados.

12.8 Configurações do VS Code para Melhorar a Experiência Git

12.8.1 Editor de diff

O VS Code tem um excelente editor de diff. Você pode configurar para que ele seja a ferramenta padrão de diff do Git (como visto no capítulo 2). Além disso, pode ajustar:

- `git.enableSmartCommit`: se verdadeiro, permite commit sem staging explícito (equivale a `git commit -a`). Cuidado!
- `git.autofetch`: faz fetch automático periodicamente.

- `git.confirmSync`: pede confirmação antes de sincronizar.
- `git.untrackedChanges`: como exibir mudanças em arquivos não rastreados.

12.8.2 Atalhos de teclado

- `Ctrl+Shift+G` – abre Source Control.
- `Ctrl+Enter` – faz commit (com mensagem).
- `Ctrl+Shift+G` seguido de `Ctrl+P` – abre paleta de comandos Git.
- `Alt+Shift+F` – formata documento (útil para Markdown).

Você pode personalizar atalhos em File → Preferences → Keyboard Shortcuts.

12.8.3 Configurando o Git Bash como terminal padrão

Para ter uma experiência consistente, configure o terminal integrado para usar Git Bash (Windows) ou bash padrão (Linux/macOS). Isso permite executar comandos Git no terminal do VS Code com o mesmo ambiente.

12.9 Dicas de Produtividade

1. **Use a barra de status:** ela mostra branch, sincronização, e você pode clicar para ações rápidas.
2. **Stage hunk (parte de um arquivo):** no diff, você pode clicar com botão direito em um trecho e escolher "Stage Selected Ranges" para adicionar apenas parte das alterações ao staging.
3. **Desfazer staging rapidamente:** clique no `-` ao lado do arquivo em Staged Changes.
4. **Compare arquivos:** no explorador, selecione um arquivo, clique com botão direito e "Select for Compare", depois escolha outro arquivo e "Compare with Selected".
5. **Abra o repositório no GitHub:** com a extensão GitHub, você pode usar o comando "GitHub: Open on GitHub" para abrir rapidamente o repositório no navegador.
6. **Use snippets de commit:** se você usa Conventional Commits, crie snippets no VS Code para agilizar.

12.10 Exercícios de Fixação

1. Qual atalho abre a guia Source Control no VS Code?

2. Como você adiciona um arquivo ao staging usando a interface?

3. Explique como resolver um conflito de merge usando o editor de merge do VS Code.

4. Cite três extensões que melhoram a experiência Git no VS Code e suas principais funções.

5. Como você publica uma branch local no GitHub pelo VS Code?

6. O que é GitLens e qual sua funcionalidade mais marcante?

7. Qual a diferença entre as extensões Git History e Git Graph?

8. Como você pode visualizar o histórico de um arquivo específico no VS Code?

9. O que faz a configuração `git.enableSmartCommit`?

10. (Desafio) Instale a extensão "GitHub Pull Requests and Issues", autentique-se e tente revisar um PR de um repositório público (ou crie um PR fictício no seu repositório e revise-o).

12.11 Resumo do Capítulo

- O VS Code oferece integração Git nativa através da guia Source Control.
- É possível realizar todas as operações básicas (commit, pull, push, branch) sem terminal.
- O editor de merge integrado simplifica a resolução de conflitos.
- Extensões como GitLens, Git Graph e Git History potencializam a visualização do histórico e outras funcionalidades.
- Configurações e atalhos adequados aumentam a produtividade.

Agora você pode trabalhar com Git diretamente do seu editor favorito, combinando o poder do terminal com a conveniência da interface gráfica. No próximo capítulo, exploraremos o GitHub Copilot, o assistente de IA que pode gerar código e documentação, integrado ao VS Code.

12.12 Referências

- VS Code Docs: "Using Git source control"
- GitLens Documentation: https://gitlens.amod.io
- Git Graph – Visual Studio Marketplace
- GitHub Pull Requests and Issues – Visual Studio Marketplace
- "Visual Studio Code: End-to-End Git Integration" – blog da Microsoft

---

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 13: Desenvolvimento com GitHub Copilot e IA

Páginas estimadas: 28

Tempo de leitura: 70–90 minutos

Objetivos do capítulo:

- Compreender o que é GitHub Copilot e como ele funciona
- Instalar e configurar o Copilot no VS Code
- Dominar as sugestões de código inline (autocomplete)
- Utilizar o Copilot Chat para interações naturais
- Explorar comandos como /doc, /explain, /fix, /tests
- Aprender a usar o @workspace para consultar todo o projeto
- Conhecer o Agent Mode para tarefas autônomas
- Adotar boas práticas e evitar armadilhas comuns
- Integrar o Copilot ao fluxo de desenvolvimento com Git

13.1 Introdução

A inteligência artificial está transformando a maneira como desenvolvemos software. O **GitHub Copilot**, criado pela GitHub em parceria com a OpenAI, é um assistente de programação baseado em IA que funciona diretamente dentro do seu editor. Ele não é um simples autocomplete: entende o contexto do seu código, suas intenções expressas em comentários e até mesmo conversas em linguagem natural, gerando desde trechos simples até funções completas, testes e documentação .

Neste capítulo, você aprenderá a usar o Copilot como um verdadeiro "par de programação" (pair programmer) . Veremos como instalá-lo, configurá-lo e, principalmente, como se comunicar com ele para extrair o máximo de produtividade, sempre com olhar crítico sobre o código gerado.

13.2 O que é GitHub Copilot?

13.2.1 Definição

GitHub Copilot é um serviço de IA que sugere código e funções inteiras em tempo real, diretamente no seu editor . Ele foi treinado em bilhões de linhas de código público de repositórios GitHub, abrangendo centenas de linguagens e frameworks .

13.2.2 Como funciona?

O Copilot analisa o contexto do seu projeto:

- O arquivo que você está editando
- Outros arquivos abertos no editor
- Comentários escritos em linguagem natural
- Nomes de funções e variáveis
- Estrutura do projeto

Com base nesse contexto, ele utiliza modelos de linguagem avançados para prever e sugerir o próximo trecho de código .

13.2.3 Planos e pré-requisitos

Para usar o Copilot, você precisa :

- Uma conta no GitHub
- Uma assinatura ativa (há plano Free com limitações mensais, ou trial de 30 dias)
- VS Code instalado
- Conexão estável com a internet (as sugestões são processadas na nuvem)

Nota: Organizações podem gerenciar assinaturas para equipes. Verifique com seu administrador se estiver em ambiente corporativo .

13.3 Instalação e Configuração

13.3.1 Instalando a extensão

1. Abra o VS Code.
2. Acesse a guia de Extensões (`Ctrl+Shift+X` ou `Cmd+Shift+X` no Mac).
3. Pesquise por "GitHub Copilot".

4. Escolha a extensão oficial (publicada por GitHub) e clique em **Install** .

Recomenda-se instalar também a extensão **GitHub Copilot Chat**, que adiciona a interface de conversa (embora versões recentes já incluam ambas em um único pacote) .

13.3.2 Autenticando

Após a instalação:

1. Um ícone do Copilot aparecerá na barra de status (canto inferior direito).
2. Clique no ícone ou aguarde a notificação para fazer login.
3. Siga o fluxo OAuth no navegador, autorizando o acesso à sua conta GitHub .
4. Após autorizar, volte ao VS Code – a autenticação estará completa.

Se precisar forçar o login: abra a Paleta de Comandos (`Ctrl+Shift+P`) e procure por "GitHub Copilot: Sign In" .

13.3.3 Verificando a ativação

- O ícone na barra de status deve mostrar "Copilot: Ready" (sem nenhum aviso).
- Abra um arquivo de código e comece a digitar. Sugestões em "cinza fantasma" (ghost text) devem aparecer .

13.3.4 Configurações iniciais recomendadas

Acesse as configurações do VS Code (`Ctrl+,`) e pesquise por "Copilot". Alguns ajustes úteis :

- `github.copilot.enable`: ativar/desativar para linguagens específicas.
- `editor.inlineSuggest.enabled`: manter ativado para ver sugestões inline.
- `github.copilot.editor.enableAutoCompletions`: controla se as sugestões aparecem automaticamente.
- `github.copilot.advanced`: permite ajustar delay de sugestões, útil em projetos grandes para evitar "ruído".

13.4 Sugestões Inline: O Coração do Copilot

13.4.1 Como aparecem as sugestões

Enquanto você digita, o Copilot exibe sugestões em texto cinza (ghost text) à frente do cursor . Elas podem ser:

- Uma única linha
- Múltiplas linhas (funções inteiras)
- Blocos de código (loops, condicionais, etc.)

13.4.2 Aceitando sugestões

- **Tab**: aceita a sugestão completa .
- **Ctrl+**→ (Cmd+→ no Mac): aceita palavra por palavra .

13.4.3 Navegando entre alternativas

Se a primeira sugestão não for ideal, o Copilot geralmente tem outras opções :

- **Alt+] (Option+] no Mac)**: próxima sugestão.
- **Alt+[(Option+[no Mac)**: sugestão anterior.

13.4.4 Rejeitando ou ignorando

- **Esc**: descarta a sugestão atual.
- Simplesmente continue digitando; a sugestão desaparecerá e o Copilot tentará se adaptar ao que você escreve .

13.4.5 Forçando uma sugestão

Se nada aparecer e você quiser uma sugestão, use `Alt+\` (Option+\ no Mac) para acionar manualmente .

13.4.6 Exercício prático 13.1

1. Crie um arquivo `calculadora.js`.
2. Digite o comentário: `// Função que soma dois números`.
3. Observe a sugestão automática. Aceite com Tab.
4. Experimente navegar por alternativas com Alt+] e Alt+[.
5. Crie um arquivo `index.html` e digite `!` (ponto de exclamação) – o Copilot pode sugerir a estrutura básica do HTML5.

13.5 Trabalhando com Comentários Inteligentes

Uma das técnicas mais poderosas para guiar o Copilot é usar comentários descritivos .

13.5.1 Escrevendo intenções claras

Quanto mais específico o comentário, melhor a sugestão.

Exemplo vago:

```
// processar dados

Exemplo eficaz:
```

// Recebe um array de números, filtra apenas os pares e retorna a soma deles

O Copilot frequentemente gera o código completo para essa descrição .

13.5.2 Especificando entradas e saídas

```
// Função que valida e-mail: retorna true se o formato for válido, false caso
contrário

13.5.3 Descrevendo algoritmos
```

// Ordena uma lista de objetos por data de criação, do mais recente para o mais antigo

13.5.4 Exercício prático 13.2

Crie um arquivo desafio.js e escreva comentários detalhados para:

- Uma função que calcula fatorial.
- Uma função que valida CPF (apenas a lógica, mesmo que simplificada).
- Uma função que busca dados de uma API e trata erros.

Observe como o Copilot reage a cada nível de detalhe.

13.6 Copilot Chat: Conversando com o Assistente

O Copilot Chat traz uma interface de conversa para dentro do VS Code, permitindo diálogos mais complexos .

13.6.1 Abrindo o Chat

- Atalho: Ctrl+Alt+I (Windows/Linux) ou Cmd+Option+I (Mac) .
- Ícone: Clique no ícone do Copilot na barra lateral (se disponível).
- Paleta de comandos: "Copilot Chat: Open Chat" .

13.6.2 Funcionalidades do Chat

Você pode fazer perguntas como :

- "Explique esta função:" (selecione o código antes)

- "Como faço para implementar autenticação JWT em Node.js?"
- "Encontre bugs neste trecho"
- "Sugira melhorias de performance"
- "Gere testes unitários para esta classe"

13.6.3 Comandos Especiais (Slash Commands)

No chat, você pode usar "/" seguido de comandos que direcionam a ação :

Comando Função

/doc Gera documentação/comentários para o código selecionado

/explain Explica o código selecionado linha a linha

/fix Tenta corrigir problemas no código

/tests Gera testes unitários

/optimize Sugere otimizações de performance

13.6.4 Variáveis de Contexto

Use # para incluir contexto específico :

- #selection: o código que você selecionou.
- #file: o arquivo atual.
- #editor: o editor atual.
- #codebase: todo o projeto (pode ser pesado).

13.6.5 Exercício prático 13.3

1. Escreva uma função simples com um bug proposital (ex: loop infinito ou condição errada).
2. Selecione a função.
3. No chat, use o comando /explain e veja a explicação.
4. Depois use /fix e veja a sugestão de correção.
5. Use /tests para gerar testes para a mesma função (após corrigida).

13.7 @workspace: Seu Projeto como Contexto

O participante @workspace é uma funcionalidade poderosa que permite ao Copilot entender todo o seu projeto .

13.7.1 O que faz?

Ao usar @workspace, o Copilot analisa:

- Todos os arquivos do workspace (respeitando .gitignore)
- Estrutura de diretórios

- Símbolos e definições
- Dependências

13.7.2 Exemplos de uso

- @workspace Onde está definida a constante de conexão com o banco?
- @workspace Como a autenticação é implementada atualmente?
- @workspace Quais arquivos dependem deste serviço?
- @workspace Mapeie todas as rotas da API e aponte as que não têm tratamento de erro

13.7.3 Aplicação prática

Em um projeto médio (ex: 30k linhas), @workspace pode responder em segundos, indicando arquivos e trechos relevantes .

13.7.4 Exercício prático 13.4

Se você tiver um projeto existente (pode ser o repositório do seu livro), abra-o no VS Code e experimente:

1. @workspace Quantos capítulos já foram escritos? (ele listará os arquivos cap*.md).
2. @workspace Qual a estrutura de pastas do projeto?
3. @workspace Onde eu uso a palavra 'rebase'? (busca contextual).

13.8 Agent Mode: Autonomia para Tarefas Complexas

O Agent Mode (modo agente) leva a automação a outro nível: o Copilot pode executar tarefas que envolvem múltiplos arquivos e etapas, de forma planejada .

13.8.1 Como ativar

No chat, selecione o modo "Agent" no menu suspenso (ao lado do campo de texto) .

13.8.2 Exemplo prático

Peça algo como:

"Crie um componente React de formulário de login com validação de campos e integração com uma API mock. Gere os arquivos necessários e organize na pasta src/components."

O agente:

- Analisa a estrutura do projeto
- Cria os arquivos (ex: LoginForm.jsx, LoginForm.css, api.js)
- Escreve o código completo

- Exibe um diff para você aceitar ou rejeitar cada alteração .

13.8.3 Cuidados com o Agent Mode

- Sem revise as alterações antes de aceitar.
- Em projetos grandes, defina bem o escopo para evitar mudanças indesejadas.
- Use com versionamento ativo (Git) para poder reverter se necessário.

13.8.4 Exercício prático 13.5

Em um projeto novo (pasta vazia), ative o Agent Mode e peça:

"Crie uma página HTML simples com um contador de cliques, usando CSS moderno e JavaScript separado. Coloque todos os arquivos na raiz."

Observe quantos arquivos são criados e como o agente organiza a solução.

13.9 Copilot e Testes: TDD Acelerado

O Copilot é excelente para gerar testes unitários, seja seguindo TDD ou cobrindo código existente .

13.9.1 Gerando testes para código existente

Selecione uma função e no chat use /tests. O Copilot gerará testes com base no contexto (framework que você costuma usar, estilo do projeto).

13.9.2 TDD com Copilot

1. Escreva o teste primeiro (descrevendo o comportamento esperado).
2. O Copilot pode sugerir a implementação que faz o teste passar.

Exemplo:

```
// Teste para função de soma
test('soma 1 + 2 e retorna 3', () => {
expect(soma(1, 2)).toBe(3);
});

Ao começar a escrever a função soma, o Copilot entenderá o contexto do teste
e sugerirá a implementação.

13.9.3 Exercício prático 13.6

1. Escreva um teste para uma função que ainda não existe (ex: multiplicar(a,
b)).
2. Veja se o Copilot sugere a implementação.
3. Aceite e execute o teste (se tiver Jest configurado, por exemplo).

---

13.10 Copilot para Documentação e Commits
```

13.10.1 Gerando documentação

Use /doc no chat sobre um trecho de código para gerar comentários estilo JSDoc, docstrings Python, etc. .

13.10.2 Gerando mensagens de commit

No VS Code, ao abrir o Source Control, você verá um ícone de "estrela" (■) ao lado do campo de mensagem de commit . Clique para que o Copilot gere uma mensagem descritiva baseada nas alterações feitas.

Isso segue as boas práticas de Conventional Commits, poupando tempo e mantendo consistência .

13.10.3 Exercício prático 13.7

1. Faça uma alteração simples em um arquivo.
2. No Source Control, clique no ícone ■ ao lado da mensagem de commit.
3. Veja a sugestão gerada. Aceite ou refine.
4. Commit.

13.11 Boas Práticas e Armadilhas Comuns

13.11.1 O que fazer (do's)

· Escreva comentários claros: seja específico sobre o que deseja .
· Revise o código gerado: o Copilot não é infalível. Sempre leia e entenda antes de confiar .
· Use testes: valide com testes automatizados o código sugerido .
· Forneça exemplos: se você tem um padrão de código, mostre alguns exemplos para o Copilot aprender .
· Mantenha arquivos relevantes abertos: o Copilot usa o contexto dos arquivos abertos para sugerir melhor .

13.11.2 O que evitar (don'ts)

· Não aceite cegamente: sugestões podem conter bugs, vulnerabilidades ou más práticas .
· Não compartilhe segredos: nunca coloque senhas, tokens ou chaves em prompts ou código que será enviado .
· Cuidado com dependências fantasmas: o Copilot pode sugerir APIs ou bibliotecas que não existem ou estão desatualizadas .
· Evite confiar em código de segurança crítico sem revisão profunda: especialmente criptografia, autenticação, etc.

13.11.3 Privacidade e dados

· O Copilot envia trechos de código para processamento na nuvem.
· Consulte a política da sua organização sobre uso de IA.
· Você pode desativar a coleta de telemetria nas configurações do VS Code .

```
13.11.4 Quando desligar o Copilot

Em projetos sensíveis ou quando estiver aprendendo um conceito do zero, pode
ser útil desabilitar temporariamente para não "pular etapas" e garantir o
entendimento.

---

13.12 Personalizando o Copilot

13.12.1 Instruções personalizadas (custom instructions)

Você pode criar um arquivo .github/copilot-instructions.md na raiz do projeto
para dar diretrizes globais ao Copilot .

Exemplo:
```

Instruções para o Copilot

- Use async/await em vez de Promises .then()
- Prefira funções puras sempre que possível
- Siga o estilo do Prettier configurado
- Comente em português (ou inglês, conforme o projeto)

13.12.2 Modos de chat personalizados

Crie arquivos em .github/chatmodes/ para definir comportamentos específicos, como "revisor de código", "especialista em segurança", etc. .

13.12.3 Desativando para linguagens específicas

Nas configurações, use github.copilot.enable e defina como false para linguagens onde não deseja sugestões (ex: Markdown, se preferir escrever sem interferência).

13.13 Fluxo de Trabalho Integrado com Git

13.13.1 Copilot durante code reviews

Use o Copilot para ajudar a revisar código de Pull Requests (com extensões apropriadas) ou para explicar código de outros colaboradores .

13.13.2 Gerando mensagens de commit (já visto)

13.13.3 Documentando mudanças

Após implementar uma feature, use /doc para gerar documentação que pode ser incluída no README ou wiki.

13.13.4 Escrevendo guias de contribuição

Peça ao Copilot para esboçar um CONTRIBUTING.md com base na estrutura do projeto.

13.14 Exercícios de Fixação

1. Quais são os pré-requisitos para usar GitHub Copilot no VS Code?
2. Como você aceita uma sugestão inline? E como vê sugestões alternativas?
3. Qual a diferença entre usar /explain no chat e simplesmente ler o código?
4. O que faz o comando @workspace? Dê um exemplo de uso prático.
5. Em que cenário o Agent Mode é mais útil?
6. Como você pode gerar uma mensagem de commit automaticamente com Copilot?
7. Cite duas boas práticas ao usar código gerado por IA.
8. Como você configuraria o Copilot para NÃO sugerir em arquivos Markdown?
9. O que é um "slash command" no Copilot Chat? Dê três exemplos.
10. (Desafio) Em um projeto pessoal, crie um arquivo de instruções personalizadas e veja se o Copilot respeita as diretrizes ao gerar novos trechos.

13.15 Resumo do Capítulo

- GitHub Copilot é um assistente de IA que sugere código em tempo real, baseado no contexto do projeto.
- A instalação é simples: extensão + autenticação com conta GitHub.
- Sugestões inline aparecem como "ghost text"; aceite com Tab, navegue com Alt+]/[.
- Copilot Chat permite conversas, explicações, correções e geração de testes.
- @workspace contextualiza todo o projeto; Agent Mode executa tarefas complexas multiarquivo.
- Comandos como /doc, /explain, /fix agilizam tarefas comuns.
- O Copilot ajuda em commits, documentação e até code review.
- É essencial revisar o código gerado, manter boas práticas de segurança e personalizar o comportamento quando necessário.

Agora você tem um "par de programação" disponível 24/7. No próximo capítulo, exploraremos automação com GitHub Actions, integrando CI/CD aos seus projetos.

13.16 Referências

- Microsoft Learn: "Develop code features using GitHub Copilot Tools"
- CODE Magazine: "Unleashing the Power of GitHub Copilot in Visual Studio Code"

• PacificTech: "Copilot■■■vscode"
• Skywork.ai: "GitHub Copilot in VS Code: Complete Integration Guide"
• Perficient Blogs: "Mastering GitHub Copilot in VS Code" e "Using GitHub Copilot in VS Code"
• ClickUp: "How to Use GitHub Copilot with VS Code (2026 Guide)"
• Tencent Cloud: "VS Code Copilot ■■■■■■■■■■■■"
• Xebia: "How to Use GitHub Copilot: The Complete Beginner's Guide"

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 14: Automação com GitHub Actions

Páginas estimadas: 30

Tempo de leitura: 80–100 minutos

Objetivos do capítulo:

- Compreender o conceito de integração contínua (CI) e entrega contínua (CD)
- Conhecer a estrutura de um workflow no GitHub Actions
- Criar workflows automatizados para testes, build e deploy
- Utilizar actions prontas do Marketplace
- Trabalhar com matrizes (matrix builds) para testar múltiplas versões
- Gerenciar segredos e variáveis de ambiente com segurança
- Produzir e consumir artefatos
- Criar uma action personalizada simples
- Aplicar na prática com exemplos reais: testes automatizados e geração de PDF do livro

---

14.1 Introdução à Automação e CI/CD

Até agora, você aprendeu a versionar código, colaborar via pull requests e gerenciar projetos. Mas o desenvolvimento moderno vai além: exige automação para garantir qualidade e agilidade. É aqui que entram os conceitos de **Integração Contínua (CI)** e **Entrega Contínua (CD)** .

- **CI (Continuous Integration):** prática de integrar código frequentemente ao repositório principal, executando testes automatizados a cada push, para detectar problemas rapidamente.
- **CD (Continuous Delivery/Deployment):** extensão da CI que automatiza a entrega do software em ambientes de homologação ou produção.

O **GitHub Actions** é a solução nativa do GitHub para CI/CD. Ele permite criar fluxos de trabalho automatizados diretamente no repositório, baseados em eventos (push, pull

request, schedule, etc.). Com ele, você pode:

- Executar testes em múltiplas plataformas e versões de linguagem.
- Compilar código e gerar artefatos.
- Fazer deploy automático em servidores ou nuvem.
- Publicar pacotes em registries (npm, Docker Hub, etc.).
- E muito mais – tudo gratuito para repositórios públicos e com uma generosa cota para privados.

Neste capítulo, você aprenderá a dominar o GitHub Actions, desde os conceitos básicos até a criação de workflows complexos, com exemplos práticos aplicados ao seu próprio livro.

14.2 Conceitos Fundamentais

Um workflow no GitHub Actions é definido por um arquivo YAML dentro da pasta `.github/workflows/` do repositório. Vamos entender seus componentes principais .

14.2.1 Workflow

É um processo automatizado composto por um ou mais **jobs**. Ele é acionado por um **evento** (ex: push, pull_request, schedule).

14.2.2 Evento

É o gatilho que inicia o workflow. Exemplos:

- `push`: quando commits são enviados para o repositório.
- `pull_request`: quando um PR é aberto, sincronizado ou fechado.
- `schedule`: agendamento no formato cron.
- `workflow_dispatch`: execução manual via interface do GitHub.
- `repository_dispatch`: acionado por API externa.

14.2.3 Job

Um job é um conjunto de **steps** executados no mesmo **runner**. Por padrão, jobs são executados em paralelo, mas podem ser configurados para depender de outros jobs.

14.2.4 Step

Cada step pode ser um comando shell ou uma **action** (reutilizável). Steps compartilham o mesmo sistema de arquivos (podem gerar artefatos que serão usados por steps posteriores no mesmo job).

14.2.5 Runner

É a máquina (virtual ou auto-hospedada) que executa o job. O GitHub fornece runners hospedados com Linux (Ubuntu), Windows e macOS. Você também pode configurar seus próprios runners.

14.2.6 Action

São unidades modulares de código reutilizável. Podem ser escritas em JavaScript, TypeScript ou container Docker, e publicadas no **GitHub Marketplace**. Usar actions prontas evita reinventar a roda.

14.3 Criando Seu Primeiro Workflow

Vamos criar um workflow simples que é executado a cada push e exibe uma mensagem.

14.3.1 Estrutura de pastas

No seu repositório, crie a pasta `.github/workflows/` (se não existir). Dentro dela, crie um arquivo `primeiro-workflow.yml`.

14.3.2 Conteúdo do arquivo

```
name: Meu Primeiro Workflow

on: [push] # gatilho: qualquer push

jobs:
job1:
runs-on: ubuntu-latest
steps:
- name: Checkout do código
uses: actions/checkout@v4 # action oficial para baixar o repositório

- name: Executar um script simples
run: echo "Olá, GitHub Actions!"

14.3.3 Explicação
```

```
· name: nome opcional do workflow.
· on: define os eventos que disparam o workflow. Neste caso, qualquer push.
· jobs: contém os jobs.
· job1: identificador do job.
· runs-on: tipo de runner (ubuntu-latest).
· steps: lista de passos.
· uses: actions/checkout@v4: action que baixa o repositório para o runner.
· run: executa um comando shell.

14.3.4 Fazendo o push

Após criar o arquivo, faça commit e push para o repositório. Acesse a aba
"Actions" do seu repositório no GitHub. Você verá o workflow sendo executado.
Clique nele para ver os logs.

14.3.5 Exercício prático 14.1

Crie o workflow acima no seu repositório e observe a execução. Altere o
comando echo para listar os arquivos do diretório com ls -la.

---

14.4 Explorando Eventos de Gatilho

O GitHub Actions oferece uma vasta gama de eventos . Vamos explorar os mais
úteis.

14.4.1 push e pull_request
```

on:

push:

branches: [main, develop] # executa apenas nas branches main e develop

pull_request:

branches: [main] # executa quando PR é aberto para main

types: [opened, synchronize, reopened] # tipos de atividade

14.4.2 schedule (cron)

```
on:
schedule:
- cron: '0 2 * * 1' # toda segunda-feira às 2h UTC

O formato cron tem cinco campos: minuto, hora, dia do mês, mês, dia da
semana.

14.4.3 workflow_dispatch (manual)
```

on:

workflow_dispatch:

inputs:

environment:

description: 'Ambiente para deploy'

required: true

default: 'homolog'

type: choice

options:

- homolog
- producao

Isso permite executar o workflow manualmente pela interface, passando parâmetros.

14.4.4 repository_dispatch (API)

```
on:
repository_dispatch:
types: [deploy-command]

Você pode acionar via API REST do GitHub.

14.4.5 Filtros avançados

É possível filtrar por caminhos de arquivo:
```

on:

push:

paths:

- 'src/**' # executa só se alterar arquivos em src/
- '!src/legado/**' # exceto a pasta legado

14.4.6 Exercício prático 14.2

Crie um workflow que seja executado:

- Em push para a branch main, mas apenas se houver alterações em capitulos/.
- Agendado para toda sexta-feira às 18h.
- Com possibilidade de execução manual recebendo um parâmetro "notificar" (booleano).

Teste cada gatilho (para o agendado, aguarde ou use workflow_dispatch para testar).

14.5 Matrix Builds e Estratégias

Uma das funcionalidades mais poderosas é a matrix strategy, que permite executar um job com diferentes combinações de parâmetros (versões de linguagem, sistemas operacionais, etc.) .

14.5.1 Exemplo de matrix simples

```
jobs:
test:
runs-on: ubuntu-latest
strategy:
matrix:
node-version: [14, 16, 18]
steps:
- uses: actions/checkout@v4
- name: Usar Node.js ${{ matrix.node-version }}
uses: actions/setup-node@v4
with:
node-version: ${{ matrix.node-version }}
- run: npm install
- run: npm test

Esse workflow executa três jobs em paralelo, um para cada versão do Node.js.

14.5.2 Matrix com múltiplas dimensões
```

strategy:

matrix:

os: [ubuntu-latest, windows-latest]

node: [16, 18]

include:

- os: ubuntu-latest

node: 20

exclude:

- os: windows-latest

node: 16

- include adiciona uma combinação extra.
- exclude remove combinações indesejadas.

14.5.3 Acessando valores da matrix

Use ${{ matrix.os }}, ${{ matrix.node }} nos steps.

14.5.4 Exercício prático 14.3

Crie um workflow que teste um projeto simples (ex: um script Python) em:

- Ubuntu com Python 3.9 e 3.10.

- Windows com Python 3.9.

Use actions/setup-python@v5. O teste pode ser apenas python --version e python -c "print('ok')".

14.6 Utilizando Actions do Marketplace

O GitHub Marketplace contém milhares de actions prontas para as mais diversas tarefas . Algumas extremamente úteis:

14.6.1 actions/checkout (já usado)

Baixa o código do repositório.

14.6.2 actions/setup-node, setup-python, setup-java, etc.

Configuram a versão da linguagem/ferramenta.

14.6.3 actions/cache

Cacheia dependências (ex: node_modules) para acelerar execuções futuras.

```
- name: Cache do npm
uses: actions/cache@v4
with:
path: ~/.npm
key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
restore-keys: |
${{ runner.os }}-node-

14.6.4 actions/upload-artifact e actions/download-artifact

Permitem salvar arquivos gerados (logs, builds) e compartilhá-los entre jobs
ou disponibilizá-los para download.

14.6.5 actions/github-script

Executa scripts JavaScript com acesso ao contexto do GitHub (API).

14.6.6 softprops/action-gh-release

Cria releases automaticamente a partir de tags.

14.6.7 Encontrando actions

No próprio GitHub, acesse Marketplace
(https://github.com/marketplace?type=actions) e pesquise.

14.6.8 Cuidados

· Sempre use a versão específica (ex: @v4) em vez de @main, para evitar
mudanças inesperadas.
· Verifique a popularidade e manutenção da action.
```

```
14.6.9 Exercício prático 14.4

Crie um workflow que:

· Use actions/checkout.
· Use actions/setup-node para Node 18.
· Instale dependências (npm install).
· Use actions/cache para cachear node_modules.
· Execute um script de teste (mesmo que simples, como echo "testes ok").

---

14.7 Criando uma Action Personalizada

Você pode criar suas próprias actions para reutilização dentro da organização
ou publicar no Marketplace . Há três tipos:

14.7.1 Action Docker

Usa um container Docker. Útil quando precisa de um ambiente específico.

14.7.2 Action JavaScript

Escrita em JavaScript (ou TypeScript), executada diretamente no runner (mais
rápido).

14.7.3 Action Composta (Composite)

Combina múltiplos steps em uma action reutilizável.

14.7.4 Exemplo: Action composta simples

Crie uma pasta actions/saudacao/ na raiz do repositório. Dentro, um arquivo
action.yml:
```

name: "Saudação"

description: "Exibe uma saudação personalizada"

inputs:

nome:

description: "Nome da pessoa"

required: true

default: "Mundo"

runs:

using: "composite"

steps:

- run: echo "Olá, ${{ inputs.nome }}!"

shell: bash

Para usar essa action em um workflow:

```
steps:
- name: Checkout
uses: actions/checkout@v4
- name: Saudação personalizada
uses: ./actions/saudacao
with:
nome: "Haroldo"

14.7.5 Publicando uma action

Para publicar no Marketplace, crie um repositório público com a action e
adicione tags de versão. O GitHub detecta automaticamente.

14.7.6 Exercício prático 14.5

Crie uma action composta que receba um nome e exiba uma mensagem de
boas-vindas. Em seguida, use-a em um workflow.

---

14.8 Exemplo Prático 1: Testes Automáticos com Node.js

Vamos criar um workflow real para um projeto Node.js com testes.

14.8.1 Estrutura do projeto (simulada)

Suponha um projeto com package.json e script de teste npm test.

14.8.2 Workflow test.yml
```

name: CI - Testes

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

test:

runs-on: ubuntu-latest

strategy:

matrix:

node-version: [16.x, 18.x, 20.x]

steps:

- name: Checkout do código

uses: actions/checkout@v4

- name: Cache do npm

uses: actions/cache@v4

with:

path: ~/.npm

key: ${{ runner.os }}-node-${{ matrix.node-version }}-${{ hashFiles('**/package-lock.json') }}

restore-keys: |

${{ runner.os }}-node-${{ matrix.node-version }}-

- name: Configurar Node.js ${{ matrix.node-version }}

uses: actions/setup-node@v4

with:

node-version: ${{ matrix.node-version }}

- name: Instalar dependências

run: npm ci

- name: Executar testes

run: npm test

- name: Upload de logs (se falhar)

if: failure()

uses: actions/upload-artifact@v4

with:

name: logs-node-${{ matrix.node-version }}

path: test-results.log

14.8.3 Explicação

- npm ci é mais rápido e confiável que npm install em CI.
- Cache armazena ~/.npm para evitar download de pacotes repetido.
- O step de upload só roda se os testes falharem (if: failure()), enviando logs para análise.

14.8.4 Exercício prático 14.6

Crie um projeto Node mínimo (ou use um existente) e implemente esse workflow. Simule um teste que falha e observe o artefato de log.

14.9 Exemplo Prático 2: Gerando PDF do Livro Automaticamente

Este exemplo é especialmente útil para você, autor. Vamos automatizar a geração do PDF do livro sempre que uma nova tag de versão for criada.

14.9.1 Pré-requisitos

- O livro está em Markdown, com capítulos na pasta capitulos/.
- Utilizamos Pandoc para gerar PDF (como discutido no início do livro).
- Precisamos de um runner com Pandoc e LaTeX instalados.

14.9.2 Workflow gerar-pdf.yml

```
name: Gerar PDF do Livro

on:
push:
tags:
- 'v*' # dispara quando uma tag começando com 'v' é criada (ex: v1.0, v2.1)
workflow_dispatch: # também permite execução manual

jobs:
build-pdf:
runs-on: ubuntu-latest
container:
image: pandoc/latex:latest # container oficial com Pandoc e LaTeX

steps:
- name: Checkout do repositório
uses: actions/checkout@v4

- name: Gerar PDF com Pandoc
run: |
mkdir -p output
pandoc \
--from markdown \
--to pdf \
--output output/livro.pdf \
--pdf-engine=xelatex \
capitulos/cap*.md

- name: Criar Release
id: create_release
uses: softprops/action-gh-release@v2
with:
files: output/livro.pdf
name: Release ${{ github.ref_name }}
body: |
PDF gerado automaticamente para a versão ${{ github.ref_name }}.
```

```
env:
GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

14.9.3 Explicação

· Usamos o container pandoc/latex que já tem tudo instalado.
· O Pandoc converte todos os capítulos (cap*.md) em um único PDF.
· A action softprops/action-gh-release anexa o PDF à release criada.
· ${{ github.ref_name }} é o nome da tag (ex: v1.0).
· secrets.GITHUB_TOKEN é um token automático fornecido pelo GitHub para
autenticação.

14.9.4 Como testar

1. Crie uma tag local: git tag -a v0.1 -m "Versão de teste"
2. Envie a tag: git push origin v0.1
3. Acesse a aba Actions e veja o workflow executar.
4. Depois, verifique a aba Releases: deve haver uma release com o PDF
anexado.

14.9.5 Exercício prático 14.7

Adapte o workflow acima para o seu repositório do livro. Se ainda não tem um
arquivo de template LaTeX personalizado, pode usar o comando simples. Teste
com uma tag.

---

14.10 Segredos e Variáveis de Ambiente

Em workflows, é comum precisar de tokens, senhas ou chaves de API. O GitHub
fornece segredos criptografados .

14.10.1 Criando segredos

Vá em Settings → Secrets and variables → Actions no seu repositório. Clique
em New repository secret. Dê um nome (ex: DOCKER_PASSWORD) e o valor.

14.10.2 Usando segredos no workflow
```

- name: Login no Docker Hub

run: echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" --password-stdin

Importante: segredos são mascarados nos logs (não aparecem em texto claro).

14.10.3 Variáveis de ambiente comuns

Você também pode definir variáveis de ambiente em nível de workflow, job ou step:

```
env:
NODE_ENV: production
```

```
jobs:
exemplo:
runs-on: ubuntu-latest
env:
API_URL: https://api.exemplo.com
steps:
- run: echo $API_URL
env:
TOKEN: ${{ secrets.TOKEN }}

14.10.4 Contexto github

O GitHub disponibiliza várias informações no contexto github, como:

· github.repository: nome do repositório
· github.ref: referência (branch/tag)
· github.event_name: nome do evento
· github.sha: hash do commit

14.10.5 Exercício prático 14.8

Crie um segredo chamado MEU_SEGREDO com um valor qualquer. Em um workflow,
use echo ${{ secrets.MEU_SEGREDO }} e veja que ele aparece como *** nos logs.
Tente também usar em uma variável de ambiente.

---

14.11 Artefatos e Caching

14.11.1 Artefatos

Artefatos são arquivos gerados durante o workflow que podem ser armazenados e
baixados posteriormente. Por exemplo, binários compilados, logs, relatórios
de teste.

· Upload: actions/upload-artifact@v4
· Download: actions/download-artifact@v4

Exemplo de upload:
```

- name: Upload artefato

uses: actions/upload-artifact@v4

with:

name: meu-artefato

path: dist/

Artefatos podem ser baixados pela interface do GitHub ou por outro job.

14.11.2 Caching

O cache armazena dependências ou pastas para acelerar execuções futuras. Exemplo com npm:

```
- name: Cache node_modules
uses: actions/cache@v4
with:
path: node_modules
key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}

A chave (key) deve ser única para cada conjunto de dependências; hashFiles
gera um hash do arquivo lock.

14.11.3 Diferença entre artefato e cache

· Cache: para acelerar execuções (dependências, compilações intermediárias).
· Artefato: para preservar resultados finais (relatórios, builds).

14.11.4 Exercício prático 14.9

Crie um workflow que:

· Gere um arquivo de log com date > log.txt.
· Faça upload desse arquivo como artefato.
· Em outro job, baixe o artefato e exiba seu conteúdo.

---

14.12 Boas Práticas e Segurança

14.12.1 Versionamento de actions

Sempre fixe a versão das actions (ex: @v4) em vez de @main ou @master, para
evitar surpresas com mudanças.

14.12.2 Mínimo privilégio

Use permissões restritas nos workflows. Por padrão, o GITHUB_TOKEN tem
permissões limitadas, mas você pode refiná-las:
```

permissions:

contents: read

pull-requests: write

14.12.3 Evitar segredos em logs

Nunca faça echo de segredos diretamente. Use ${{ secrets.NOME }} apenas onde necessário.

14.12.4 Reutilização de workflows

Para evitar repetição, você pode usar workflows reutilizáveis (chamados por outros workflows) ou templates.

14.12.5 Monitoramento

A aba Actions mostra o histórico de execuções. Configure alertas para falhas (por e-mail ou Slack) usando actions de notificação.

14.12.6 Exercício prático 14.10

Revise um workflow seu e adicione permissões explícitas mínimas. Pesquise sobre workflow_call e tente criar um workflow reutilizável simples.

14.13 Integração com VS Code

O VS Code oferece extensões que facilitam o desenvolvimento de workflows.

14.13.1 Extensão "GitHub Actions"

- Nome: GitHub Actions (por GitHub)
- Funcionalidades:
- Visualizar workflows e suas execuções diretamente no editor.
- Editar arquivos YAML com syntax highlighting e validação.
- Acionar workflows manualmente.

14.13.2 Extensão "YAML" (Red Hat)

Fornece validação de esquema para arquivos YAML, incluindo autocompletar para GitHub Actions (baseado no esquema oficial).

14.13.3 Criando workflows no VS Code

Com as extensões instaladas, crie um arquivo .github/workflows/novo.yml. O VS Code oferece snippets e validação em tempo real.

14.13.4 Depuração local

Embora não seja possível executar workflows localmente, você pode usar a ferramenta act (https://github.com/nektos/act) para testar workflows em container localmente, simulando o GitHub Actions.

14.13.5 Exercício prático 14.11

Instale as extensões mencionadas e crie um novo workflow diretamente no VS Code. Use os snippets para preencher a estrutura.

14.14 Exercícios de Fixação

1. O que é um workflow no GitHub Actions?
2. Quais são os principais eventos que podem disparar um workflow?

3. Como você limita um workflow para executar apenas em pushes para a branch main e em pull requests para main?

4. Explique o que é uma matrix strategy e dê um exemplo.

5. Qual a diferença entre actions/cache e actions/upload-artifact?

6. Como você armazena um token de acesso com segurança em um workflow?

7. Descreva um workflow que gere um PDF a partir de arquivos Markdown e o anexe a uma release.

8. Qual action você usaria para configurar o Node.js em uma versão específica?

9. Como você define uma variável de ambiente em nível de job?

10. (Desafio) Crie um workflow que, ao ser executado manualmente (workflow_dispatch), aceite um parâmetro "linguagem" e execute testes em Node.js ou Python conforme o valor escolhido. Use uma matrix condicional.

14.15 Resumo do Capítulo

- GitHub Actions permite automatizar tarefas de CI/CD diretamente no repositório.
- Workflows são definidos em YAML na pasta .github/workflows/.
- Eventos como push, pull_request, schedule e workflow_dispatch disparam os workflows.
- Jobs agrupam steps, que podem ser comandos ou actions reutilizáveis.
- Matrix strategy executa jobs em múltiplas combinações de parâmetros.
- O Marketplace oferece milhares de actions prontas.
- Podemos criar nossas próprias actions (compostas, Docker, JavaScript).
- Segredos e variáveis de ambiente protegem informações sensíveis.
- Artefatos e cache otimizam e preservam resultados.
- Boas práticas incluem versionar actions, definir permissões mínimas e evitar expor segredos.
- O VS Code possui extensões que auxiliam na criação e visualização de workflows.

Agora você pode automatizar praticamente qualquer tarefa do seu projeto diretamente no GitHub. No próximo capítulo, colocaremos tudo em prática com um projeto real: gerenciar um TCC usando Git.

14.16 Referências

- GitHub Docs: "Understanding GitHub Actions"
- GitHub Docs: "Workflow syntax for GitHub Actions"
- GitHub Marketplace: https://github.com/marketplace?type=actions
- "act" – Run GitHub Actions locally: https://github.com/nektos/act
- Blog do GitHub: "GitHub Actions: agora com execução manual e mais"

- Documentação das actions oficiais (actions/checkout, actions/setup-node, etc.)

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 15: Projeto Prático 1 – Gerenciando um TCC ou Artigo Científico com Git

Páginas estimadas: 22

Tempo de leitura: 60–75 minutos

Objetivos do capítulo:

- Aplicar os conceitos de Git e GitHub no contexto acadêmico
- Configurar um repositório para um trabalho de conclusão de curso (TCC)
- Utilizar branches para organizar versões (rascunho, revisão, final)
- Colaborar com orientadores e colegas usando Issues e Pull Requests
- Gerenciar referências bibliográficas com Git
- Automatizar a geração do PDF do trabalho
- Versionar documentos do Office (Word, Excel) com Git LFS
- Publicar o trabalho final no GitHub Pages

15.1 Introdução

O Trabalho de Conclusão de Curso (TCC) é um dos maiores desafios acadêmicos. Envolve pesquisa, escrita, revisões, formatação e, muitas vezes, colaboração com orientador e colegas. Tradicionalmente, isso é feito com dezenas de arquivos `versao_final.docx`, `versao_revisada.docx`, `versao_final_mesmo.docx`, enviados por e-mail – um pesadelo de versionamento.

O Git, combinado com o GitHub, pode transformar esse processo em algo organizado, rastreável e seguro. Neste capítulo, você aprenderá a aplicar todo o conhecimento adquirido para gerenciar um TCC (ou qualquer artigo científico) de forma profissional.

15.2 Configurando o Repositório para um Trabalho Acadêmico

15.2.1 Estrutura de pastas sugerida

Antes de mais nada, defina uma estrutura organizada. Exemplo:

```
tcc/
■■■ README.md
■■■ .gitignore
■■■ texto/
■ ■■■ capitulos/
■ ■ ■■■ 01-introducao.md
■ ■ ■■■ 02-revisao.md
■ ■ ■■■ 03-metodologia.md
■ ■ ■■■ 04-resultados.md
■ ■ ■■■ 05-conclusao.md
■ ■■■ pre-textuais/
■ ■ ■■■ resumo.md
■ ■ ■■■ abstract.md
■ ■ ■■■ sumario.md
■ ■■■ pos-textuais/
■ ■ ■■■ apendices.md
■ ■ ■■■ anexos.md
■ ■■■ referencias.bib
■■■ imagens/
■ ■■■ figura1.png
■ ■■■ figura2.svg
■■■ dados/
■ ■■■ planilha-resultados.xlsx
■■■ scripts/
■ ■■■ analise.R
■■■ output/
```

■■■ tcc.pdf (gerado automaticamente)

15.2.2 Criando o repositório

```
mkdir tcc
cd tcc
git init
git branch -m main

Crie os arquivos iniciais (README.md, .gitignore) e faça o primeiro commit:
```

echo "# Meu TCC" > README.md

git add README.md

git commit -m "docs: adiciona README inicial"

15.2.3 Configurando o .gitignore

Para um TCC, você pode querer ignorar arquivos temporários e de output:

Arquivos temporários

*.bak

*.swp

*.tmp

PDFs gerados (se forem sempre gerados por script)

/output/

Dados sensíveis (se houver)

/dados/privado/

Arquivos do sistema

.DS_Store

Thumbs.db

15.2.4 Criando a estrutura de pastas

```
mkdir -p texto/capitulos texto/pre-textuais texto/pos-textuais imagens dados
scripts output

Adicione arquivos placeholder para cada capítulo (opcional). Depois, commit:
```

git add .

git commit -m "chore: adiciona estrutura inicial do TCC"

15.3 Usando Branches para Revisões e Versões

No desenvolvimento de um TCC, você terá diferentes estágios:

- Rascunho: primeira versão, ainda bruta.
- Revisão: após feedback do orientador.
- Final: versão entregue.
- Banca: versão com correções pós-banca.

Você pode usar branches para organizar isso.

15.3.1 Branch principal: main (versão final)

A branch main conterá a versão mais atualizada e aprovada do trabalho. Inicialmente, pode estar vazia ou com a estrutura.

15.3.2 Branch de desenvolvimento: dev (rascunho)

Crie uma branch dev para trabalhar nos rascunhos:

```
git switch -c dev

Agora você pode escrever à vontade, fazer commits frequentes, sem medo de
"quebrar" a versão estável.

15.3.3 Branches para revisões específicas

Quando receber feedback do orientador, crie uma branch para aquela revisão:
```

git switch -c revisao-orientador-2025-03-20

Isso permite que você experimente as correções sem afetar o fluxo principal da dev.

15.3.4 Mesclando após aprovação

Quando uma revisão for aprovada, mescle-a na dev e depois, quando estiver pronto para uma versão "quase final", mescle na main e crie uma tag.

```
git switch dev
git merge revisao-orientador-2025-03-20
git tag -a v0.9 -m "Versão pré-banca"
git switch main
git merge dev
git tag -a v1.0 -m "Versão final para banca"

15.3.5 Exercício prático 15.1

Crie as branches dev e revisao1 no seu repositório de TCC. Faça algumas
alterações em cada uma e pratique os merges. Use git log --graph --oneline
para visualizar o histórico.

---

15.4 Colaboração com Orientadores e Colegas

O GitHub facilita a colaboração com orientadores e colegas, mesmo que eles
não conheçam Git profundamente (eles podem usar a interface web).

15.4.1 Convidando colaboradores

No repositório do GitHub, vá em Settings → Collaborators and teams → Add
people. Adicione o e-mail do seu orientador. Ele receberá um convite e poderá
acessar o repositório (se for privado, precisa de conta GitHub).

15.4.2 Orientador revisando via Issues

O orientador pode abrir Issues apontando problemas ou sugestões. Exemplo:

Título: Revisão do Capítulo 2
Descrição: Na seção 2.3, a explicação sobre metodologia ficou confusa. Sugiro
reescrever focando nos passos práticos. Veja o artigo X como referência.

Você pode responder, marcar como resolvida quando corrigir.

15.4.3 Pull Requests para revisões formais

Se o orientador quiser revisar alterações, ele pode fazer um fork (se não for
colaborador) ou vocês podem usar Pull Requests. Você cria uma branch com as
alterações, abre um PR e o orientador comenta diretamente nas linhas do
código (texto). Isso é extremamente útil para revisões detalhadas.

15.4.4 Usando GitHub Discussions para debates

Para discussões mais amplas (escolha do tema, metodologia), ative as
Discussions no repositório. É um fórum integrado.

15.4.5 Exercício prático 15.2

Convide um colega para colaborar em um repositório de teste. Peça que ele
abra uma issue sugerindo uma melhoria. Depois, crie uma branch, implemente a
```

```
melhoria e abra um Pull Request. Simule o processo de revisão com
comentários.

---

15.5 Gerenciando Referências Bibliográficas

Referências são parte crucial de um TCC. O formato mais adequado para
versionamento é o BibTeX (arquivo .bib).

15.5.1 Criando um arquivo referencias.bib

Exemplo de entrada:
```

@article{knuth1984,

author = {Donald E. Knuth},

title = {Literate Programming},

journal = {The Computer Journal},

year = {1984},

volume = {27},

number = {2},

pages = {97--111}

}

15.5.2 Versionando o .bib

O arquivo .bib é texto puro, perfeito para Git. Cada vez que você adiciona uma referência, faz um commit. O histórico mostra exatamente quando e por que cada referência foi incluída.

15.5.3 Usando referências no texto

Se você escreve em Markdown/LaTeX, pode citar as referências com @knuth1984 e depois usar o Pandoc + filtro pandoc-citeproc para gerar a bibliografia formatada (ABNT, APA, etc.). Veremos isso na automação.

15.5.4 Ferramentas auxiliares

- Zotero ou Mendeley podem exportar em BibTeX. Mantenha o arquivo .bib sincronizado com o repositório.
- JabRef é um gerenciador de referências open source que trabalha diretamente com .bib.

15.5.5 Exercício prático 15.3

Crie um arquivo referencias.bib com três referências de artigos ou livros da sua área. Adicione ao repositório e faça commit.

15.6 Automatizando a Geração do PDF

Um dos maiores ganhos de produtividade é gerar o PDF automaticamente sempre que você faz uma alteração, usando GitHub Actions.

15.6.1 Pré-requisitos

- Os capítulos em Markdown (ou LaTeX).
- Um arquivo mestre que os combine (ex: tcc.md que inclui os capítulos via !include ou simplesmente concatenando).
- Pandoc e LaTeX disponíveis (usaremos container).

15.6.2 Workflow de geração

Já vimos um exemplo similar no capítulo 14. Vamos adaptar para o TCC:

```
name: Gerar PDF do TCC

on:
push:
branches: [ main ]
paths:
- 'texto/**'
- 'referencias.bib'
workflow_dispatch:

jobs:
build-pdf:
runs-on: ubuntu-latest
container:
image: pandoc/latex:latest

steps:
- uses: actions/checkout@v4

- name: Concatenar capítulos
run: |
cat texto/pre-textuais/*.md > tcc-completo.md
cat texto/capitulos/*.md >> tcc-completo.md
cat texto/pos-textuais/*.md >> tcc-completo.md

- name: Gerar PDF
run: |
pandoc tcc-completo.md \
--from markdown \
--to pdf \
--output tcc.pdf \
--pdf-engine=xelatex \
--bibliography=referencias.bib \
--citeproc \
--csl=abnt.csl # se tiver um arquivo de estilo ABNT
```

```
- name: Upload do PDF
uses: actions/upload-artifact@v4
with:
name: tcc-pdf
path: tcc.pdf

15.6.3 Adicionando estilo ABNT

Você precisa de um arquivo CSL (Citation Style Language) para o formato ABNT.
Baixe de https://github.com/citation-style-language/styles (procure por
abnt.csl) e coloque na raiz do repositório. No workflow, referencie com
--csl=abnt.csl.

15.6.4 Gerando release com PDF

Assim como no capítulo 14, você pode adicionar um step que cria uma release e
anexa o PDF quando uma tag é criada.

15.6.5 Exercício prático 15.4

Implemente o workflow acima no seu repositório de TCC (ou teste em um
repositório separado). Faça um push na branch main e veja o PDF ser gerado e
disponibilizado como artefato.

---

15.7 Versionando Documentos do Office (com Git LFS)

Arquivos binários como .docx, .xlsx, .pptx não são ideais para o Git, pois
não permitem diff e incham o repositório. Mas às vezes são necessários (ex:
planilha de dados, questionários).

A solução é o Git Large File Storage (LFS) , que substitui os arquivos por
ponteiros e armazena o conteúdo em um servidor separado.

15.7.1 Instalando Git LFS

· Windows: Baixe o instalador em https://git-lfs.com.
· macOS: brew install git-lfs
· Linux: Use o gerenciador de pacotes (ex: sudo apt install git-lfs).

Depois, em cada repositório, inicie:
```

git lfs install

15.7.2 Rastreando arquivos com LFS

```
git lfs track "*.docx"
git lfs track "*.xlsx"
git lfs track "*.pdf"

Isso cria/atualiza o arquivo .gitattributes. Adicione e commit:
```

git add .gitattributes

git commit -m "chore: configura Git LFS para documentos Office"

Agora, ao adicionar arquivos .docx, eles serão gerenciados pelo LFS.

15.7.3 Limitações

- O GitHub oferece 1 GB de armazenamento LFS gratuito e 1 GB de largura de banda mensal. Para TCCs, é suficiente.
- Colaboradores também precisam ter Git LFS instalado para clonar corretamente.

15.7.4 Alternativa: converter para Markdown

Sempre que possível, prefira formatos texto. Por exemplo, planilhas podem ser exportadas para CSV. Isso evita o LFS.

15.7.5 Exercício prático 15.5

Crie um arquivo .docx de teste, configure o Git LFS no repositório, adicione o arquivo e faça push. Depois, clone o repositório em outra pasta e verifique se o arquivo foi baixado corretamente.

15.8 Publicando o Trabalho com GitHub Pages

Você pode publicar uma versão web do seu TCC (HTML) usando GitHub Pages . Isso é ótimo para compartilhar com a banca ou como portfólio.

15.8.1 Convertendo para HTML

Com Pandoc, você pode gerar HTML em vez de PDF:

```
pandoc tcc-completo.md --to html --output docs/index.html --bibliografia
referencias.bib --citeproc --csl=abnt.csl

15.8.2 Configurando o Pages

1. Crie uma pasta docs/ na raiz.
2. Coloque o index.html e os recursos (CSS, imagens) lá.
3. No GitHub, vá em Settings → Pages.
4. Em "Branch", selecione main e a pasta /docs.
5. Salve. Seu site estará disponível em
https://haroldocapucci.github.io/nome-do-repo/.

15.8.3 Automatizando com Actions

Você pode criar um workflow que gera o HTML e publica no Pages
automaticamente. O GitHub tem uma action própria: peaceiris/actions-gh-pages.
```

- name: Deploy para GitHub Pages

uses: peaceiris/actions-gh-pages@v4

with:

github_token: ${{ secrets.GITHUB_TOKEN }}

publish_dir: ./docs

publish_branch: gh-pages

15.8.4 Exercício prático 15.6

Crie uma versão HTML simples do seu TCC (pode ser apenas o resumo) e publique no GitHub Pages. Compartilhe o link.

15.9 Dicas Extras para TCC com Git

15.9.1 Commits semânticos

Use Conventional Commits para manter o histórico organizado:

- docs: adiciona introdução
- fix: corrige erro na seção 3.2
- refactor: reorganiza capítulo 4
- feat: adiciona gráfico de resultados

15.9.2 Backup automático

Com o GitHub, seu TCC já está backupado na nuvem. Mas você pode fazer push automático periódico com um cron job (não recomendado para evitar conflitos).

15.9.3 Trabalhando offline

O Git é distribuído: você pode trabalhar no metrô, fazer commits, e depois sincronizar. Perfeito para quem não tem internet sempre.

15.9.4 Controle de versão de imagens

Imagens também são binárias. Considere usar LFS para pastas de imagens, mas prefira formatos leves (SVG, PNG otimizado).

15.9.5 Template de TCC

Crie um template com a estrutura e compartilhe com colegas. Você pode fazer um fork do template e adaptar.

15.10 Exercícios de Fixação

1. Como você organizaria as branches para um TCC com versões: rascunho, revisão orientador, versão banca, versão final?
2. Qual a vantagem de usar Issues para comunicação com o orientador?
3. Por que o formato BibTeX é mais adequado para versionamento do que um arquivo do Word com referências?

4. Descreva um workflow do GitHub Actions que gere o PDF do TCC automaticamente.
5. O que é Git LFS e quando usá-lo em um TCC?
6. Como você publicaria seu TCC em formato web usando GitHub Pages?
7. Crie um exemplo de mensagem de commit seguindo o padrão Conventional Commits para a adição de um novo capítulo.
8. (Desafio) Configure um repositório completo para um TCC fictício, incluindo estrutura de pastas, branches, um workflow de PDF e um script simples de análise de dados em R ou Python.

15.11 Resumo do Capítulo

- Um repositório Git bem estruturado é ideal para gerenciar TCCs.
- Branches permitem organizar rascunhos, revisões e versões finais.
- Issues e Pull Requests facilitam a colaboração com orientadores.
- Referências em BibTeX são versionáveis e integráveis à geração de PDF.
- GitHub Actions automatiza a compilação do PDF a cada alteração.
- Git LFS resolve o problema de arquivos binários grandes.
- GitHub Pages oferece uma versão web do trabalho.

Agora você pode aplicar esses conceitos ao seu próprio TCC e impressionar a banca com a organização e profissionalismo. No próximo capítulo, veremos como contribuir com projetos open source, outro grande uso do Git e GitHub.

15.12 Referências

- Pro Git Book – Capítulos sobre Git LFS e GitHub
- GitHub Docs: "Git Large File Storage"
- GitHub Docs: "GitHub Pages"
- Pandoc User Guide
- Citation Style Language (CSL) – abnt.csl
- Conventional Commits

Git e GitHub: Do Iniciante ao Profissional com VS Code

Autor: Haroldo Capucci

Capítulo 16: Projeto Prático 2 – Colaboração em um Projeto Open Source

Páginas estimadas: 24

Tempo de leitura: 70–90 minutos

Objetivos do capítulo:

- Compreender a cultura e os benefícios do open source
- Encontrar um projeto adequado ao seu nível de habilidade
- Analisar issues e escolher uma tarefa para contribuir
- Realizar todo o fluxo: fork, clone, branch, commit, push e pull request
- Interagir com mantenedores e comunidade de forma eficaz
- Lidar com revisões, feedback e ajustes no PR
- Adotar boas práticas para contribuições duradouras

16.1 Introdução

Contribuir para projetos open source é uma das experiências mais gratificantes para um desenvolvedor. Além de melhorar suas habilidades técnicas, você aprende a trabalhar em equipe, a ler código de outras pessoas e a se comunicar de forma clara. Para muitos, é também um trampolim para oportunidades profissionais.

Neste capítulo, você aplicará tudo o que aprendeu nos capítulos anteriores em um cenário real: contribuir com um projeto open source no GitHub. Vamos desde a escolha do projeto até o momento em que seu Pull Request é aceito e mesclado.

16.2 O Ecossistema Open Source

16.2.1 O que é open source?

Open source significa que o código fonte de um projeto é disponibilizado publicamente, permitindo que qualquer pessoa use, estude, modifique e distribua, geralmente sob uma licença que garante essas liberdades (MIT, GPL, Apache, etc.).

16.2.2 Por que contribuir?

- **Aprendizado:** você lê código de qualidade e recebe feedback de desenvolvedores experientes.
- **Portfólio:** contribuições contam muito em processos seletivos.
- **Networking:** você conhece pessoas da comunidade global.
- **Gratificação:** ajudar projetos que você usa é uma forma de retribuir.
- **Visibilidade:** seu nome fica associado a projetos conhecidos.

16.2.3 Tipos de contribuição

Nem toda contribuição é código. Você pode ajudar com:

- Documentação (corrigir typos, traduzir, melhorar exemplos).
- Relatar bugs (issues bem detalhadas).
- Revisar Pull Requests de outros.
- Responder dúvidas em discussões ou fóruns.
- Design, acessibilidade, testes.

Para começar, contribuir com documentação é um ótimo primeiro passo.

16.3 Encontrando um Projeto para Contribuir

16.3.1 Critérios para escolher um projeto

- **Tecnologias que você conhece:** escolha algo em uma linguagem/framework que você domina.
- **Ativo:** veja se há commits recentes, issues sendo respondidas, PRs mesclados.
- **Acolhedor:** projetos com labels como `good first issue`, `help wanted` ou `beginner-friendly` indicam que estão abertos a novos contribuidores.
- **Tamanho:** projetos muito grandes (ex: Kubernetes) podem ser complexos; comece com projetos de médio porte.

16.3.2 Onde procurar

- **Explorar o GitHub:** Use a busca com filtros. Exemplo: `language:python state:open label:"good first issue"`.
- **GitHub Explore:** github.com/explore
- **First Timers Only:** [firsttimersonly.com](https://www.firsttimersonly.com) – lista de issues para iniciantes.
- **Up For Grabs:** up-for-grabs.net
- **CodeTriage:** [codetriage.com](https://www.codetriage.com) – receba issues por e-mail de projetos que você acompanha.

16.3.3 Analisando um projeto candidato

Suponha que você encontre um projeto interessante, como o **freeCodeCamp** ou uma biblioteca que você usa. Antes de sair contribuindo, explore:

- Leia o `README.md` e o `CONTRIBUTING.md` (se existir).
- Veja as issues abertas e fechadas para entender o fluxo.
- Observe como os mantenedores se comunicam. São educados? Respondem rápido?
- Verifique se o projeto tem testes e se eles passam.

16.3.4 Exercício prático 16.1

Pesquise no GitHub por issues com a label `good-first-issue` em uma linguagem que você conhece. Escolha três projetos potenciais e analise cada um segundo os critérios acima. Anote suas impressões.

16.4 Escolhendo uma Issue e Se Apresentando

16.4.1 Tipos de issue adequadas para iniciantes

- **docs:** correção de typos, melhoria em exemplos.
- **bug:** geralmente mais complexo, mas pode ter bugs simples.
- **enhancement:** pequenas melhorias.
- **good first issue:** especialmente sinalizada para novatos.

16.4.2 Comunique-se antes de começar

Antes de sair codificando, **comente na issue** se apresentando e perguntando se pode trabalhar nela. Exemplo:

> Olá! Gostaria de contribuir com este projeto. Essa issue me parece adequada para começar. Posso ficar responsável por ela? Caso positivo, pretendo seguir as diretrizes de contribuição. Obrigado!

Isso evita que duas pessoas trabalhem na mesma coisa ao mesmo tempo e mostra que você é organizado.

16.4.3 Aguarde o sinal verde

O mantenedor pode responder algo como "Claro, fique à vontade!". Se não responder em alguns dias, você pode considerar que está liberado (mas sempre prefira aguardar). Se a issue já estiver atribuída a alguém, escolha outra.

16.4.4 Exercício prático 16.2

Encontre uma issue não atribuída em um dos projetos que você selecionou. Deixe um comentário se oferecendo para trabalhar nela. Aguarde a resposta.

16.5 Preparando o Ambiente Local

16.5.1 Fazendo o fork do repositório

1. No GitHub, acesse o repositório original.
2. Clique no botão **Fork** (canto superior direito).
3. Escolha sua conta. O GitHub criará uma cópia em `https://github.com/seu-usuario/nome-do-projeto`.

16.5.2 Clonando o fork localmente

```
git clone https://github.com/seu-usuario/nome-do-projeto.git
cd nome-do-projeto

16.5.3 Adicionando o repositório original como upstream

Isso permitirá sincronizar seu fork com as alterações do projeto original.
```

git remote add upstream https://github.com/original/nome-do-projeto.git

git remote -v # confira

16.5.4 Criando uma branch para sua contribuição

Nunca trabalhe na main do seu fork. Crie uma branch descritiva:

```
git switch -c fix/typo-readme # ou feat/nova-funcionalidade

16.5.5 Configurando o ambiente de desenvolvimento

Siga as instruções do README ou CONTRIBUTING.md para instalar dependências,
rodar testes, etc. Pode ser necessário criar um ambiente virtual, instalar
pacotes, etc.

16.5.6 Exercício prático 16.3

Realize o fork e o clone do projeto escolhido. Adicione o upstream e crie uma
branch para a issue que você escolheu. Tente rodar o projeto localmente
seguindo as instruções.

---

16.6 Fazendo as Alterações e Commit

16.6.1 Codificando com atenção

· Siga o estilo de código do projeto (indentação, nomes, etc.). Muitos
projetos têm linters configurados.
· Se for uma correção de bug, teste antes e depois para garantir que
funciona.
· Se for documentação, verifique a ortografia e clareza.

16.6.2 Commits atômicos e mensagens claras

Faça commits pequenos, cada um com uma mudança lógica. Use mensagens no
padrão Conventional Commits, se o projeto adotar (geralmente está em
CONTRIBUTING.md). Exemplo:

fix: corrige erro de digitação no README

16.6.3 Testando localmente

Execute os testes do projeto para garantir que você não quebrou nada. Se não
houver testes automatizados, pelo menos teste manualmente a funcionalidade
afetada.

16.6.4 Exercício prático 16.4

Implemente a alteração necessária para resolver a issue. Faça commits com
mensagens descritivas. Execute os testes (se houver) e verifique se tudo
passa.

---

16.7 Enviando as Alterações e Abrindo o Pull Request

16.7.1 Push da branch para o seu fork
```

git push origin fix/typo-readme

16.7.2 Abrindo o Pull Request pelo GitHub

1. Acesse seu fork no GitHub. O GitHub geralmente exibe um banner sugerindo "fix/typo-readme had recent pushes" com um botão Compare & pull request.
2. Se não aparecer, vá até a aba Pull Requests e clique em New pull request.
3. Certifique-se de que:

• base repository: é o original (ex: original/nome-do-projeto) e a branch base (geralmente main ou develop).

• head repository: é o seu fork e a branch fix/typo-readme.

4. O GitHub mostrará um diff das alterações. Confira.
5. Preencha o título e a descrição do PR (veja seção 16.7.3).
6. Clique em Create pull request.

16.7.3 Escrevendo uma boa descrição

Siga o template do projeto (se houver). Caso contrário, use:

```
## Descrição
Corrige um erro de digitação no README (issue #123).

## Tipo de mudança
- [x] Correção de documentação

## Checklist
- [x] Li as diretrizes de contribuição.
- [x] Testei localmente.
- [x] Atualizei a documentação (se aplicável).

Inclua Closes #123 na descrição para que a issue seja automaticamente fechada
quando o PR for mesclado.

16.7.4 Exercício prático 16.5

Abra o Pull Request para a issue que você está trabalhando. Certifique-se de
preencher a descrição adequadamente e de referenciar a issue.

---

16.8 Acompanhando a Revisão e Interagindo com a Comunidade

16.8.1 O que esperar após abrir o PR

· Mantenedores ou outros contribuidores podem comentar, pedir alterações ou
aprovar.
· O GitHub pode rodar automaticamente testes (CI) – verifique se passaram.
· Seja paciente; nem sempre a resposta é imediata.

16.8.2 Respondendo a comentários
```

```
· Seja educado e agradeça pelo feedback.
· Se pedirem alterações, faça os ajustes na mesma branch, commit e push. O PR
será atualizado automaticamente.
· Se discordar de algo, explique seu ponto de vista de forma construtiva.
· Marque os comentários como resolvidos após implementar as sugestões.

16.8.3 Fazendo alterações solicitadas
```

git switch fix/typo-readme

faça as alterações

git add .

git commit -m "fix: ajusta conforme revisão"

git push origin fix/typo-readme

16.8.4 Lidando com conflitos

Se, enquanto você trabalhava, a branch principal do projeto avançou, pode haver conflitos. O GitHub avisará. Para resolver:

```
git switch main
git pull upstream main
git switch fix/typo-readme
git merge main # ou git rebase main
# resolva conflitos, se houver
git add .
git commit -m "merge: resolve conflitos com main"
git push origin fix/typo-readme

16.8.5 Exercício prático 16.6

Simule um cenário de revisão: peça a um colega para comentar no seu PR (ou
faça você mesmo um comentário). Responda ao comentário e faça um novo commit
com ajustes fictícios. Observe como o PR é atualizado.

---

16.9 Quando o PR é Aprovado e Mesclado

16.9.1 O momento da aprovação

Um mantenedor dará Approve e, em seguida, fará o merge (ou você mesmo, se
tiver permissão). O GitHub pode oferecer opções: merge commit, squash and
merge, rebase and merge.

16.9.2 Após o merge

· A issue referenciada será fechada automaticamente (se você usou Closes
#123).
```

```
· Seu commit fará parte do projeto oficial!
· Você pode deletar a branch do seu fork (opcional).

16.9.3 Sincronizando seu fork

Após o merge, atualize seu fork para refletir o estado mais recente do
projeto:
```

git switch main

git pull upstream main

git push origin main

Agora sua main está igual à do projeto original.

16.9.4 Comemore!

Você contribuiu com sucesso para um projeto open source. Adicione essa conquista ao seu currículo e portfólio.

16.9.5 Exercício prático 16.7

Se seu PR foi aceito, sincronize seu fork. Se ainda não foi aceito, pratique o fluxo completo em um repositório de teste com um amigo.

16.10 Boas Práticas para Contribuições Futuras

16.10.1 Comece pequeno

Não tente resolver um problema enorme no primeiro PR. Contribuições pequenas e consistentes constroem confiança.

16.10.2 Leia o CONTRIBUTING.md

Cada projeto tem suas regras. Respeite-as.

16.10.3 Comunique-se antes de grandes mudanças

Se você planeja uma contribuição significativa, abra uma issue primeiro para discutir com os mantenedores.

16.10.4 Seja paciente e respeitoso

Mantenedores são voluntários e podem levar tempo para responder. Nunca seja rude ou impaciente.

16.10.5 Ajude a revisar PRs de outros

Depois que você se sentir confortável, comece a revisar PRs de outros iniciantes. É uma ótima forma de aprender e contribuir.

16.10.6 Mantenha-se atualizado

Acompanhe o projeto: dê star, watch, participe de discussões. Quanto mais você se envolver, mais natural será contribuir.

16.11 Exemplo Real de Contribuição

Vamos simular um exemplo concreto. Suponha que você queira contribuir com o projeto awesome-list (uma lista curada de recursos incríveis).

1. Issue: falta adicionar um link para um artigo recente sobre Git.
2. Fork e clone.
3. Branch: add-git-article.
4. Alteração: edita o arquivo README.md adicionando o link na seção apropriada.
5. Commit: docs: adiciona artigo sobre Git LFS.
6. Push e PR com descrição clara.
7. Revisão: mantenedor pede para colocar em ordem alfabética.
8. Ajuste: novo commit, push.
9. Aprovação e merge.
10. Sincronização do fork.

16.12 Contribuindo com Código vs. Documentação

16.12.1 Documentação

- Mais fácil para começar.
- Geralmente não exige ambiente de desenvolvimento complexo.
- Grande impacto na usabilidade do projeto.

16.12.2 Código

- Exige compreensão mais profunda.
- Pode exigir testes.
- Maior visibilidade e aprendizado técnico.

Escolha o que se adequa ao seu momento.

16.13 Exercícios de Fixação

1. Quais são os principais benefícios de contribuir para projetos open source?
2. Como você pode encontrar issues adequadas para iniciantes? Cite três plataformas.
3. Por que é importante comentar na issue antes de começar a trabalhar?

4. Descreva o fluxo completo desde o fork até o Pull Request.
5. O que fazer quando um revisor pede alterações no seu PR?
6. Como resolver conflitos que surgem durante a vida de um PR?
7. O que significa Closes #123 na descrição de um PR?
8. Após o merge do seu PR, o que você deve fazer para sincronizar seu fork?
9. Liste três boas práticas para contribuições futuras.
10. (Desafio) Escolha um projeto open source real, faça uma contribuição de documentação (por menor que seja) e complete o ciclo. Compartilhe o link do PR com um colega.

16.14 Resumo do Capítulo

- Contribuir para open source é uma experiência enriquecedora.
- Escolha projetos ativos e acolhedores, com issues para iniciantes.
- Comunique-se antes de começar.
- Siga o fluxo: fork → clone → branch → alteração → commit → push → PR.
- Interaja com respeito durante a revisão.
- Após o merge, sincronize seu fork.
- Comece pequeno, documentação é um ótimo ponto de partida.

Agora você está pronto para se tornar um cidadão ativo na comunidade open source. Parabéns por chegar até aqui!

16.15 Referências

- GitHub Guides: "Contributing to Open Source"
- Open Source Guide: https://opensource.guide
- First Timers Only: https://www.firsttimersonly.com
- "How to Contribute to Open Source" – freeCodeCamp
- Pro Git Book – Capítulo sobre contribuição
- Documentação do GitHub: "Creating a pull request from a fork"

Apêndice A: Glossário Completo de Termos Git/GitHub

Páginas estimadas: 12

Objetivo: Consulta rápida de todos os termos técnicos utilizados no livro, com definições claras e objetivas.

A.1 Termos Gerais

Branch (Ramificação)

: Linha independente de desenvolvimento. Permite trabalhar em funcionalidades isoladas sem afetar a branch principal. Criar uma branch é instantâneo, pois é apenas um ponteiro para um commit.

Clone

: Cópia local de um repositório remoto. Inclui todo o histórico, branches e tags. Comando: `git clone <url>`.

Commit

: Snapshot do projeto em um determinado momento. Cada commit possui um hash único, autor, data e mensagem descritiva. É a unidade fundamental do histórico.

Conflito (Conflict)

: Ocorre quando duas branches alteram a mesma parte do mesmo arquivo de maneiras diferentes e o Git não consegue decidir automaticamente qual versão manter. Exige resolução manual.

Detached HEAD

: Estado em que o ponteiro HEAD não está em nenhuma branch, mas sim em um commit específico. Ocorre ao fazer checkout de um commit antigo. Alterações feitas nesse estado podem ser perdidas se não forem salvas em uma nova branch.

Diff

: Visualização das diferenças entre versões de arquivos. Pode comparar working directory, staging area, commits ou branches. Comando: `git diff`.

Fork

: Cópia de um repositório alheio para sua conta no GitHub. Permite modificar o código livremente e depois propor contribuições via Pull Request.

HEAD

: Ponteiro que indica onde você está no momento (qual commit ou branch está ativo). Geralmente aponta para a branch atual.

Issue

: Unidade de trabalho no GitHub, usada para relatar bugs, sugerir melhorias ou tarefas. Pode ser associada a labels, milestones e projetos.

Merge

: Integração de duas branches. Pode ser fast-forward (avanço linear) ou gerar um commit de merge (quando há divergência). Comando: `git merge <branch>`.

Pull Request (PR)

: Solicitação para que um mantenedor incorpore suas alterações (de um fork ou branch) ao repositório principal. Inclui discussão, revisão e, eventualmente, merge.

Rebase

: Reescreve a história aplicando commits de uma branch sobre outra, criando um histórico linear. Útil localmente, mas perigoso em branches compartilhadas.

Reflog

: Registro de todas as movimentações do HEAD no repositório local. Permite recuperar commits "perdidos" após operações destrutivas. Comando: `git reflog`.

Remote (Remoto)

: Repositório hospedado em outro local (ex: GitHub). Nomes comuns: `origin` (principal) e `upstream` (original, em forks).

Repository (Repositório)

: Pasta especial (contém subpasta `.git`) que armazena todos os arquivos, histórico e configurações do projeto versionado.

Staging Area (Index)

: Área intermediária onde os arquivos são preparados antes do commit. Permite escolher quais alterações farão parte do próximo snapshot. Comando: `git add`.

Stash

: Área temporária que guarda alterações não commitadas, permitindo mudar de branch ou limpar o working directory temporariamente. Comando: `git stash`.

Tag

: Marcador estático que aponta para um commit específico, usado geralmente para versões (v1.0, v2.1). Diferente de branch, não se move.

Upstream

: Termo usado para o repositório original em um fluxo de fork. Ex: `git remote add upstream <url-do-original>`.

Working Directory (Árvore de Trabalho)

: Diretório onde você edita os arquivos. É uma das três áreas do Git (junto com staging e repositório).

A.2 Comandos Git (Resumo)

Comando	Descrição
`git init`	Inicializa um novo repositório local
`git clone <url>`	Clona um repositório remoto
`git status`	Mostra o estado dos arquivos (modificados, staged, etc.)
`git add <arquivo>`	Adiciona arquivo ao staging
`git commit -m "msg"`	Cria um commit com a mensagem fornecida
`git log`	Exibe o histórico de commits
`git log --oneline --graph`	Histórico resumido com gráfico
`git diff`	Mostra diferenças não staged
`git diff --staged`	Mostra diferenças entre staging e último commit
`git branch`	Lista branches locais
`git branch <nome>`	Cria nova branch
`git switch <branch>`	Troca para a branch especificada
`git switch -c <branch>`	Cria e troca para nova branch
`git merge <branch>`	Mescla a branch especificada na atual
`git rebase <branch>`	Reaplica commits da branch atual sobre outra
`git rebase -i HEAD~n`	Rebase interativo dos últimos n commits
`git remote add <nome> <url>`	Adiciona um repositório remoto
`git push <remote> <branch>`	Envia commits para o remoto
`git pull <remote> <branch>`	Baixa e integra alterações do remoto
`git fetch <remote>`	Baixa referências do remoto sem integrar
`git stash`	Guarda alterações temporariamente
`git stash pop`	Aplica e remove o stash mais recente

Comando	Descrição
`git stash list`	Lista stashes salvos
`git tag <nome>`	Cria uma tag leve
`git tag -a <nome> -m "msg"`	Cria uma tag anotada
`git restore <arquivo>`	Descarta alterações no working directory
`git restore --staged <arquivo>`	Remove arquivo do staging
`git reset <commit>`	Move a branch para um commit anterior (modos: --soft, --mixed, --hard)
`git revert <commit>`	Cria um novo commit que desfaz as alterações de um commit anterior
`git bisect`	Busca binária para encontrar commit que introduziu bug
`git reflog`	Mostra o histórico de movimentações do HEAD

A.3 Termos GitHub

Actions

: Serviço de CI/CD integrado ao GitHub. Permite automatizar workflows (testes, deploy, etc.) baseados em eventos.

Code Spaces

: Ambiente de desenvolvimento na nuvem, integrado ao GitHub.

Discussions

: Fórum integrado ao repositório para conversas, perguntas e anúncios.

Gist

: Trechos de código ou notas versionadas, podendo ser públicos ou secretos.

GitHub CLI (`gh`)

: Ferramenta de linha de comando para interagir com o GitHub sem sair do terminal.

GitHub Pages

: Serviço de hospedagem de sites estáticos diretamente a partir de um repositório.

Git LFS (Large File Storage)

: Extensão para versionar arquivos grandes (vídeos, imagens, binários) de forma eficiente.

Label

: Rótulo usado para categorizar issues e pull requests (ex: bug, enhancement, good first issue).

Milestone

: Marco que agrupa issues e pull requests com um objetivo comum e data alvo.

Organization

: Conta corporativa que agrupa vários repositórios e membros, com permissões centralizadas.

Project

: Quadro Kanban integrado para planejamento e acompanhamento de tarefas.

Release

: Versão empacotada do projeto, associada a uma tag, com notas de versão e arquivos para download.

Secret

: Variável criptografada usada em Actions para armazenar senhas, tokens, etc.

Sponsors

: Programa de financiamento de desenvolvedores e projetos open source.

Template

: Modelo pré-definido para issues, pull requests ou até repositórios inteiros.

Wiki

: Espaço de documentação versionado, separado do código principal.

A.4 Termos do VS Code para Git

Termo	Descrição
Source Control	Guia do VS Code que integra operações Git
Timeline	Painel que mostra eventos (commits, salvamentos) de um arquivo
GitLens	Extensão que amplia funcionalidades Git: blame inline, explorador, etc.
Git Graph	Extensão que exibe gráfico interativo de commits
GitHub Pull Requests	Extensão oficial para revisar PRs dentro do editor

Termo	Descrição
Inline Blame	Anotação ao lado de cada linha mostrando quem modificou por último

---

A.5 Convenções e Padrões

Conventional Commits

: Padrão para mensagens de commit. Formato: `<tipo>(escopo): <descrição>` (ex: `feat: adiciona botão de login`). Tipos comuns: feat, fix, docs, style, refactor, test, chore.

Git Flow

: Fluxo de trabalho com branches específicas: `main`, `develop`, `feature/*`, `release/*`, `hotfix/*`.

GitHub Flow

: Fluxo simplificado: branch `main` sempre pronta para deploy; branches de feature; Pull Requests; merge.

Semantic Versioning (SemVer)

: Versionamento semântico: `vMAIOR.MENOR.PATCH` (ex: v2.1.3). MAIOR para mudanças incompatíveis, MENOR para novas funcionalidades compatíveis, PATCH para correções.

A.6 Termos Relacionados a Autenticação

HTTPS

: Protocolo de autenticação que exige usuário e token (senha não é mais aceita no terminal do GitHub). Requer token pessoal.

SSH

: Protocolo seguro baseado em chaves pública/privada. Mais prático e seguro. Chave pública adicionada ao GitHub; chave privada fica no computador.

Personal Access Token (PAT)

: Token gerado no GitHub para autenticação via HTTPS. Substitui a senha. Pode ter escopos limitados.

GPG Key

: Chave usada para assinar commits, garantindo autenticidade. Commits assinados mostram selo "Verified" no GitHub.

A.7 Termos de Colaboração

Assignee

: Pessoa responsável por uma issue ou pull request.

Reviewer

: Pessoa designada para revisar um Pull Request.

Code Review

: Processo de análise do código proposto em um PR, com comentários e sugestões.

Merge Conflict

: Ver "Conflito".

Squash

: Ação de combinar múltiplos commits em um só durante o merge de um PR.

Upstream

: Em forks, repositório original. Em outros contextos, repositório de origem.

A.8 Exercícios de Fixação do Glossário

1. Defina, com suas palavras: branch, commit, merge e rebase.
2. Qual a diferença entre `git fetch` e `git pull`?
3. O que é um fork e em que contexto ele é usado?
4. Explique o que significa "detached HEAD" e como sair desse estado.
5. Qual a função do `git stash`? Dê um exemplo prático.
6. O que são GitHub Actions e para que servem?
7. Cite três tipos de labels comuns em issues.
8. O que é um milestone?
9. Qual a diferença entre HTTPS e SSH para autenticação no GitHub?
10. O que significa o selo "Verified" em um commit?

A.9 Referências do Glossário

- Pro Git Book (2ª edição) – Glossário oficial
- GitHub Docs – Glossary
- Conventional Commits – Site oficial
- SemVer.org

Apêndice B: Cheat Sheet de Comandos Git

Páginas estimadas: 8

Objetivo: Consulta rápida com os principais comandos Git organizados por categoria.

B.1 Configuração Inicial

Comando	Descrição
`git config --global user.name "Seu Nome"`	Configura nome do autor
`git config --global user.email "seu@email.com"`	Configura e-mail do autor
`git config --global core.editor "code --wait"`	Define VS Code como editor padrão
`git config --global init.defaultBranch main`	Define branch padrão como main
`git config --global alias.s status`	Cria atalho `git s` para status
`git config --global --list`	Lista todas as configurações
`git config --global credential.helper cache`	Cache de credenciais (15 min padrão)

B.2 Criando e Clonando Repositórios

Comando	Descrição
`git init`	Inicializa repositório local
`git init -b main`	Inicializa com branch main
`git clone <url>`	Clona repositório remoto
`git clone --depth 1 <url>`	Clona apenas o último commit (shallow clone)

B.3 Operações Básicas (Add, Commit, Status)

Comando	Descrição
`git status`	Mostra estado dos arquivos
`git status -s`	Versão resumida
`git add <arquivo>`	Adiciona arquivo ao staging
`git add .`	Adiciona todos os arquivos (cuidado)
`git add -p`	Adiciona interativamente por partes (hunks)
`git commit -m "mensagem"`	Cria commit com mensagem
`git commit -a -m "msg"`	Adiciona e commita em um passo (apenas arquivos rastreados)
`git commit --amend -m "nova msg"`	Altera último commit (mensagem ou inclusão)
`git rm <arquivo>`	Remove arquivo e prepara remoção
`git mv <origem> <destino>`	Move ou renomeia arquivo

B.4 Histórico e Navegação

Comando	Descrição
`git log`	Exibe histórico completo
`git log --oneline`	Histórico resumido (um commit por linha)
`git log --graph --oneline --all`	Gráfico de todas as branches
`git log --author="nome"`	Filtra por autor
`git log --grep="palavra"`	Busca na mensagem
`git log -- <arquivo>`	Histórico de um arquivo específico
`git show <commit>`	Detalhes de um commit
`git show <commit>:<arquivo>`	Conteúdo de um arquivo em um commit específico
`git diff`	Compara working directory com staging
`git diff --staged`	Compara staging com último commit
`git diff <commit1> <commit2>`	Compara dois commits

Comando	Descrição
`git diff <branch1>..<branch2>`	Compara duas branches

B.5 Branches

Comando	Descrição
`git branch`	Lista branches locais
`git branch -a`	Lista todas (locais e remotas)
`git branch <nome>`	Cria nova branch (sem mudar)
`git switch <branch>`	Troca para branch existente
`git switch -c <branch>`	Cria e troca para nova branch
`git checkout -b <branch>`	Alternativa antiga para criar e trocar
`git branch -m <novo-nome>`	Renomeia branch atual
`git branch -d <branch>`	Deleta branch (se mesclada)
`git branch -D <branch>`	Força deleção (mesmo não mesclada)
`git merge <branch>`	Mescla branch na atual
`git merge --abort`	Aborta merge com conflito
`git rebase <branch>`	Reaplica commits da branch atual sobre outra
`git rebase -i HEAD~n`	Rebase interativo dos últimos n commits
`git rebase --continue`	Continua rebase após resolver conflitos
`git rebase --abort`	Aborta rebase

B.6 Stash (Trabalho Temporário)

Comando	Descrição
`git stash`	Guarda alterações não commitadas
`git stash push -m "mensagem"`	Stash com descrição
`git stash list`	Lista stashes

Comando	Descrição
`git stash apply`	Aplica último stash (mantém na lista)
`git stash pop`	Aplica e remove da lista
`git stash drop`	Remove último stash
`git stash clear`	Remove todos os stashes
`git stash branch <branch>`	Cria branch a partir de stash

B.7 Repositórios Remotos

Comando	Descrição
`git remote -v`	Lista remotos com URLs
`git remote add <nome> <url>`	Adiciona novo remote
`git remote rename <antigo> <novo>`	Renomeia remote
`git remote remove <nome>`	Remove remote
`git push -u <remote> <branch>`	Envia branch e define upstream
`git push`	Envia para upstream configurado
`git push --force`	Força push (cuidado!)
`git push --force-with-lease`	Força push mais seguro
`git push origin --delete <branch>`	Deleta branch remota
`git pull`	Busca e mescla (fetch + merge)
`git pull --rebase`	Busca e faz rebase em vez de merge
`git fetch`	Busca referências sem mesclar
`git fetch --all`	Busca de todos os remotos
`git remote show <remote>`	Informações detalhadas do remote

B.8 Tags

Comando	Descrição
`git tag`	Lista tags
`git tag -a v1.0 -m "mensagem"`	Cria tag anotada
`git tag v1.0`	Cria tag leve
`git show v1.0`	Mostra detalhes da tag
`git push origin v1.0`	Envia tag específica para remoto
`git push origin --tags`	Envia todas as tags
`git tag -d v1.0`	Deleta tag local
`git push origin --delete v1.0`	Deleta tag remota

B.9 Desfazendo e Recuperando

Comando	Descrição
`git restore <arquivo>`	Descarta alterações no working directory
`git restore --staged <arquivo>`	Remove arquivo do staging
`git reset HEAD~1`	Remove último commit (mantém alterações)
`git reset --hard HEAD~1`	Remove último commit e alterações (perigoso)
`git revert <commit>`	Cria novo commit desfazendo o especificado
`git reflog`	Mostra movimentações do HEAD (para recuperar commits perdidos)
`git checkout <hash>`	Vai para commit específico (detached HEAD)
`git checkout <hash> -- <arquivo>`	Restaura arquivo de um commit antigo

B.10 Busca Binária (Bisect)

Comando	Descrição
`git bisect start`	Inicia sessão bisect
`git bisect good <commit>`	Marca commit como bom

Comando	Descrição
`git bisect bad <commit>`	Marca commit como ruim
`git bisect reset`	Finaliza bisect
`git bisect run <script>`	Automatiza com script de teste

B.11 Submódulos

Comando	Descrição
`git submodule add <url> <caminho>`	Adiciona submódulo
`git submodule update --init --recursive`	Inicializa e atualiza submódulos após clone
`git submodule update --remote`	Atualiza submódulos para último commit
`git submodule foreach git pull`	Executa comando em cada submódulo

B.12 Git LFS (Large File Storage)

Comando	Descrição
`git lfs install`	Inicializa LFS no repositório
`git lfs track "*.psd"`	Rastreia arquivos .psd com LFS
`git lfs track`	Lista padrões rastreados
`git lfs ls-files`	Lista arquivos rastreados pelo LFS

B.13 GitHub CLI (gh)

Comando	Descrição
`gh auth login`	Autentica no GitHub
`gh repo create <nome>`	Cria repositório remoto
`gh repo clone <nome>`	Clona repositório

Comando	Descrição
`gh pr create`	Cria Pull Request
`gh pr list`	Lista PRs
`gh pr checkout <numero>`	Checkout de PR localmente
`gh issue list`	Lista issues
`gh release create v1.0`	Cria release

B.14 VS Code Atalhos para Git

Atalho	Ação
`Ctrl+Shift+G`	Abre Source Control
`Ctrl+Enter`	Faz commit (com mensagem)
`Ctrl+Shift+G` seguido de `Ctrl+P`	Abre paleta de comandos Git
Clique na branch na barra inferior	Trocar/criar branch
Ícone de sincronização na barra inferior	Pull + Push
`Ctrl+Shift+V`	Preview Markdown

B.15 Resumo de Fluxo Diário

Apêndice C: Guia de Boas Práticas para Nomes de Branches e Commits

Páginas estimadas: 6

Objetivo: Estabelecer convenções e boas práticas para manter a organização e legibilidade do histórico, facilitando o trabalho em equipe e a revisão de código.

C.1 Por que padronizar?

Em projetos colaborativos, a consistência é fundamental. Quando todos seguem as mesmas convenções para nomear branches e escrever mensagens de commit, o histórico se torna autoexplicativo, a navegação fica mais fácil e a integração com ferramentas automáticas (como geradores de changelog) é simplificada.

C.2 Nomes de Branches

C.2.1 Estrutura recomendada

Uma branch deve ter um nome descritivo, curto e que indique seu propósito. Formatos comuns:

<tipo>/<descrição-curta>

<id-da-issue>-<descrição-curta>

<iniciais-do-autor>/<descrição>

Tipos comuns:

- `feature/` – nova funcionalidade
- `bugfix/` – correção de bug
- `hotfix/` – correção urgente em produção
- `release/` – preparação de release
- `docs/` – alterações na documentação
- `refactor/` – refatoração de código
- `test/` – adição ou correção de testes
- `chore/` – tarefas de manutenção (build, dependências, etc.)

Exemplos:

- `feature/autenticacao-google`

- `bugfix/correcao-login-123`
- `hotfix/critical-security-patch`
- `docs/atualiza-readme`
- `refactor/otimiza-consultas`

C.2.2 Boas práticas

- Use letras minúsculas e hífens para separar palavras (kebab-case).
- Seja conciso, mas claro: `feature/adiciona-botao` em vez de `feature/abc`.
- Inclua o número da issue quando relevante: `bugfix/123-corrige-login`.
- Evite caracteres especiais como `@`, `#`, `$`, `%`.
- Após mesclar a branch, delete-a (local e remotamente) para manter o repositório limpo.

C.2.3 Exemplos de nomes ruins e bons

Ruim	Bom
`minhas-alteracoes`	`feature/novo-cabecalho`
`fix`	`bugfix/123-corrige-erro-validacao`
`nova-branch`	`hotfix/seguranca-sql-injection`
`teste123`	`test/adiciona-tests-unitarios`

C.3 Mensagens de Commit

C.3.1 O padrão Conventional Commits

O formato mais aceito atualmente é o **Conventional Commits** , que estrutura a mensagem em:

<tipo>(escopo opcional): <descrição curta>

[corpo opcional]

[rodapé opcional]

Tipos principais:

- `feat`: nova funcionalidade
- `fix`: correção de bug
- `docs`: documentação
- `style`: formatação (espaços, ponto-e-vírgula, etc.) – não altera código

- `refactor`: refatoração sem mudar comportamento
- `perf`: melhoria de performance
- `test`: adiciona ou corrige testes
- `chore`: tarefas de build, dependências, etc.

Escopo opcional: indica o módulo afetado (ex: `feat(auth): adiciona validação de token`).

Descrição: deve ser curta (até 50 caracteres), em imperativo, começando com letra minúscula e sem ponto final.

Exemplos:

feat: adiciona botão de login na página inicial

fix(api): corrige timeout na requisição de dados

docs: atualiza README com instruções de instalação

style: remove espaços extras no arquivo CSS

refactor: extrai função de validação para módulo separado

test: adiciona testes para o componente de formulário

chore: atualiza dependências do projeto

C.3.2 Corpo da mensagem

Quando necessário, o corpo deve explicar o **porquê** da mudança, não o **como** (o código já mostra o como). Use quebras de linha a cada 72 caracteres.

Exemplo:

fix: corrige erro ao salvar usuário sem email

O erro ocorria porque o campo email estava sendo validado

mesmo quando não era obrigatório. Agora a validação só ocorre

se o email for fornecido.

Closes #123

C.3.3 Rodapé

Pode conter referências a issues (`Closes #123`, `Fixes #456`) ou notas de breaking change (`BREAKING CHANGE: a API de login foi modificada`).

C.3.4 Regras de ouro

1. **Primeira linha até 50 caracteres.**

2. **Maiúscula?** Não, a primeira palavra deve ser minúscula (exceto se for um nome próprio).
3. **Ponto final?** Não, a primeira linha não deve ter ponto final.
4. **Imperativo:** "Adiciona" em vez de "Adicionado" ou "Adicionando".
5. **Separe o corpo da primeira linha com uma linha em branco.**

C.3.5 Exemplos de commits ruins e bons

Ruim	Bom
`alteracoes no codigo`	`feat: adiciona validação de email no cadastro`
`corrigi bug`	`fix: resolve erro 500 ao logar com senha inválida`
`atualiza docs`	`docs: adiciona exemplo de uso da API no README`
`mudancas`	`refactor: simplifica lógica do loop de iteração`

C.3.6 Commit atômico

Cada commit deve representar uma **única mudança lógica**. Evite misturar correções de bug com novas funcionalidades no mesmo commit. Isso facilita o entendimento, o revert e a revisão.

C.4 Convenções para Pull Requests

C.4.1 Título do PR

Siga o mesmo padrão dos commits: um título claro que resuma a contribuição.

C.4.2 Descrição do PR

Use um template, se disponível. Inclua:

- **Motivação:** por que a mudança é necessária?
- **O que foi feito:** resumo das alterações.
- **Como testar:** passos para validar.
- **Checklist:** confirmação de que as diretrizes foram seguidas.
- **Issues relacionadas:** `Closes #123`, `Relacionado a #456`.

C.4.3 Exemplo